HTML Basics by SKILLHUB - Day 1

✅ 1. What is HTML?

English:
HTML stands for HyperText Markup Language. It's used to structure content on the web.

Hindi:
HTML का मतलब है हाइपरटेक्स्ट मार्कअप लैंग्वेज। इसका उपयोग वेब पर कंटेंट की संरचना के लिए किया जाता है।

Marathi:
HTML म्हणजे हायपरटेक्स्ट मार्कअप लँग्वेज. हे वेबवरील सामग्री संरचित करण्यासाठी वापरले जाते.

Analogy:
Think of HTML as the framework of a building—it gives shape and structure, but no design.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML

✅ 2. What are tags in HTML?

English:
Tags are HTML keywords enclosed in angle brackets (e.g., `<p>`, `<div>`) used to define elements.

Hindi:
टैग HTML कीवर्ड होते हैं जो कोणीय ब्रैकेट्स में लिखे जाते हैं, जैसे `<p>`, `<div>`।

Marathi:
HTML टॅग्ज हे अँगल ब्रॅकेटमध्ये लिहिलेले कीवर्ड असतात, जसे `<p>`, `<div>`.

Analogy:
Like labeling drawers—each tag shows what content goes inside.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML/Element

✅ 3. What is the difference between HTML tags and elements?

SKILLHUB

English:
A tag is just the markup like `<p>`. An element includes the tag, its content, and the closing tag: `<p>`Hello</p>`.

Hindi:
टैग सिर्फ मार्कअप होता है जैसे `<p>`। एलिमेंट टैग, उसकी कंटेंट, और क्लोजिंग टैग को इनक्लूड करता है: `<p>`Hello</p>`।

Marathi:
टॅग फक्त मार्कअप असतो जसे `<p>`. एलिमेंट टॅग, त्याची कंटेंट आणि क्लोजिंग टॅग समाविष्ट करतो: `<p>`Hello</p>`।

Analogy:
Like the difference between a label and a labeled box—the element includes everything.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML/Element

✅ 4. What are semantic HTML elements?

English:
Semantic elements clearly describe their meaning to both browser and developer (e.g., `<header>`, `<nav>`, `<article>`, `<section>`).

Hindi:
सेमैंटिक एलिमेंट्स अपने मतलब को browser और developer दोनों के लिए क्लियरली डिस्क्राइब करते हैं (जैसे `<header>`, `<nav>`, `<article>`, `<section>`)।

Marathi:
सेमैंटिक एलिमेंट्स त्यांचे अर्थ browser आणि developer दोघांसाठी स्पष्टपणे सांगतात (जसे `<header>`, `<nav>`, `<article>`, `<section>`)।

Analogy:
Like using descriptive labels instead of generic ones—makes content meaningful.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML/Element

✅ 5. What are HTML attributes?

English:
Attributes provide additional information about HTML elements (e.g., `class`, `id`, `src`, `href`).

SKILLHUB

Hindi:
एट्रिब्यूट्स HTML एलिमेंट्स के बारे में एक्स्ट्रा जानकारी प्रदान करते हैं (जैसे `class`, `id`, `src`, `href`)।

Marathi:
एट्रिब्यूट्स HTML एलिमेंट्स बद्दल अतिरिक्त माहिती पुरवतात (जसे `class`, `id`, `src`, `href`)।

Analogy:
Like additional details on a name tag—provides more context about the element.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML/Attributes

---

SKILLHUB

CSS Essentials by SKILLHUB - Day 2

✅ 6. What is CSS?

English:
CSS (Cascading Style Sheets) is used to style and layout web pages, including colors, fonts, and positioning.

Hindi:
CSS (कैस्केडिंग स्टाइल शीट्स) वेब पेजेस को स्टाइल और लेआउट करने के लिए इस्तेमाल किया जाता है, जिसमें कलर्स, फॉन्ट्स, और पोजिशनिंग शामिल है।

Marathi:
CSS (कैस्केडिंग स्टाइल शीट्स) वेब पेजेस ला स्टाइल आणि लेआउट करण्यासाठी वापरले जाते, ज्यात कलर्स, फॉन्ट्स आणि पोजिशनिंग समाविष्ट आहे.

Analogy:
Like clothing for HTML—CSS dresses up the plain structure with colors and styles.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/CSS

✅ 7. What are the ways to apply CSS to HTML?

English:
Inline styles (style attribute), internal styles (`<style>` tag), external stylesheets (`<link>` tag).

Hindi:
इनलाइन स्टाइल्स (style एट्रिब्यूट), इंटरनल स्टाइल्स (`<style>` टैग), एक्सटर्नल स्टाइलशीट्स (`<link>` टैग)।

Marathi:
इनलाइन स्टाइल्स (style एट्रिब्यूट), इंटरनल स्टाइल्स (`<style>` टैग), एक्सटर्नल स्टाइलशीट्स (`<link>` टैग)।

Analogy:
Like different ways to paint a house: directly on walls (inline), in a can (internal), or from a factory (external).

Learn more:

- https://developer.mozilla.org/en-US/docs/Learn/CSS/First_steps/How_CSS_is_structured

---

SKILLHUB

✅ 8. What is CSS specificity?

English:
Specificity determines which CSS rule applies when multiple rules target the same element. ID selectors > class selectors > element selectors.

Hindi:
स्पेसिफिसिटी तय करती है कि जब मल्टीपल रूल्स एक ही एलिमेंट को टारगेट करते हैं तो कौन सा CSS रूल अप्लाई होता है। ID सेलेक्टर्स > क्लास सेलेक्टर्स > एलिमेंट सेलेक्टर्स।

Marathi:
स्पेसिफिसिटी ठरवते की जेव्हा मल्टीपल रूल्स एकाच एलिमेंट ला टारगेट करतात तेव्हा कोणता CSS रूल अप्लाई होतो. ID सेलेक्टर्स > क्लास सेलेक्टर्स > एलिमेंट सेलेक्टर्स।

Analogy:
Like authority levels: the most specific rule (like a direct order) overrides general instructions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/CSS/Specificity

---

✅ 9. What is the CSS box model?

English:
Every element is a box with content, padding, border, and margin areas.

Hindi:
हर एलिमेंट एक बॉक्स है जिसमें कंटेंट, पैडिंग, बॉर्डर, और मार्जिन एरियाज़ होते हैं।

Marathi:
प्रत्येक एलिमेंट एक बॉक्स आहे ज्यात कंटेंट, पैडिंग, बॉर्डर आणि मार्जिन एरियाज असतात.

Analogy:
Like a gift in a box: content is the gift, padding is bubble wrap, border is the box, margin is space around the box.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Box_Model/Introduction_to_the_CSS_box_model

---

✅ 10. What are CSS flexbox and grid?

SKILLHUB

English:
Flexbox: one-dimensional layout for arranging items in rows or columns. Grid: two-dimensional layout for complex layouts.

Hindi:
फ्लेक्सबॉक्स: आइटम्स को रोस या कॉलम्स में अरेंज करने के लिए one-dimensional लेआउट। ग्रिड: complex लेआउट्स के लिए two-dimensional लेआउट।

Marathi:
फ्लेक्सबॉक्स: आइटम्स ला रोस किंवा कॉलम्स मध्ये अरेंज करण्यासाठी one-dimensional लेआउट. ग्रिड: complex लेआउट्स साठी two-dimensional लेआउट.

Analogy:
Flexbox is like arranging books on a single shelf, grid is like organizing books on multiple shelves in a bookcase.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flexible_Box_Layout/Basic_Concepts_of_Flexbox
- https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout

---

✅ 11. What are media queries in CSS?

English:
Media queries apply different styles based on device characteristics like screen size, orientation, resolution.

Hindi:
मीडिया क्वेरीज़ डिवाइस कैरेक्टेरिस्टिक्स जैसे स्क्रीन साइज़, ओरिएंटेशन, रेजोल्यूशन के आधार पर different स्टाइल्स अप्लाई करते हैं।

Marathi:
मीडिया क्वेरीज़ डिवाइस कैरेक्टेरिस्टिक्स जसे स्क्रीन साइज़, ओरिएंटेशन, रेजोल्यूशन वर आधारित different स्टाइल्स अप्लाई करतात.

Analogy:
Like choosing different outfits for different occasions—media queries select appropriate styles for different devices.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/CSS/Media_Queries

---

SKILLHUB

✅ 12. What is CSS-in-JS?

English:
CSS-in-JS is a pattern where CSS is written in JavaScript, allowing dynamic styling and scoped styles.

Hindi:
CSS-in-JS एक पैटर्न है जहां CSS जावास्क्रिप्ट में लिखा जाता है, जो dynamic styling और scoped styles की अनुमति देता है।

Marathi:
CSS-in-JS एक पैटर्न आहे जिथे CSS जावास्क्रिप्ट मध्ये लिहिले जाते, जे dynamic styling आणि scoped styles ची परवानगी देते.

Analogy:
Like mixing paint inside the room instead of preparing it separately—styles are created right where they're used.

Learn more:

- https://styled-components.com/docs/basics

SKILLHUB

JavaScript Fundamentals by SKILLHUB - Day 3

✅ 13. What is JavaScript?

English:
JavaScript is a high-level, interpreted programming language that enables interactive web pages. It runs in web browsers and can manipulate HTML/CSS content dynamically. Created by Brendan Eich in 1995, it's now an essential part of web development alongside HTML and CSS.

Hindi:
जावास्क्रिप्ट एक उच्च-स्तरीय, व्याख्या की गई प्रोग्रामिंग भाषा है जो इंटरैक्टिव वेब पेज सक्षम करती है। यह वेब ब्राउज़र में चलती है और HTML/CSS कंटेंट को डायनामिक रूप से मैनिपुलेट कर सकती है। 1995 में ब्रेंडन आइच द्वारा बनाई गई, यह अब HTML और CSS के साथ वेब डेवलपमेंट का एक आवश्यक हिस्सा है।

Marathi:
जावास्क्रिप्ट ही उच्च-स्तरीय, भाषांतरित प्रोग्रामिंग भाषा आहे जी परस्परसंवादी वेब पेज सक्षम करते. ती वेब ब्राउझरमध्ये चालते आणि HTML/CSS सामग्रीला डायनॅमिकपणे बदलू शकते. 1995 मध्ये ब्रेंडन आइच यांनी तयार केलेली, ती आता HTML आणि CSS सोबत वेब डेव्हलपमेंटचा एक आवश्यक भाग आहे.

Analogy:
Think of JavaScript as the chef in a restaurant—HTML provides the ingredients (structure), CSS styles the presentation (appearance), but JavaScript adds the magic by making everything interactive, like responding to customer orders dynamically.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript
- https://javascript.info/intro

✅ 14. What is the difference between null and undefined in JavaScript?

English:
null is an intentional absence of value (programmer sets it), while undefined means a variable has been declared but not assigned a value. null is a primitive value representing "nothing", undefined indicates "not defined". Both are falsy, but null === undefined is false due to different types.

Hindi:
null एक जानबूझकर मूल्य की अनुपस्थिति है (प्रोग्रामर द्वारा सेट किया जाता है), जबकि undefined का मतलब है कि एक वेरिएबल घोषित किया गया है लेकिन उसे कोई मूल्य असाइन नहीं किया गया है। null एक प्रिमिटिव मूल्य है जो "कुछ

SKILLHUB

नहीं" का प्रतिनिधित्व करता है, undefined इंगित करता है कि "परिभाषित नहीं"। दोनों falsy हैं, लेकिन null === undefined false है क्योंकि अलग-अलग प्रकार हैं।

Marathi:
null ही हेतुपुरस्सर मूल्याची अनुपस्थिती आहे (प्रोग्रामरने सेट केलेली), तर undefined म्हणजे व्हेरिएबल घोषित केला आहे पण त्याला मूल्य दिलेले नाही. null हे "काही नाही" दर्शवणारे प्रिमिटिव्ह मूल्य आहे, undefined सूचवते की "परिभाषित नाही". दोन्ही falsy आहेत, पण null === undefined false आहे कारण वेगवेगळे प्रकार आहेत.

Analogy:
Like booking a hotel room: null means you intentionally canceled the reservation (set it to nothing), undefined means you never made a reservation at all (forgot to define it).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/null
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/undefined

---

✅ 15. What is the difference between == and === in JavaScript?

English:
== (loose equality) performs type coercion before comparison, converting values to the same type. === (strict equality) compares both value and type without conversion. Always use === to avoid unexpected type coercion bugs. == can give surprising results like 0 == false being true.

Hindi:
== (लूज इक्वॉलिटी) तुलना से पहले टाइप कोर्सन करता है, मूल्यों को एक ही प्रकार में कन्वर्ट करता है। === (स्ट्रिक्ट इक्वॉलिटी) बिना कन्वर्जन के मूल्य और प्रकार दोनों की तुलना करता है। अप्रत्याशित टाइप कोर्सन बग्स से बचने के लिए हमेशा === का इस्तेमाल करें। == अजीब रिजल्ट दे सकता है जैसे 0 == false true होना।

Marathi:
== (लूज समानता) तुलना करण्यापूर्वी प्रकार रूपांतरण करते, मूल्ये समान प्रकारात रूपांतरित करते. === (कडक समानता) रूपांतरण न करता मूल्य आणि प्रकार दोन्हीची तुलना करते. अनपेक्षित प्रकार रूपांतरण बग्स टाळण्यासाठी नेहमी === वापरा. == अशा आश्चर्यकारक परिणाम देऊ शकते जसे 0 == false खरे असणे.

Analogy:
Like checking if two people are the same: == is like asking "do they look similar after changing clothes?" (type coercion), === is like asking "are they exactly the same person with same clothes?" (strict comparison).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Equality_comparisons_and_sameness

---

SKILLHUB

✅ 16. What is the difference between let, const, and var in JavaScript?

English:
var is function-scoped and hoisted, let and const are block-scoped. var allows redeclaration, let doesn't. const creates immutable bindings (can't reassign), let allows reassignment. Use const by default, let when reassignment needed, avoid var in modern code.

Hindi:
var फंक्शन-स्कोप्ड है और होइस्टेड, let और const ब्लॉक-स्कोप्ड हैं। var पुनः घोषणा की अनुमति देता है, let नहीं देता। const अपरिवर्तनीय बाइंडिंग बनाता है (री-असाइन नहीं कर सकता), let री-असाइनमेंट की अनुमति देता है। डिफॉल्ट रूप से const का इस्तेमाल करें, जब री-असाइनमेंट की जरूरत हो तब let, मॉडर्न कोड में var से बचें।

Marathi:
var फंक्शन-स्कोप्ड आहे आणि होइस्टेड, let आणि const ब्लॉक-स्कोप्ड आहेत. var पुन्हा घोषणा करण्यास परवानगी देतो, let देत नाही. const अपरिवर्तनीय बाइंडिंग तयार करते (पुन्हा नियुक्त करू शकत नाही), let पुन्हा नियुक्त करण्यास परवानगी देतो. डिफॉल्ट म्हणून const वापरा, पुन्हा नियुक्त करण्याची गरज असल्यावर let, आधुनिक कोडमध्ये var टाळा.

Analogy:
Like containers: var is a big bucket (function scope), let is a smaller box (block scope), const is a locked safe (can't change contents).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/var
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/const

---

✅ 17. What is hoisting in JavaScript?

English:
Hoisting moves variable and function declarations to the top of their scope during compilation. var variables are hoisted with undefined value, let/const are hoisted but not initialized (temporal dead zone). Function declarations are fully hoisted, but function expressions are not.

Hindi:
होइस्टिंग वेरिएबल और फंक्शन डिक्लेरेशन्स को उनके स्कोप के टॉप पर कंपाइलेशन के दौरान मूव करता है। var वेरिएबल्स undefined मूल्य के साथ होइस्ट होते हैं, let/const होइस्ट होते हैं लेकिन इनिशियलाइज़ नहीं होते (टेम्पोरल डेड ज़ोन)। फंक्शन डिक्लेरेशन्स पूरी तरह से होइस्ट होते हैं, लेकिन फंक्शन एक्सप्रेशन्स नहीं।

Marathi:
होइस्टिंग व्हेरिएबल आणि फंक्शन घोषणा त्यांच्या स्कोपच्या शीर्षस्थानी संकलनादरम्यान हलवते. var व्हेरिएबल्स

SKILLHUB

undefined मूल्यासह होइस्ट होतात, let/const होइस्ट होतात पण इनिशियलाइझ होत नाहीत (टेम्पोरल डेड झोन). फंक्शन घोषणा पूर्णपणे होइस्ट होतात, पण फंक्शन एक्सप्रेशन्स होत नाहीत.

Analogy:
Like moving all the ingredients to the kitchen counter before cooking—you can access them, but some might not be prepared yet (undefined).

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Hoisting
- https://javascript.info/var

✅ 18. How do you convert a string to a number in JavaScript?

English:
JavaScript provides several methods: Number() constructor converts to number or NaN, parseInt() parses integer from string (ignores decimals), parseFloat() parses floating-point numbers, unary + operator converts string to number. Choose based on your specific needs and error handling requirements.

Hindi:
जावास्क्रिप्ट कई तरीके प्रदान करता है: Number() कंस्ट्रक्टर संख्या या NaN में कन्वर्ट करता है, parseInt() स्ट्रिंग से इंटीजर पार्स करता है (डेसिमल्स को इग्नोर करता है), parseFloat() फ्लोटिंग-पॉइंट नंबर्स पार्स करता है, unary + ऑपरेटर स्ट्रिंग को संख्या में कन्वर्ट करता है। अपनी विशिष्ट जरूरतों और एरर हैंडलिंग आवश्यकताओं के आधार पर चुनें।

Marathi:
JavaScript provides several methods: Number() constructor converts to number or NaN, parseInt() parses integer from string (ignores decimals), parseFloat() parses floating-point numbers, unary + operator converts string to number.

Analogy:
Like different kitchen tools: Number() is a universal converter, parseInt() is a chopper (removes decimals), parseFloat() handles fractions, unary + is quick conversion.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/parseInt

---

✅ 19. What is the purpose of the array slice method?

SKILLHUB

English:
The slice() method returns a shallow copy of a portion of an array into a new array object. It takes two optional parameters: start index (inclusive) and end index (exclusive). slice() does not modify the original array, making it safe for creating copies or extracting subarrays.

Hindi:
slice() मेथड ऐरे का एक हिस्सा लेकर नए ऐरे ऑब्जेक्ट में shallow copy रिटर्न करता है। यह दो ऑप्शनल पैरामीटर्स लेता है: start index (inclusive) और end index (exclusive)। slice() ओरिजिनल ऐरे को मोडिफाई नहीं करता, जिससे यह कॉपी बनाने या सब-ऐरे एक्सट्रेक्ट करने के लिए safe है।

Marathi:
slice() method array चा एक भाग घेऊन नवीन array object मध्ये shallow copy परत करते. हे दोन optional parameters घेते: start index (inclusive) आणि end index (exclusive). slice() मूळ array ला बदलत नाही, ज्यामुळे हे copy तयार करण्यासाठी किंवा subarrays काढण्यासाठी सुरक्षित आहे.

Analogy:
Like cutting a cake: slice() gives you pieces without changing the original cake, perfect for sharing portions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice

---

✅ 20. What is the purpose of the array splice method?

English:
The splice() method changes the contents of an array by removing or replacing existing elements and/or adding new elements in place. It takes parameters: start index, delete count, and optional items to add. splice() modifies the original array and returns the deleted elements.

Hindi:
splice() मेथड ऐरे की कंटेंट को existing elements को रिमूव या रिप्लेस करके और/या नए elements को ऐड करके चेंज करता है। यह पैरामीटर्स लेता है: start index, delete count, और ऑप्शनल items to add। splice() ओरिजिनल ऐरे को मोडिफाई करता है और डिलीट किए गए elements को रिटर्न करता है।

Marathi:
splice() method array ची content बदलते existing elements काढून किंवा बदलून आणि/किंवा नवीन elements जोडून. हे parameters घेते: start index, delete count, आणि optional items to add. splice() मूळ array ला बदलते आणि deleted elements परत करते.

Analogy:
Like editing a document: splice() removes words and inserts new ones directly in the original text.

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/splice

---

✅ 21. What is the difference between slice and splice?

English:
slice() creates a new array copy without modifying original, splice() modifies the original array. slice() takes start/end indices, splice() takes start index, delete count, and optional additions. slice() is non-destructive, splice() is destructive.

Hindi:
slice() ओरिजिनल को मोडिफाई किए बिना नया ऐरे कॉपी बनाता है, splice() ओरिजिनल ऐरे को मोडिफाई करता है। slice() start/end indices लेता है, splice() start index, delete count, और ऑप्शनल additions लेता है। slice() non-destructive है, splice() destructive है।

Marathi:
slice() मूळ न बदलता नवीन array copy तयार करते, splice() मूळ array ला बदलते. slice() start/end indices घेते, splice() start index, delete count आणि optional additions घेते. slice() non-destructive आहे, splice() destructive आहे.

Analogy:
slice() is photocopying pages (original stays), splice() is cutting and pasting in the original book.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/splice

---

✅ 22. What is a higher order function?

English:
A higher-order function is a function that takes one or more functions as arguments, returns a function as its result, or both. Examples include map(), filter(), reduce(), forEach(). They enable functional programming patterns and code reusability.

Hindi:
हायर-ऑर्डर फंक्शन एक ऐसा फंक्शन है जो एक या अधिक फंक्शन्स को आर्गुमेंट्स के रूप में लेता है, फंक्शन को अपने रिजल्ट के रूप में रिटर्न करता है, या दोनों। उदाहरणों में map(), filter(), reduce(), forEach() शामिल हैं। वे फंक्शनल प्रोग्रामिंग पैटर्न और कोड रियूसेबिलिटी को enable करते हैं।

Marathi:
higher-order function एक असा function आहे जो एक किंवा अधिक functions ला arguments म्हणून घेते,

SKILLHUB

function ला त्याच्या result म्हणून परत करते, किंवा दोन्ही. उदाहरणांमध्ये map(), filter(), reduce(), forEach() समाविष्ट आहेत. ते functional programming patterns आणि code reusability सक्षम करतात.

Analogy:
Like a chef who uses tools (functions) to prepare ingredients (data)—higher-order functions orchestrate other functions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Higher-order_function
- https://javascript.info/array-methods

SKILLHUB

JavaScript Functions & Scope by SKILLHUB - Day 4

✅ 23. What are arrow function?

English:
Arrow functions are a concise syntax for writing function expressions using => operator. They don't have their own this, arguments, super, or new.target bindings. Arrow functions cannot be used as constructors and are best for non-method functions.

Hindi:
एरो फंक्शन्स => ऑपरेटर का इस्तेमाल करके फंक्शन एक्सप्रेशन्स लिखने के लिए concise syntax हैं। उनके पास अपना this, arguments, super, या new.target bindings नहीं होता। एरो फंक्शन्स कंस्ट्रक्टर्स के रूप में इस्तेमाल नहीं किए जा सकते और non-method functions के लिए best हैं।

Marathi:
Arrow functions concise syntax आहे function expressions लिहिण्यासाठी => operator वापरून. त्यांना स्वतःचे this, arguments, super किंवा new.target bindings नसतात. Arrow functions constructors म्हणून वापरल्या जाऊ शकत नाहीत आणि non-method functions साठी सर्वोत्तम आहेत.

Analogy:
Like shorthand notes: arrow functions are quick, compact way to write functions, but lack some features of full functions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

✅ 24. What are first class function?

English:
First-class functions are functions that can be treated like any other variable: assigned to variables, passed as arguments, returned from other functions, stored in data structures. JavaScript functions are first-class citizens.

Hindi:
फर्स्ट-क्लास फंक्शन्स वे फंक्शन्स हैं जो किसी अन्य वेरिएबल की तरह ट्रीट किए जा सकते हैं: वेरिएबल्स को असाइन किए जा सकते हैं, आर्गुमेंट्स के रूप में पास किए जा सकते हैं, अन्य फंक्शन्स से रिटर्न किए जा सकते हैं, डेटा स्ट्रक्चर्स में स्टोर किए जा सकते हैं। जावास्क्रिप्ट फंक्शन्स फर्स्ट-क्लास citizens हैं।

SKILLHUB

Marathi:
first-class functions असे functions आहेत जे कोणत्याही अन्य variable प्रमाणे वागवले जाऊ शकतात: variables ला assign केले जाऊ शकतात, arguments म्हणून पास केले जाऊ शकतात, इतर functions मधून परत केले जाऊ शकतात, data structures मध्ये साठवले जाऊ शकतात. JavaScript functions first-class citizens आहेत.

Analogy:
Like VIP citizens: functions get all the privileges of regular variables—they can go anywhere and do anything.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/First-class_Function

---

✅ 25. What is a pure function?

English:
A pure function always returns the same output for the same input and has no side effects (doesn't modify external state, make API calls, or cause mutations). Pure functions are predictable, testable, and enable optimizations like memoization.

Hindi:
प्योर फंक्शन हमेशा same input के लिए same output रिटर्न करता है और कोई side effects नहीं होते (external state को मोडिफाई नहीं करता, API calls नहीं करता, या mutations का कारण नहीं बनता)। प्योर फंक्शन्स predictable, testable हैं, और memoization जैसे optimizations को enable करते हैं।

Marathi:
pure function नेहमी समान input साठी समान output परत करते आणि कोणतेही side effects नसतात (external state ला बदलत नाही, API calls करत नाही, किंवा mutations चा कारण होत नाही). Pure functions predictable, testable आहेत आणि memoization सारख्या optimizations सक्षम करतात.

Analogy:
Like a perfect calculator: same inputs always give same outputs, no hidden changes or surprises.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Pure_function

---

✅ 26. What is the scope chain in JavaScript?

English:
The scope chain is the hierarchy of scopes that JavaScript uses to resolve variable lookups. When a variable is referenced, JavaScript searches from the innermost scope outward until it finds the variable or reaches global scope. Lexical scoping determines the chain at compile time.

SKILLHUB

Hindi:
स्कोप चेन वह hierarchy of scopes है जिसका इस्तेमाल जावास्क्रिप्ट वेरिएबल lookups को resolve करने के लिए करता है। जब कोई वेरिएबल referenced होता है, तो जावास्क्रिप्ट innermost scope से outward search करता है जब तक वह वेरिएबल नहीं मिल जाता या global scope तक नहीं पहुंच जाता। lexical scoping chain को compile time पर determine करता है।

Marathi:
scope chain ही hierarchy of scopes आहे जिचा वापर JavaScript variable lookups resolve करण्यासाठी करतो. जेव्हा variable referenced केले जाते, तेव्हा JavaScript innermost scope पासून outward search करते जोपर्यंत variable सापडत नाही किंवा global scope पर्यंत पोहोचत नाही. lexical scoping chain ला compile time वर ठरवते.

Analogy:
Like asking for help up the family tree: check with parents, then grandparents, until you find who knows the answer.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Scope
- https://javascript.info/closure

✅ 27. What are modules in javascript?

English:
Modules are reusable pieces of code that can be imported and exported between files. ES6 modules use import/export syntax, CommonJS uses require/module.exports. Modules provide encapsulation, avoid global namespace pollution, and enable better code organization.

Hindi:
मॉड्यूल्स reusable pieces of code हैं जो files के बीच import और export किए जा सकते हैं। ES6 मॉड्यूल्स import/export syntax का इस्तेमाल करते हैं, CommonJS require/module.exports का इस्तेमाल करता है। मॉड्यूल्स encapsulation प्रदान करते हैं, global namespace pollution से बचते हैं, और better code organization को enable करते हैं।

Marathi:
modules ही reusable pieces of code आहेत जी files दरम्यान import आणि export केली जाऊ शकतात. ES6 modules import/export syntax वापरतात, CommonJS require/module.exports वापरते. Modules encapsulation पुरवतात, global namespace pollution टाळतात आणि better code organization सक्षम करतात.

Analogy:
Like LEGO blocks: each module is a self-contained piece that connects with others to build larger structures.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

---

✅ 28. What is a closure in JavaScript?

English:
A closure is a function that has access to variables in its outer (enclosing) scope even after the outer function has returned. Closures are created every time a function is created, at function creation time. They enable data privacy and create function factories.

Hindi:
क्लोजर एक फंक्शन है जिसके पास उसके outer (enclosing) scope में वेरिएबल्स तक access होता है भले ही outer फंक्शन return हो गया हो। क्लोजर्स हर बार बनाए जाते हैं जब कोई फंक्शन बनाया जाता है, फंक्शन creation time पर। वे data privacy को enable करते हैं और function factories बनाते हैं।

Marathi:
closure हा असा function आहे ज्याला त्याच्या outer (enclosing) scope मधील variables पर्यंत प्रवेश आहे जरी outer function परतला असला तरीही. Closures प्रत्येक वेळी तयार होतात जेव्हा function तयार केला जातो, function creation time वर. ते data privacy सक्षम करतात आणि function factories तयार करतात.

Analogy:
Like a backpack: a function carries variables from its birthplace environment wherever it goes.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures

---

✅ 29. What is callback in JavaScript?

English:
A callback is a function passed as an argument to another function, which is then invoked inside the outer function to complete some routine or action. Callbacks enable asynchronous programming and are commonly used with event handlers, timers, and API calls.

Hindi:
कॉलबैक एक फंक्शन है जो दूसरे फंक्शन को आर्गुमेंट के रूप में पास किया जाता है, जिसे फिर outer फंक्शन के अंदर invoke किया जाता है ताकि कुछ routine या action को complete किया जा सके। कॉलबैक्स asynchronous programming को enable करते हैं और event handlers, timers, और API calls के साथ commonly इस्तेमाल किए जाते हैं।

Marathi:
callback हा असा function आहे जो दुसऱ्या function ला argument म्हणून पास केला जातो, जो नंतर outer

SKILLHUB

function च्या आत invoke केला जातो काही routine किंवा action पूर्ण करण्यासाठी. Callbacks asynchronous programming सक्षम करतात आणि event handlers, timers आणि API calls सोबत सामान्यपणे वापरले जातात.

Analogy:
Like leaving a phone number: "call me back when you're done" - the callback function is called when the task completes.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Callback_function

---

✅ 30. What is a callback hell in javascript?

English:
Callback hell (also known as pyramid of doom) occurs when multiple nested callbacks create deeply indented, hard-to-read code. It happens in asynchronous code when each callback contains another callback, leading to unmaintainable code. Promises and async/await solve this problem.

Hindi:
कॉलबैक हेल (pyramid of doom भी कहलाता है) तब होता है जब multiple nested कॉलबैक्स deeply indented, hard-to-read कोड बनाते हैं। यह asynchronous कोड में होता है जब हर कॉलबैक में दूसरा कॉलबैक होता है, जिससे unmaintainable कोड बनता है। promises और async/await इस problem को solve करते हैं।

Marathi:
callback hell (pyramid of doom म्हणूनही ओळखले जाते) जेव्हा अनेक nested callbacks खोल indented, hard-to-read code तयार करतात तेव्हा होते. हे asynchronous code मध्ये होते जेव्हा प्रत्येक callback मध्ये दुसरा callback असतो, ज्यामुळे unmaintainable code तयार होतो. Promises आणि async/await या problem ला सोडवतात.

Analogy:
Like a maze of hallways: nested callbacks create confusing, hard-to-follow paths that are difficult to maintain.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

---

✅ 31. What is memoization?

English:
Memoization is an optimization technique that caches the results of expensive function calls and returns the cached result when the same inputs occur again. It trades memory for performance by storing computation results in a cache object.

SKILLHUB

Hindi:
मेमोइजेशन एक optimization technique है जो expensive फंक्शन calls के रिजल्ट्स को cache करती है और जब same inputs फिर से आते हैं तो cached रिजल्ट रिटर्न करती है। यह memory को performance के लिए trade करती है computation रिजल्ट्स को cache object में स्टोर करके।

Marathi:
memoization ही optimization technique आहे जी expensive function calls चे results cache करते आणि जेव्हा समान inputs पुन्हा येतात तेव्हा cached result परत करते. ती memory ला performance साठी trade करते computation results ला cache object मध्ये साठवून.

Analogy:
Like remembering answers: instead of recalculating expensive operations, memoization remembers previous results.

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Memoization

---

✅ 32. What is the purpose of the "use strict" statement in JavaScript?

English:
"use strict" enables strict mode, which catches common coding mistakes and prevents unsafe actions. It eliminates silent errors, throws errors for variable misuse, prevents accidental global variables, and enforces cleaner code. It's scoped and can be applied globally or per function.

Hindi:
"use strict" strict mode को enable करता है, जो common coding mistakes को catch करता है और unsafe actions को prevent करता है। यह silent errors को eliminate करता है, variable misuse के लिए errors throw करता है, accidental global variables को prevent करता है, और cleaner code को enforce करता है। यह scoped है और globally या per function apply किया जा सकता है।

Marathi:
"use strict" strict mode सक्षम करते, जो common coding mistakes catch करते आणि unsafe actions prevent करते. ते silent errors eliminate करते, variable misuse साठी errors throw करते, accidental global variables prevent करते आणि cleaner code enforce करते. हे scoped आहे आणि globally किंवा per function apply केले जाऊ शकते.

Analogy:
Like strict parenting: enforces rules and catches mistakes early, preventing bigger problems later.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Strict_mode

SKILLHUB

SKILLHUB

JavaScript Advanced Concepts by SKILLHUB - Day 5

✅ 33. What is the difference between synchronous and asynchronous code in JavaScript?

English:
Synchronous code executes line by line, blocking further execution until each operation completes. Asynchronous code allows operations to run in parallel, not blocking the main thread. Async operations use callbacks, promises, or async/await to handle results when they complete.

Hindi:
सिंक्रोनस कोड line by line execute होता है, हर operation complete होने तक further execution को block करता है। असिंक्रोनस कोड operations को parallel में run करने की अनुमति देता है, main thread को block नहीं करता। async operations रिजल्ट्स को हैंडल करने के लिए callbacks, promises, या async/await का इस्तेमाल करते हैं जब वे complete होते हैं।

Marathi:
synchronous code line by line execute होते, प्रत्येक operation complete होईपर्यंत further execution ला block करते. Asynchronous code operations ला parallel मध्ये run करण्यास परवानगी देते, main thread ला block करत नाही. Async operations results handle करण्यासाठी callbacks, promises किंवा async/await वापरतात जेव्हा ते complete होतात.

Analogy:
Synchronous is like waiting in line at a bank (one person at a time), asynchronous is like texting while cooking (multiple things happening simultaneously).

Learn more:

- https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous

---

✅ 34. What are promises in JavaScript?

English:
Promises represent the eventual completion or failure of an asynchronous operation. A promise is in one of three states: pending, fulfilled, or rejected. They provide .then() and .catch() methods for handling success/failure, and are more readable than nested callbacks.

SKILLHUB

Hindi:
प्रॉमिसेज asynchronous operation की eventual completion या failure को represent करते हैं। एक प्रॉमिस तीन states में से एक में होता है: pending, fulfilled, या rejected। वे success/failure को हैंडल करने के लिए .then() और .catch() methods प्रदान करते हैं, और nested callbacks से ज्यादा readable हैं।

Marathi:
promises asynchronous operation ची eventual completion किंवा failure दर्शवतात. Promise तीन states पैकी एकात असते: pending, fulfilled किंवा rejected. ते success/failure handle करण्यासाठी .then() आणि .catch() methods पुरवतात आणि nested callbacks पेक्षा अधिक readable आहेत.

Analogy:
Like ordering food: promise is the receipt (pending), fulfilled is getting your meal, rejected is restaurant being closed.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

---

✅ 35. What is the difference between async/await and promises in JavaScript?

English:
async/await is syntactic sugar built on top of promises, making asynchronous code look synchronous. async functions always return promises, await pauses execution until the promise resolves. Both handle the same asynchronous operations, but async/await is more readable for complex flows.

Hindi:
async/await promises के ऊपर built syntactic sugar है, जो asynchronous कोड को synchronous दिखता है। async functions हमेशा promises रिटर्न करते हैं, await execution को pause करता है जब तक promise resolve नहीं हो जाता। दोनों same asynchronous operations को हैंडल करते हैं, लेकिन async/await complex flows के लिए ज्यादा readable है।

Marathi:
async/await promises वर built syntactic sugar आहे, जे asynchronous code ला synchronous दिसते करते. async functions नेहमी promises परत करतात, await execution ला pause करते जोपर्यंत promise resolve होत नाही. दोन्ही समान asynchronous operations handle करतात, पण async/await complex flows साठी अधिक readable आहे.

Analogy:
Promises are like ordering takeout (you get a promise/receipt), async/await is like eating at the restaurant (waiter handles everything seamlessly).

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Promises

---

✅ 36. How do you handle errors in javascript?

English:
JavaScript error handling uses try/catch blocks for synchronous errors, .catch() method for promises, and try/catch in async functions. throw creates custom errors, Error objects provide stack traces. Global error handlers catch unhandled errors.

Hindi:
जावास्क्रिप्ट error handling synchronous errors के लिए try/catch blocks, promises के लिए .catch() method, और async functions में try/catch का इस्तेमाल करता है। throw custom errors बनाता है, Error objects stack traces प्रदान करते हैं। global error handlers unhandled errors को catch करते हैं।

Marathi:
JavaScript error handling synchronous errors साठी try/catch blocks, promises साठी .catch() method आणि async functions मध्ये try/catch वापरते. throw custom errors तयार करते, Error objects stack traces पुरवतात. global error handlers unhandled errors catch करतात.

Analogy:
Like safety nets: try/catch catches expected errors, global handlers catch unexpected falls.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling

---

✅ 37. What is NaN in javascript?

English:
NaN (Not a Number) is a special value representing invalid number results. It's returned from operations like 0/0, parseInt("abc"), or Math.sqrt(-1). NaN is the only value not equal to itself (NaN !== NaN). Use isNaN() or Number.isNaN() to check for NaN.

Hindi:
NaN (Not a Number) एक special value है जो invalid number results को represent करता है। यह 0/0, parseInt("abc"), या Math.sqrt(-1) जैसे operations से रिटर्न होता है। NaN only value है जो खुद से equal नहीं है (NaN !== NaN)। NaN को check करने के लिए isNaN() या Number.isNaN() का इस्तेमाल करें।

Marathi:
NaN (Not a Number) हा special value आहे जो invalid number results दर्शवतो. तो 0/0, parseInt("abc") किंवा Math.sqrt(-1) सारख्या operations मधून परत येतो. NaN हा एकमेव value आहे जो स्वतःशी समान नाही (NaN !== NaN). NaN check करण्यासाठी isNaN() किंवा Number.isNaN() वापरा.

SKILLHUB

Analogy:
Like a broken calculator: when you try impossible math, you get "Error" - that's NaN for numbers.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/NaN

✅ 38. What is a cookie in javascript?

English:
Cookies are small text files stored by browsers to remember user information. They can store data up to 4KB and have expiration dates. Used for session management, personalization, and tracking. JavaScript can read/write cookies via document.cookie, but has limitations compared to modern storage APIs.

Hindi:
कुकीज़ small text files हैं जो browsers द्वारा user information को remember करने के लिए स्टोर किए जाते हैं। वे 4KB तक डेटा स्टोर कर सकते हैं और expiration dates होते हैं। session management, personalization, और tracking के लिए इस्तेमाल किए जाते हैं। JavaScript document.cookie के माध्यम से cookies को read/write कर सकता है, लेकिन modern storage APIs की तुलना में limitations हैं।

Marathi:
cookies ही छोटी text files आहेत जी browsers द्वारे user information लक्षात ठेवण्यासाठी साठवली जातात. ती 4KB पर्यंत data साठवू शकतात आणि expiration dates असतात. session management, personalization आणि tracking साठी वापरली जातात. JavaScript document.cookie द्वारे cookies वाचू/लिहू शकते, पण modern storage APIs पेक्षा limitations आहेत.

Analogy:
Like a store loyalty card: remembers your preferences and past visits for personalized service.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies

✅ 39. What are the differences between cookie, local storage and session storage?

English:
Cookies: small (4KB), sent with HTTP requests, expiration dates, server-accessible. LocalStorage: larger (5-10MB), client-side only, persists until manually cleared, no expiration. SessionStorage: similar to LocalStorage but clears when tab closes. LocalStorage for long-term, SessionStorage for session data.

SKILLHUB

Hindi:
कुकीज़: small (4KB), HTTP requests के साथ sent होते हैं, expiration dates, server-accessible। LocalStorage: larger (5-10MB), client-side only, manually cleared होने तक persist करता है, no expiration। SessionStorage: LocalStorage से similar लेकिन tab close होने पर clear होता है। LocalStorage long-term के लिए, SessionStorage session data के लिए।

Marathi:
cookies: छोटे (4KB), HTTP requests सोबत पाठवले जातात, expiration dates, server-accessible. LocalStorage: मोठे (5-10MB), client-side only, manually cleared होईपर्यंत persist करते, no expiration. SessionStorage: LocalStorage सारखे पण tab बंद झाल्यावर clear होते. LocalStorage long-term साठी, SessionStorage session data साठी.

Analogy:
Cookie is a business card (small, shared), LocalStorage is a filing cabinet (large, personal), SessionStorage is a desk (temporary workspace).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

✅ 40. What is AJAX?

English:
AJAX (Asynchronous JavaScript and XML) enables updating web pages without full reloads by making background HTTP requests. Uses XMLHttpRequest or Fetch API to send/receive data asynchronously. Despite the name, it commonly uses JSON instead of XML. Foundation of modern single-page applications.

Hindi:
AJAX (Asynchronous JavaScript and XML) full reload के बिना web pages को update करने की अनुमति देता है background HTTP requests बनाकर। XMLHttpRequest या Fetch API का इस्तेमाल asynchronously data send/receive करने के लिए करता है। नाम के बावजूद, यह commonly XML के बजाय JSON इस्तेमाल करता है। modern single-page applications की foundation है।

Marathi:
AJAX (Asynchronous JavaScript and XML) संपूर्ण reload शिवाय web pages update करण्यास परवानगी देतो background HTTP requests तयार करून. XMLHttpRequest किंवा Fetch API वापरून asynchronously data send/receive करण्यासाठी वापरतो. नावानुसार, तो सामान्यपणे XML ऐवजी JSON वापरतो. modern single-page applications ची foundation आहे.

Analogy:
Like texting while watching TV: AJAX works in background to update content without interrupting your viewing experience.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX

✅ 41. How do you find operating system details in javascript?

English:
Use navigator.platform or navigator.userAgent to detect OS. navigator.platform gives OS name, navigator.userAgent provides detailed browser/OS string. Modern approach uses feature detection over OS detection. navigator.oscpu provides CPU/OS info in some browsers.

Hindi:
OS detect करने के लिए navigator.platform या navigator.userAgent का इस्तेमाल करें। navigator.platform OS name देता है, navigator.userAgent detailed browser/OS string प्रदान करता है। modern approach OS detection के बजाय feature detection का इस्तेमाल करता है। navigator.oscpu कुछ browsers में CPU/OS info प्रदान करता है।

Marathi:
OS details शोधण्यासाठी navigator.platform किंवा navigator.userAgent वापरा. navigator.platform OS name देते, navigator.userAgent detailed browser/OS string पुरवते. modern approach OS detection ऐवजी feature detection वापरते. navigator.oscpu काही browsers मध्ये CPU/OS info पुरवते.

Analogy:
Like checking ID at a party: navigator tells you "what kind of device" you're dealing with.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Navigator

✅ 42. What is the Document Object Model (DOM)?

English:
DOM is a programming interface for HTML/XML documents that represents the page as a tree of objects. JavaScript can manipulate DOM to change content, structure, and styling dynamically. Each HTML element becomes a node in the DOM tree with properties and methods for interaction.

Hindi:
DOM HTML/XML documents के लिए programming interface है जो page को objects के tree के रूप में represent करता है। JavaScript DOM को manipulate करके content, structure, और styling को dynamically change कर सकता है। हर HTML element DOM tree में node बन जाता है properties और methods के साथ interaction के लिए।

SKILLHUB

Marathi:
DOM HTML/XML documents साठी programming interface आहे जे page ला objects च्या tree प्रमाणे represent करते. JavaScript DOM manipulate करून content, structure आणि styling dynamically बदलू शकते. प्रत्येक HTML element DOM tree मध्ये node बनते properties आणि methods सोबत interaction साठी.

Analogy:
Like a family tree: DOM shows how all HTML elements are related and connected to each other.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

---

SKILLHUB

JavaScript DOM & Browser APIs by SKILLHUB - Day 6

✅ 43. What is the difference between the DOM and HTML?

English:
HTML is the markup language that structures content, DOM is the live object representation of that HTML in memory. DOM allows JavaScript to dynamically manipulate the page structure, content, and styling. HTML is static markup, DOM is the interactive programming interface.

Hindi:
HTML markup language है जो content को structure करता है, DOM उस HTML का live object representation है memory में। DOM JavaScript को page structure, content, और styling को dynamically manipulate करने की अनुमति देता है। HTML static markup है, DOM interactive programming interface है।

Marathi:
HTML markup language आहे जे content ला structure करते, DOM त्या HTML चे live object representation आहे memory मध्ये. DOM JavaScript ला page structure, content आणि styling dynamically manipulate करण्यास परवानगी देतो. HTML static markup आहे, DOM interactive programming interface आहे.

Analogy:
HTML is the blueprint, DOM is the actual built house you can walk through and modify.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

✅ 44. What is the difference between the DOMContentLoaded event and the load event?

English:
DOMContentLoaded fires when HTML document is loaded and parsed, before stylesheets, images, and subframes finish loading. Load event fires when the entire page (including all resources) has finished loading. Use DOMContentLoaded for faster script execution, load for complete page readiness.

Hindi:
DOMContentLoaded तब fire होता है जब HTML document load और parse हो जाता है, stylesheets, images, और subframes finish loading से पहले। Load event तब fire होता है जब entire page (सभी resources सहित)

SKILLHUB

finish loading हो जाता है। DOMContentLoaded को faster script execution के लिए इस्तेमाल करें, load को complete page readiness के लिए।

Marathi:
DOMContentLoaded जेव्हा HTML document load आणि parse झाला असेल तेव्हा fire होते, stylesheets, images आणि subframes finish loading पूर्वी. Load event जेव्हा संपूर्ण page (सर्व resources समाविष्ट) finish loading झाला असेल तेव्हा fire होते. DOMContentLoaded ला faster script execution साठी वापरा, load ला complete page readiness साठी.

Analogy:
DOMContentLoaded is like house foundation ready (basic structure done), load is like house fully furnished and ready to live in.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Document/DOMContentLoaded_event

---

✅ 45. What is the difference between innerHTML and innerText?

English:
innerHTML gets/sets HTML content including tags, innerText gets/sets plain text content only (ignores HTML tags). innerHTML can execute scripts and is vulnerable to XSS, innerText is safer but slower. textContent is similar to innerText but handles whitespace differently.

Hindi:
innerHTML HTML content को get/set करता है tags सहित, innerText सिर्फ plain text content को get/set करता है (HTML tags को ignore करता है)। innerHTML scripts execute कर सकता है और XSS के लिए vulnerable है, innerText safer है लेकिन slower है। textContent innerText से similar है लेकिन whitespace को differently handle करता है।

Marathi:
innerHTML HTML content get/set करते tags समाविष्ट, innerText फक्त plain text content get/set करते (HTML tags ignore करते). innerHTML scripts execute करू शकते आणि XSS साठी vulnerable आहे, innerText safer आहे पण slower आहे. textContent innerText सारखे आहे पण whitespace ला differently handle करते.

Analogy:
innerHTML is like rich text (with formatting), innerText is like plain text (just the words).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Element/innerHTML
- https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/innerText

SKILLHUB

✅ 46. What is the role of the Window object in the DOM?

English:
Window object represents the browser window/tab and is the global object in browser JavaScript. It contains properties like document, location, history, and methods like alert(), setTimeout(). All global variables and functions become properties of the window object.

Hindi:
Window object browser window/tab को represent करता है और browser JavaScript में global object है। इसमें document, location, history जैसे properties और alert(), setTimeout() जैसे methods होते हैं। सभी global variables और functions window object के properties बन जाते हैं।

Marathi:
Window object browser window/tab दर्शवते आणि browser JavaScript मध्ये global object आहे. त्यात document, location, history सारखी properties आणि alert(), setTimeout() सारखी methods असतात. सर्व global variables आणि functions window object ची properties बनतात.

Analogy:
Window is the browser's control room: it holds all the tools and information about the current browsing session.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Window

SKILLHUB

✅ 47. What is a DOM node in JavaScript?

English:
A DOM node is any object in the DOM tree representing part of the document. Types include Element nodes (HTML tags), Text nodes (content), Comment nodes, Document nodes. All nodes inherit from Node interface with properties like nodeType, nodeName, childNodes, and methods like appendChild(), removeChild().

Hindi:
DOM node DOM tree में कोई भी object है जो document का part represent करता है। types में Element nodes (HTML tags), Text nodes (content), Comment nodes, Document nodes शामिल हैं। सभी nodes Node interface से inherit करते हैं properties जैसे nodeType, nodeName, childNodes, और methods जैसे appendChild(), removeChild() के साथ।

Marathi:
DOM node हा DOM tree मधील कोणताही object आहे जो document चा भाग दर्शवतो. types मध्ये Element nodes (HTML tags), Text nodes (content), Comment nodes, Document nodes समाविष्ट आहेत. सर्व nodes

Node interface पासून inherit करतात properties जसे nodeType, nodeName, childNodes आणि methods जसे appendChild(), removeChild() सोबत.

Analogy:
Like family members in a family tree: each node represents a person (element) with relationships to others.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Node

✅ 48. How do you prevent default behavior of an event in the DOM using JavaScript?

English:
Call event.preventDefault() method on the event object within an event handler. This stops the browser's default action for that event, like preventing form submission, link navigation, or right-click context menus. Must be called before the default action occurs.

Hindi:
event handler के अंदर event object पर event.preventDefault() method को call करें। यह उस event के लिए browser की default action को stop करता है, जैसे form submission, link navigation, या right-click context menus को prevent करना। default action से पहले call किया जाना चाहिए।

Marathi:
event handler च्या आत event object वर event.preventDefault() method call करा. हे त्या event साठी browser ची default action थांबवते, जसे form submission, link navigation किंवा right-click context menus prevent करणे. default action पूर्वी call केले पाहिजे.

Analogy:
Like saying "stop" to a waiter: preventDefault() tells the browser "don't do your usual thing with this action."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault

---

✅ 49. What is event propagation?

English:
Event propagation is how events travel through the DOM tree. Capturing phase: event travels from

SKILLHUB

root to target, Target phase: event reaches target, Bubbling phase: event bubbles from target back to root. Use event.stopPropagation() to prevent further propagation.

Hindi:
event propagation वह तरीका है जिससे events DOM tree के माध्यम से travel करते हैं। Capturing phase: event root से target तक travel करता है, Target phase: event target तक पहुंचता है, Bubbling phase: event target से वापस root तक bubble करता है। further propagation को prevent करने के लिए event.stopPropagation() का इस्तेमाल करें।

Marathi:
event propagation हा मार्ग आहे ज्याद्वारे events DOM tree द्वारे travel करतात. Capturing phase: event root पासून target पर्यंत travel करते, Target phase: event target पर्यंत पोहोचते, Bubbling phase: event target पासून परत root पर्यंत bubble करते. further propagation prevent करण्यासाठी event.stopPropagation() वापरा.

Analogy:
Like ripples in a pond: event starts at one point and spreads outward through all the connected elements.

Learn more:

- https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events

---

✅ 50. What is call stack in javascript?

English:
Call stack is a data structure that keeps track of function calls in JavaScript. When a function is called, it's added to the stack. When it returns, it's removed. Stack overflow occurs when too many functions are called recursively. JavaScript is single-threaded, so only one call stack exists.

Hindi:
call stack एक data structure है जो JavaScript में function calls को track रखता है। जब कोई function called होता है, तो वह stack में add होता है। जब वह return करता है, तो remove हो जाता है। बहुत सारे functions recursively call करने पर stack overflow होता है। JavaScript single-threaded है, इसलिए सिर्फ एक call stack exist करता है।

Marathi:
call stack ही data structure आहे जी JavaScript मध्ये function calls track ठेवते. जेव्हा function called केला जातो, तो stack मध्ये add केला जातो. जेव्हा तो return करतो, तो remove केला जातो. खूप functions recursively call केल्यावर stack overflow होते. JavaScript single-threaded आहे, म्हणून फक्त एक call stack अस्तित्वात आहे.

Analogy:
Like a stack of plates: you add plates (function calls) to the top, and remove from the top when done.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Call_stack

---

✅ 51. What is an event loop?

English:
Event loop is JavaScript's mechanism for handling asynchronous operations. It continuously checks the call stack and callback queue. When stack is empty, it moves callbacks from queue to stack for execution. This enables non-blocking I/O despite JavaScript being single-threaded.

Hindi:
event loop JavaScript की mechanism है asynchronous operations को handle करने के लिए। यह continuously call stack और callback queue को check करता है। जब stack empty होता है, तो यह queue से callbacks को stack में move करके execution के लिए लेता है। यह JavaScript के single-threaded होने के बावजूद non-blocking I/O को enable करता है।

Marathi:
event loop ही JavaScript ची mechanism आहे asynchronous operations handle करण्यासाठी. ती continuously call stack आणि callback queue check करते. जेव्हा stack empty असते, तेव्हा ती queue मधून callbacks ला stack मध्ये move करते execution साठी. हे JavaScript single-threaded असूनही non-blocking I/O सक्षम करते.

Analogy:
Like a restaurant kitchen: chef (call stack) handles orders one by one, but waiters (event loop) continuously check for new orders (callbacks) to bring to the chef.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/EventLoop

---

✅ 52. What is BOM?

English:
BOM (Browser Object Model) provides JavaScript access to browser functionality beyond the document. Includes window, navigator, screen, location, history objects. Unlike DOM (document-focused), BOM handles browser window management, navigation, and device information.

Hindi:
BOM (Browser Object Model) document के अलावा browser functionality तक JavaScript access प्रदान करता है। इसमें window, navigator, screen, location, history objects शामिल हैं। DOM (document-focused) के विपरीत, BOM browser window management, navigation, और device information को handle करता है।

SKILLHUB

Marathi:
BOM (Browser Object Model) document व्यतिरिक्त browser functionality पर्यंत JavaScript प्रवेश पुरवते. त्यात window, navigator, screen, location, history objects समाविष्ट आहेत. DOM (document-focused) च्या विपरीत, BOM browser window management, navigation आणि device information handle करते.

Analogy:
DOM is the house interior, BOM is everything outside: the neighborhood, roads, and utilities around the house.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Window

✅ 53. What is the use of setTimeOut() in javascript?

English:
setTimeout() executes a function or code snippet after a specified delay in milliseconds. It returns a timeout ID for cancellation. Useful for delaying execution, creating animations, or implementing polling. The delay is minimum time - actual execution may be longer if call stack is busy.

Hindi:
setTimeout() एक specified delay (milliseconds में) के बाद function या code snippet को execute करता है। यह cancellation के लिए timeout ID रिटर्न करता है। execution को delay करने, animations बनाने, या polling implement करने के लिए useful है। delay minimum time है - अगर call stack busy है तो actual execution longer हो सकता है।

Marathi:
setTimeout() एका specified delay (milliseconds मध्ये) नंतर function किंवा code snippet execute करते. हे cancellation साठी timeout ID परत करते. execution ला delay करण्यासाठी, animations तयार करण्यासाठी किंवा polling implement करण्यासाठी useful आहे. delay minimum time आहे - जर call stack busy असेल तर actual execution longer होऊ शकते.

Analogy:
Like setting an alarm clock: "wake me up in 5 minutes" - setTimeout() says "run this code after X milliseconds."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/setTimeout

---

✅ 54. What is the use of setInterval() in javascript?

SKILLHUB

English:
setInterval() repeatedly executes a function or code at specified intervals (in milliseconds). It returns an interval ID for stopping. Used for periodic tasks like updating clocks, polling APIs, or creating animations. Unlike setTimeout(), it continues until cleared.

Hindi:
setInterval() specified intervals (milliseconds में) पर repeatedly function या code को execute करता है। यह stopping के लिए interval ID रिटर्न करता है। clocks update करने, APIs को poll करने, या animations बनाने जैसे periodic tasks के लिए इस्तेमाल किया जाता है। setTimeout() के विपरीत, यह cleared होने तक continue करता है।

Marathi:
setInterval() specified intervals (milliseconds मध्ये) वर repeatedly function किंवा code execute करते. हे stopping साठी interval ID परत करते. clocks update करण्यासाठी, APIs poll करण्यासाठी किंवा animations तयार करण्यासाठी सारखे periodic tasks साठी वापरले जाते. setTimeout() च्या विपरीत, ते cleared होईपर्यंत continue करते.

Analogy:
Like a heartbeat monitor: setInterval() checks and updates at regular intervals, continuously.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/setInterval

---

✅ 55. What is the purpose of clearTimeout method and clearInterval?

English:
clearTimeout() stops a timeout set by setTimeout(), clearInterval() stops an interval set by setInterval(). Both take the ID returned by their respective timer functions. Essential for preventing memory leaks and stopping unwanted repeated execution.

Hindi:
clearTimeout() setTimeout() द्वारा set किए गए timeout को stop करता है, clearInterval() setInterval() द्वारा set किए गए interval को stop करता है। दोनों अपने respective timer functions द्वारा रिटर्न किए गए ID को लेते हैं। memory leaks को prevent करने और unwanted repeated execution को stop करने के लिए essential हैं।

Marathi:
clearTimeout() setTimeout() द्वारे set केलेला timeout थांबवते, clearInterval() setInterval() द्वारे set केलेला interval थांबवते. दोन्ही त्यांच्या respective timer functions द्वारे परत केलेला ID घेतात. memory leaks prevent करण्यासाठी आणि unwanted repeated execution थांबवण्यासाठी essential आहेत.

Analogy:
Like alarm clock's snooze button: clearTimeout() cancels a single alarm, clearInterval() stops a repeating alarm.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/clearTimeout
- https://developer.mozilla.org/en-US/docs/Web/API/clearInterval

---

✅ 56. How do you redirect new page in javascript?

English:
Use window.location.href = "url" for immediate redirect, window.location.replace("url") to replace current history entry, or window.location.assign("url") for standard navigation. window.open("url") opens in new window/tab. Choose based on whether you want to keep history.

Hindi:
immediate redirect के लिए window.location.href = "url" का इस्तेमाल करें, current history entry को replace करने के लिए window.location.replace("url"), या standard navigation के लिए window.location.assign("url") I window.open("url") new window/tab में open करता है। history को keep करना चाहते हैं या नहीं इसके आधार पर चुनें।

Marathi:
immediate redirect साठी window.location.href = "url" वापरा, current history entry replace करण्यासाठी window.location.replace("url"), किंवा standard navigation साठी window.location.assign("url"). window.open("url") नवीन window/tab मध्ये उघडते. history ठेवायचे आहे की नाही यावर अवलंबून निवडा.

Analogy:
Like different ways to change rooms: href is teleporting, replace is remodeling the current room, assign is walking through the door.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Location

✅ 57. What is a JavaScript object?

English:
JavaScript object is a collection of key-value pairs where keys are strings/symbols and values can be any type. Created with object literals {}, constructor functions, or Object.create(). Objects can have properties and methods, support inheritance through prototype chain.

Hindi:
JavaScript object key-value pairs का collection है जहां keys strings/symbols होते हैं और values कोई भी type हो सकते हैं। object literals {}, constructor functions, या Object.create() से created किए जाते हैं। objects के properties और methods हो सकते हैं, prototype chain के माध्यम से inheritance support करते हैं।

SKILLHUB

Marathi:
JavaScript object हा key-value pairs चा collection आहे जिथे keys strings/symbols असतात आणि values कोणत्याही प्रकारचे असू शकतात. object literals {}, constructor functions किंवा Object.create() द्वारे created केले जातात. objects ला properties आणि methods असू शकतात, prototype chain द्वारे inheritance support करतात.

Analogy:
Like a person's profile: name, age, hobbies are properties, and actions like "speak" are methods.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object

---

SKILLHUB

JavaScript Objects & Prototypes by SKILLHUB - Day 7

✅ 58. What is the difference between dot notation and bracket notation when accessing properties of an object?

English:
Dot notation (obj.prop) is cleaner but only works with valid identifier names. Bracket notation (obj["prop"]) works with any string, including spaces, hyphens, or computed property names. Use bracket notation when property names are dynamic or contain special characters.

Hindi:
dot notation (obj.prop) cleaner है लेकिन सिर्फ valid identifier names के साथ काम करता है। bracket notation (obj["prop"]) किसी भी string के साथ काम करता है, spaces, hyphens, या computed property names सहित। जब property names dynamic हैं या special characters contain करते हैं तब bracket notation का इस्तेमाल करें।

Marathi:
dot notation (obj.prop) cleaner आहे पण फक्त valid identifier names सोबत काम करते. bracket notation (obj["prop"]) कोणत्याही string सोबत काम करते, spaces, hyphens किंवा computed property names समाविष्ट. जेव्हा property names dynamic आहेत किंवा special characters समाविष्ट आहेत तेव्हा bracket notation वापरा.

Analogy:
Dot notation is like calling someone by first name (simple, direct), bracket notation is like looking up a phone number (works with any identifier).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Property_accessors

✅ 59. What is an object literal in javascript?

English:
Object literal is a simple way to create objects using curly braces {} with key-value pairs separated by commas. Properties can be added directly: {name: "John", age: 30}. More concise than constructor functions, commonly used for creating single objects or configuration objects.

SKILLHUB

Hindi:
object literal curly braces {} का इस्तेमाल करके objects बनाने का simple तरीका है key-value pairs के साथ commas से separated। properties directly add की जा सकती हैं: {name: "John", age: 30}। constructor functions से ज्यादा concise, single objects या configuration objects बनाने के लिए commonly इस्तेमाल किया जाता है।

Marathi:
object literal curly braces {} वापरून objects तयार करण्याचा सोपा मार्ग आहे key-value pairs सोबत commas ने separated. properties direct add केल्या जाऊ शकतात: {name: "John", age: 30}. constructor functions पेक्षा अधिक concise, single objects किंवा configuration objects तयार करण्यासाठी commonly वापरले जाते.

Analogy:
Like writing a shopping list: quick, direct way to list items and their quantities without formal structure.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Object_initializer

---

✅ 60. How would you clone an object in JavaScript?

English:
Shallow clone: Object.assign({}, obj) or spread operator {...obj}. Deep clone: JSON.parse(JSON.stringify(obj)) for simple objects, or libraries like Lodash for complex objects. Shallow copy shares references, deep copy creates independent copies of nested objects.

Hindi:
shallow clone: Object.assign({}, obj) या spread operator {...obj}। deep clone: simple objects के लिए JSON.parse(JSON.stringify(obj)), या complex objects के लिए Lodash जैसे libraries। shallow copy references share करता है, deep copy nested objects की independent copies बनाता है।

Marathi:
shallow clone: Object.assign({}, obj) किंवा spread operator {...obj}. deep clone: simple objects साठी JSON.parse(JSON.stringify(obj)), किंवा complex objects साठी Lodash सारखी libraries. shallow copy references share करते, deep copy nested objects च्या independent copies तयार करते.

Analogy:
Shallow clone is photocopying the first page only, deep clone is copying the entire multi-page document.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/assign

SKILLHUB

✅ 61. What is a JSON?

English:
JSON (JavaScript Object Notation) is a lightweight data format for exchanging data. Based on JavaScript object syntax but language-independent. Uses key-value pairs, arrays, strings, numbers, booleans, and null. JSON.stringify() converts objects to JSON, JSON.parse() converts JSON to objects.

Hindi:
JSON (JavaScript Object Notation) data exchange के लिए lightweight data format है। JavaScript object syntax पर based है लेकिन language-independent है। key-value pairs, arrays, strings, numbers, booleans, और null का इस्तेमाल करता है। JSON.stringify() objects को JSON में convert करता है, JSON.parse() JSON को objects में convert करता है।

Marathi:
JSON (JavaScript Object Notation) data exchange साठी lightweight data format आहे. JavaScript object syntax वर based आहे पण language-independent आहे. key-value pairs, arrays, strings, numbers, booleans आणि null वापरते. JSON.stringify() objects ला JSON मध्ये convert करते, JSON.parse() JSON ला objects मध्ये convert करते.

Analogy:
Like a universal translator: JSON converts complex objects into a simple, readable format that any programming language can understand.

Learn more:

- https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON

✅ 62. What is prototype chain?

English:
Prototype chain is JavaScript's inheritance mechanism where objects inherit properties/methods from their prototype. When accessing a property, JavaScript searches the object, then its prototype, then the prototype's prototype, until null. Enables shared behavior and memory efficiency.

Hindi:
prototype chain JavaScript की inheritance mechanism है जहां objects अपने prototype से properties/methods inherit करते हैं। जब property access की जाती है, तो JavaScript object को search करता है, फिर उसके prototype को, फिर prototype के prototype को, जब तक null नहीं मिल जाता। shared behavior और memory efficiency को enable करता है।

Marathi:
prototype chain ही JavaScript ची inheritance mechanism आहे जिथे objects त्यांच्या prototype पासून

SKILLHUB

properties/methods inherit करतात. जेव्हा property access केली जाते, तेव्हा JavaScript object ला search करते, नंतर त्याचे prototype, नंतर prototype चे prototype, जोपर्यंत null सापडत नाही. shared behavior आणि memory efficiency सक्षम करते.

Analogy:
Like family inheritance: you inherit traits from parents, who inherited from grandparents, creating a chain of inherited characteristics.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

---

✅ 63. How does prototypal inheritance work in JavaScript?

English:
Prototypal inheritance allows objects to inherit properties/methods from other objects through the prototype chain. Every object has a prototype (accessible via **proto** or Object.getPrototypeOf()). When a property is accessed, JavaScript walks the prototype chain until found or returns undefined.

Hindi:
prototypal inheritance objects को अन्य objects से properties/methods inherit करने की अनुमति देता है prototype chain के माध्यम से। हर object का prototype होता है (**proto** या Object.getPrototypeOf() के माध्यम से accessible)। जब property access की जाती है, तो JavaScript prototype chain को walk करता है जब तक found नहीं होता या undefined रिटर्न करता है।

Marathi:
prototypal inheritance objects ला इतर objects पासून properties/methods inherit करण्यास परवानगी देतो prototype chain द्वारे. प्रत्येक object ला prototype असतो (**proto** किंवा Object.getPrototypeOf() द्वारे accessible). जेव्हा property access केली जाते, तेव्हा JavaScript prototype chain ला walk करते जोपर्यंत सापडत नाही किंवा undefined परत करतो.

Analogy:
Like inheriting family recipes: each generation learns cooking methods from their ancestors through the family line.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

---

✅ 64. What is the difference between an object's prototype and its constructor function?

SKILLHUB

English:
Constructor function creates objects and sets their initial prototype. The prototype is the object that will be assigned as the prototype of instances created by the constructor. Constructor functions have a prototype property, instances have **proto** pointing to that prototype object.

Hindi:
constructor function objects बनाता है और उनका initial prototype set करता है। prototype वह object है जो constructor द्वारा बनाए गए instances का prototype assign किया जाएगा। constructor functions का prototype property होता है, instances का **proto** उस prototype object की ओर point करता है।

Marathi:
constructor function objects तयार करते आणि त्यांचा initial prototype set करते. prototype हा object आहे जो constructor द्वारे तयार केलेल्या instances चा prototype assign केला जाईल. constructor functions ला prototype property असते, instances चा **proto** त्या prototype object कडे point करतो.

Analogy:
Constructor is the factory that builds cars, prototype is the blueprint/template all cars are based on.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/constructor

---

✅ 65. How do you add properties and methods to an object's prototype in JavaScript?

English:
Add to prototype: Constructor.prototype.newProperty = value or Constructor.prototype.newMethod = function() {}. This makes the property/method available to all instances. Changes affect all existing instances since they share the same prototype object.

Hindi:
prototype में add करें: Constructor.prototype.newProperty = value या Constructor.prototype.newMethod = function() {}। यह property/method सभी instances के लिए available बनाता है। changes सभी existing instances को affect करते हैं क्योंकि वे same prototype object share करते हैं।

Marathi:
prototype मध्ये add करा: Constructor.prototype.newProperty = value किंवा Constructor.prototype.newMethod = function() {}. हे property/method सर्व instances साठी available बनवते. changes सर्व existing instances ला affect करतात कारण ते समान prototype object share करतात.

Analogy:
Like adding a new tool to a shared workshop: all workers (instances) can now use the new tool.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/prototype

---

✅ 66. What is the difference between Object.prototype and Object.proto in JavaScript?

English:
Object.prototype is the prototype object used when creating new objects with Object.create() or {}. Object.**proto** is a getter/setter for an object's prototype (deprecated, use Object.getPrototypeOf/setPrototypeOf). prototype is a property on constructor functions, **proto** is on object instances.

Hindi:
Object.prototype वह prototype object है जो Object.create() या {} के साथ new objects बनाने में इस्तेमाल किया जाता है। Object.**proto** एक object के prototype के लिए getter/setter है (deprecated, Object.getPrototypeOf/setPrototypeOf का इस्तेमाल करें)। prototype constructor functions पर property है, **proto** object instances पर है।

Marathi:
Object.prototype हा prototype object आहे जो Object.create() किंवा {} सोबत नवीन objects तयार करण्यासाठी वापरला जातो. Object.**proto** एका object च्या prototype साठी getter/setter आहे (deprecated, Object.getPrototypeOf/setPrototypeOf वापरा). prototype constructor functions वर property आहे, **proto** object instances वर आहे.

Analogy:
prototype is the master blueprint in the factory, **proto** is the specific blueprint attached to each product.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/proto

---

✅ 67. How do you check if an object inherits from a specific prototype in JavaScript?

English:
Use instanceof operator: obj instanceof Constructor checks if Constructor.prototype is in obj's prototype chain. Or Object.getPrototypeOf(obj) === Constructor.prototype. instanceof works with inheritance chains, direct prototype checks are more specific.

SKILLHUB

Hindi:
instanceof operator का इस्तेमाल करें: obj instanceof Constructor check करता है कि Constructor.prototype obj के prototype chain में है या नहीं। या Object.getPrototypeOf(obj) === Constructor.prototype। instanceof inheritance chains के साथ काम करता है, direct prototype checks ज्यादा specific हैं।

Marathi:
instanceof operator वापरा: obj instanceof Constructor check करते की Constructor.prototype obj च्या prototype chain मध्ये आहे की नाही. किंवा Object.getPrototypeOf(obj) === Constructor.prototype. instanceof inheritance chains सोबत काम करते, direct prototype checks अधिक specific आहेत.

Analogy:
Like checking family lineage: instanceof asks "is this person a member of the Johnson family?" by checking the family tree (prototype chain).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/instanceof

---

✅ 68. How do you override a method in an object's prototype in JavaScript?

English:
Override by assigning a new function to the prototype property: Constructor.prototype.methodName = function() { /* new implementation */ }. This affects all instances. For instance-specific override, assign directly to the instance: instance.methodName = function() {}. Prototype overrides are shared, instance overrides are unique.

Hindi:
prototype property को new function assign करके override करें: Constructor.prototype.methodName = function() { /* new implementation */ }। यह सभी instances को affect करता है। instance-specific override के लिए, directly instance को assign करें: instance.methodName = function() {}। prototype overrides shared होते हैं, instance overrides unique होते हैं।

Marathi:
prototype property ला new function assign करून override करा: Constructor.prototype.methodName = function() { /* new implementation */ }. हे सर्व instances ला affect करते. instance-specific override साठी, direct instance ला assign करा: instance.methodName = function() {}. prototype overrides shared असतात, instance overrides unique असतात.

Analogy:
Like updating a family recipe: change it in the family cookbook (prototype) affects everyone, or make personal notes (instance) for just yourself.

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

---

✅ 69. What is the difference between Object.create() and new Object() in JavaScript?

English:
Object.create(proto) creates an object with specified prototype, new Object() creates object inheriting from Object.prototype. Object.create(null) creates object with no prototype. Object.create() is more flexible for inheritance, new Object() always creates basic objects.

Hindi:
Object.create(proto) specified prototype के साथ object बनाता है, new Object() Object.prototype से inherit करने वाला object बनाता है। Object.create(null) no prototype वाला object बनाता है। Object.create() inheritance के लिए ज्यादा flexible है, new Object() हमेशा basic objects बनाता है।

Marathi:
Object.create(proto) specified prototype सोबत object तयार करते, new Object() Object.prototype पासून inherit करणारा object तयार करते. Object.create(null) कोणताही prototype नसलेला object तयार करते. Object.create() inheritance साठी अधिक flexible आहे, new Object() नेहमी basic objects तयार करते.

Analogy:
Object.create() is like adopting a child with chosen family background, new Object() is like having a child who automatically gets the standard family inheritance.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create

---

✅ 70. What is a regular expression(regex)?

English:
Regular expression is a pattern for matching character combinations in strings. Used for validation, search/replace, parsing. Created with /pattern/flags or RegExp constructor. Supports quantifiers, character classes, groups, lookarounds. Powerful but complex - test thoroughly.

Hindi:
regular expression strings में character combinations को match करने के लिए pattern है। validation, search/replace, parsing के लिए इस्तेमाल किया जाता है। /pattern/flags या RegExp constructor से created। quantifiers, character classes, groups, lookarounds support करता है। powerful लेकिन complex - thoroughly test करें।

SKILLHUB

Marathi:
regular expression ही strings मध्ये character combinations match करण्यासाठी pattern आहे. validation, search/replace, parsing साठी वापरली जाते. /pattern/flags किंवा RegExp constructor द्वारे created. quantifiers, character classes, groups, lookarounds support करते. powerful पण complex - thoroughly test करा.

Analogy:
Like a smart search filter: instead of exact words, regex finds patterns like "any word ending with 'ing'" or "phone numbers in this format."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

---

✅ 71. What is the difference between a regular expression and a string?

English:
String is literal text data, regex is a pattern for matching text. Strings use exact matching, regex uses pattern matching with special characters. Regex can match multiple variations, validate formats, extract data. Strings are simpler, regex is more powerful but slower.

Hindi:
string literal text data है, regex text को match करने के लिए pattern है। strings exact matching का इस्तेमाल करते हैं, regex special characters के साथ pattern matching करता है। regex multiple variations को match कर सकता है, formats validate कर सकता है, data extract कर सकता है। strings simpler हैं, regex ज्यादा powerful लेकिन slower है।

Marathi:
string literal text data आहे, regex text match करण्यासाठी pattern आहे. strings exact matching वापरतात, regex special characters सोबत pattern matching करते. regex multiple variations match करू शकते, formats validate करू शकते, data extract करू शकते. strings simpler आहेत, regex अधिक powerful पण slower आहे.

Analogy:
String is like searching for exact recipe name, regex is like searching for "anything with chocolate and nuts."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp

---

SKILLHUB

✅ 72. What is the syntax for creating a regular expression pattern?

English:
Literal syntax: /pattern/flags (e.g., /hello/i for case-insensitive). Constructor: new RegExp("pattern", "flags"). Common flags: g (global), i (case-insensitive), m (multiline). Escape special characters with backslash. Test with .test() method or use with string methods.

Hindi:
literal syntax: /pattern/flags (e.g., /hello/i case-insensitive के लिए)। constructor: new RegExp("pattern", "flags")। common flags: g (global), i (case-insensitive), m (multiline)। backslash से special characters escape करें। .test() method से test करें या string methods के साथ इस्तेमाल करें।

Marathi:
literal syntax: /pattern/flags (e.g., /hello/i case-insensitive साठी). constructor: new RegExp("pattern", "flags"). common flags: g (global), i (case-insensitive), m (multiline). backslash ने special characters escape करा. .test() method ने test करा किंवा string methods सोबत वापरा.

Analogy:
Like writing a search formula: /cat/i means "find 'cat' ignoring uppercase/lowercase" - the slashes contain the pattern, letters after are instructions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/RegExp

SKILLHUB

JavaScript Advanced Features by SKILLHUB - Day 8

✅ 73. What is the difference between the asterisk (*) and the plus sign (+) in regular expressions?

English:
Asterisk (*) matches zero or more of the preceding element (optional, can be absent). Plus (+) matches one or more of the preceding element (required, must appear at least once). Both are greedy by default. Use *? or +? for non-greedy matching.

Hindi:
asterisk (*) preceding element का zero या more match करता है (optional, absent हो सकता है)। plus (+) preceding element का one या more match करता है (required, कम से कम एक बार appear होना चाहिए)। दोनों default में greedy हैं। non-greedy matching के लिए *? या +? का इस्तेमाल करें।

Marathi:
asterisk (*) preceding element चा zero किंवा अधिक match करते (optional, absent असू शकते). plus (+) preceding element चा one किंवा अधिक match करते (required, किमान एकदा appear व्हायला हवे). दोन्ही default मध्ये greedy आहेत. non-greedy matching साठी *? किंवा +? वापरा.

Analogy:
Like grocery shopping: * means "buy as many apples as you want (including zero)", + means "buy at least one apple (one or more)."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Quantifiers

✅ 74. How do you use the question mark (?) in regular expressions?

English:
Question mark (?) makes the preceding element optional (zero or one occurrence). Also used for non-greedy quantifiers (*?, +?, ??) and in lookarounds. In character classes, ? is literal. After quantifiers, makes them non-greedy.

Hindi:
question mark (?) preceding element को optional बनाता है (zero या one occurrence)। non-greedy

SKILLHUB

quantifiers (*?, +?, ??) और lookarounds में भी इस्तेमाल किया जाता है। character classes में ? literal है। quantifiers के बाद, उन्हें non-greedy बनाता है।

Marathi:
question mark (?) preceding element ला optional बनवते (zero किंवा one occurrence). non-greedy quantifiers (*?, +?, ??) आणि lookarounds मध्येही वापरला जातो. character classes मध्ये ? literal आहे. quantifiers नंतर, त्यांना non-greedy बनवते.

Analogy:
Like optional side dish: the main course is required, but ? means "add fries if you want, but it's okay without them too."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

---

✅ 75. How do you specify a range of characters in a character class?

English:
Use hyphens in character classes: [a-z] for lowercase letters, [A-Z] for uppercase, [0-9] for digits, [a-zA-Z0-9] for alphanumeric. Place hyphen at start/end for literal hyphen: [-abc] or [abc-]. Ranges are inclusive and based on Unicode values.

Hindi:
character classes में hyphens का इस्तेमाल करें: [a-z] lowercase letters के लिए, [A-Z] uppercase के लिए, [0-9] digits के लिए, [a-zA-Z0-9] alphanumeric के लिए। literal hyphen के लिए start/end में place करें: [-abc] या [abc-]। ranges inclusive हैं और Unicode values पर based हैं।

Marathi:
character classes मध्ये hyphens वापरा: [a-z] lowercase letters साठी, [A-Z] uppercase साठी, [0-9] digits साठी, [a-zA-Z0-9] alphanumeric साठी. literal hyphen साठी start/end मध्ये place करा: [-abc] किंवा [abc-]. ranges inclusive आहेत आणि Unicode values वर based आहेत.

Analogy:
Like specifying age range: [18-65] means "anyone from 18 to 65 years old, inclusive."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Character_Classes

---

SKILLHUB

✅ 76. What is the difference between a greedy and a non-greedy match in regular expressions?

English:
Greedy quantifiers (*, +, {n,}) match as much as possible. Non-greedy (?, +?, {n,}?) match as little as possible. Greedy: /"."/ matches everything between first/last quotes. Non-greedy: /".?"/ matches between first matching quotes.

Hindi:
greedy quantifiers (*, +, {n,}) जितना possible हो उतना match करते हैं। non-greedy (?, +?, {n,}?) जितना कम possible हो उतना match करते हैं। greedy: /"."/ first/last quotes के बीच everything match करता है। non-greedy: /".?"/ first matching quotes के बीच match करता है।

Marathi:
greedy quantifiers (*, +, {n,}) जितके possible तितके match करतात. non-greedy (?, +?, {n,}?) जितके कमी possible तितके match करतात. greedy: /"."/ first/last quotes दरम्यान everything match करते. non-greedy: /".?"/ first matching quotes दरम्यान match करते.

Analogy:
Greedy shopper buys everything on the shelf, non-greedy shopper buys only what they need for today.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Quantifiers

---

✅ 77. How do you use the pipe (|) operator in regular expressions?

English:
Pipe (|) is alternation operator for "OR" logic. Matches either the pattern before or after the pipe. Use parentheses for grouping: /(cat|dog)/ matches "cat" or "dog". Can chain multiple alternatives: /(red|blue|green)/. Order matters - first match wins.

Hindi:
pipe (|) alternation operator है "OR" logic के लिए। pipe से पहले या बाद के pattern को match करता है। grouping के लिए parentheses का इस्तेमाल करें: /(cat|dog)/ "cat" या "dog" को match करता है। multiple alternatives chain कर सकते हैं: /(red|blue|green)/। order matters - first match wins।

Marathi:
pipe (|) alternation operator आहे "OR" logic साठी. pipe पूर्वी किंवा नंतरच्या pattern ला match करते. grouping साठी parentheses वापरा: /(cat|dog)/ "cat" किंवा "dog" ला match करते. multiple alternatives chain करू शकता: /(red|blue|green)/. order matters - first match wins.

SKILLHUB

Analogy:
Like choosing from menu: "fish OR chicken OR beef" - the | separates your options.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

---

✅ 78. What is the purpose of the caret (^) and dollar sign ($) characters in regular expressions?

English:
Caret (^) matches beginning of string/line (with multiline flag). Dollar ($) matches end of string/line. Together /^pattern$/ matches entire string. Without multiline, ^/$ match string start/end. With multiline flag, match line boundaries within multiline strings.

Hindi:
caret (^) string/line की beginning को match करता है (multiline flag के साथ)। dollar ($) string/line के end को match करता है। साथ में /^pattern$/ entire string को match करता है। multiline के बिना, ^/$ string start/end match करते हैं। multiline flag के साथ, multiline strings में line boundaries match करते हैं।

Marathi:
caret (^) string/line च्या beginning ला match करते (multiline flag सोबत). dollar ($) string/line च्या end ला match करते. एकत्र /^pattern$/ संपूर्ण string ला match करते. multiline शिवाय, ^/$ string start/end match करतात. multiline flag सोबत, multiline strings मध्ये line boundaries match करतात.

Analogy:
Like bookends: ^ marks the start of the story, $ marks the end - ensuring the pattern matches the whole book, not just a chapter.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Assertions

---

✅ 79. What are some common use cases for regular expressions?

English:
Email validation, phone number formatting, URL parsing, password strength checking, code syntax highlighting, log file parsing, search/replace in text editors, input sanitization, data extraction from unstructured text, form validation.

SKILLHUB

Hindi:
email validation, phone number formatting, URL parsing, password strength checking, code syntax highlighting, log file parsing, text editors में search/replace, input sanitization, unstructured text से data extraction, form validation।

Marathi:
email validation, phone number formatting, URL parsing, password strength checking, code syntax highlighting, log file parsing, text editors मध्ये search/replace, input sanitization, unstructured text मधून data extraction, form validation.

Analogy:
Like a Swiss Army knife: regex solves many text processing problems - from validating emails to extracting data from messy text.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

---

✅ 80. How do you match a specific number of characters in a regular expression?

English:
Use curly braces for exact quantifiers: {n} for exactly n occurrences, {n,} for n or more, {n,m} for between n and m. Examples: \d{3} matches exactly 3 digits, [a-z]{2,5} matches 2-5 lowercase letters. Combine with other patterns for precise matching.

Hindi:
exact quantifiers के लिए curly braces का इस्तेमाल करें: {n} exactly n occurrences के लिए, {n,} n या more के लिए, {n,m} n और m के बीच के लिए। examples: \d{3} exactly 3 digits match करता है, [a-z]{2,5} 2-5 lowercase letters match करता है। precise matching के लिए अन्य patterns के साथ combine करें।

Marathi:
exact quantifiers साठी curly braces वापरा: {n} exactly n occurrences साठी, {n,} n किंवा अधिक साठी, {n,m} n आणि m दरम्यान साठी. examples: \d{3} exactly 3 digits match करते, [a-z]{2,5} 2-5 lowercase letters match करते. precise matching साठी इतर patterns सोबत combine करा.

Analogy:
Like specifying exact measurements: "exactly 3 cups" {3}, "at least 2 cups" {2,}, "between 1-3 cups" {1,3}.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Quantifiers

---

SKILLHUB

✅ 81. How do you match a specific character that has a special meaning in a regular expression?

English:
Escape special characters with backslash (): . for literal dot, * for literal asterisk, + for literal plus, ? for literal question mark, { for literal curly brace. Inside character classes, most characters lose special meaning except ,], and -.

Hindi:
backslash () से special characters को escape करें: . literal dot के लिए, * literal asterisk के लिए, + literal plus के लिए, ? literal question mark के लिए, { literal curly brace के लिए। character classes के अंदर, ,], और - को छोड़कर ज्यादातर characters अपना special meaning खो देते हैं।

Marathi:
backslash () ने special characters ला escape करा: . literal dot साठी, * literal asterisk साठी, + literal plus साठी, ? literal question mark साठी, { literal curly brace साठी. character classes च्या आत, ,], आणि - वगळता बहुतेक characters त्यांचे special meaning गमावतात.

Analogy:
Like saying the character's name instead of its action: when you want to talk about "period" instead of "any character", you escape it.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions

---

✅ 82. How do you use lookarounds in regular expressions?

English:
Lookarounds match patterns based on context without including context in match. Positive lookahead (?=pattern) matches if pattern follows. Negative lookahead (?!pattern) matches if pattern doesn't follow. Lookbehinds (?<=pattern) and (?

Hindi:
lookarounds context के आधार पर patterns को match करते हैं बिना context को match में include किए। positive lookahead (?=pattern) तब match करता है जब pattern follows करता है। negative lookahead (?!pattern) तब match करता है जब pattern follow नहीं करता। lookbehinds (?<=pattern) और (?

Marathi:
lookarounds context वर आधारित patterns ला match करतात context ला match मध्ये include न करता. positive lookahead (?=pattern) match करते जेव्हा pattern follow करते. negative lookahead (?!pattern) match करते जेव्हा pattern follow करत नाही. lookbehinds (?<=pattern) आणि (?

Analogy:

SKILLHUB

Like checking the weather before dressing: lookahead checks "is it sunny ahead?" without including weather in your outfit choice.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions/Assertions

---

✅ 83. What is the window.location object in JavaScript?

English:
window.location represents current browser URL and provides methods to navigate. Contains properties: href (full URL), protocol, host, hostname, port, pathname, search (query string), hash. Methods: assign(), replace(), reload(). Essential for URL manipulation and navigation.

Hindi:
window.location current browser URL को represent करता है और navigate करने के लिए methods प्रदान करता है। properties contain करता है: href (full URL), protocol, host, hostname, port, pathname, search (query string), hash। methods: assign(), replace(), reload()। URL manipulation और navigation के लिए essential है।

Marathi:
window.location current browser URL दर्शवते आणि navigate करण्यासाठी methods पुरवते. properties समाविष्ट: href (full URL), protocol, host, hostname, port, pathname, search (query string), hash. methods: assign(), replace(), reload(). URL manipulation आणि navigation साठी essential आहे.

Analogy:
Like a GPS device: location object tells you where you are (current URL) and how to get to new places (navigation methods).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Location

✅ 84. What are the properties of the window.location object?

English:
Key properties: href (full URL), protocol (http/https), host (hostname:port), hostname (domain), port, pathname (path after domain), search (query parameters), hash (fragment identifier). All are readable/writable except some in older browsers.

Hindi:
key properties: href (full URL), protocol (http/https), host (hostname:port), hostname (domain), port,

SKILLHUB

pathname (domain के बाद का path), search (query parameters), hash (fragment identifier)। सभी readable/writable हैं सिवाय कुछ old browsers में।

Marathi:
key properties: href (full URL), protocol (http/https), host (hostname:port), hostname (domain), port, pathname (domain नंतरचा path), search (query parameters), hash (fragment identifier). सर्व readable/writable आहेत काही old browsers वगळता.

Analogy:
Like address components: href is the complete address, hostname is the building name, pathname is the apartment number.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Location

✅ 85. How do you redirect to another page using JavaScript's window.location object?

English:
Set location.href = "newURL" for standard redirect (adds to history), or location.replace("newURL") to replace current history entry. Or location.assign("newURL") same as href. Choose based on whether you want users to return with back button.

Hindi:
standard redirect के लिए location.href = "newURL" set करें (history में add होता है), या current history entry को replace करने के लिए location.replace("newURL")। या location.assign("newURL") href के same। back button से वापस आने देना चाहते हैं या नहीं इसके आधार पर चुनें।

Marathi:
standard redirect साठी location.href = "newURL" set करा (history मध्ये add होते), किंवा current history entry replace करण्यासाठी location.replace("newURL"). किंवा location.assign("newURL") href सारखे. back button ने परत येण्यास परवानगी द्यायची आहे की नाही यावर अवलंबून निवडा.

Analogy:
Like changing destinations: href is "go to new place and remember how to get back", replace is "teleport and forget the old location."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Location/assign

---

SKILLHUB

✅ 86. How do you reload the current page using JavaScript's window.location object?

English:
Use location.reload() to reload from cache, or location.reload(true) to force reload from server. Or set location.href = location.href to reload. reload() is preferred as it's cleaner and allows cache control.

Hindi:
cache से reload करने के लिए location.reload() का इस्तेमाल करें, या server से force reload के लिए location.reload(true)। या location.href = location.href set करके reload करें। reload() preferred है क्योंकि यह cleaner है और cache control की अनुमति देता है।

Marathi:
cache मधून reload करण्यासाठी location.reload() वापरा, किंवा server मधून force reload साठी location.reload(true). किंवा location.href = location.href set करून reload करा. reload() preferred आहे कारण ते cleaner आहे आणि cache control ची परवानगी देते.

Analogy:
Like refreshing your coffee: reload() is "top up from the same cup", reload(true) is "make a fresh cup."

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Location/reload

---

✅ 87. How do you get the value of a query parameter from the URL using JavaScript's window.location object?

English:
Access location.search for query string, then parse manually or use URLSearchParams: new URLSearchParams(location.search).get('paramName'). For older browsers, split and decode manually. Always decode URI components to handle special characters.

Hindi:
query string के लिए location.search access करें, फिर manually parse करें या URLSearchParams इस्तेमाल करें: new URLSearchParams(location.search).get('paramName')। old browsers के लिए, manually split और decode करें। special characters को handle करने के लिए हमेशा URI components decode करें।

Marathi:
query string साठी location.search access करा, नंतर manually parse करा किंवा URLSearchParams वापरा: new URLSearchParams(location.search).get('paramName'). old browsers साठी, manually split आणि decode करा. special characters handle करण्यासाठी नेहमी URI components decode करा.

SKILLHUB

Analogy:
Like checking baggage tags: URLSearchParams helps you find "what's in the red suitcase?" by reading the query parameter labels.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams

---

✅ 88. How do you compare two dates in JavaScript?

English:
Convert dates to timestamps with getTime() or Date.now(), then compare numbers. Direct comparison with < > works for Date objects. Be aware of timezone issues. For same day check, compare year/month/day separately or use toDateString().

Hindi:
dates को getTime() या Date.now() से timestamps में convert करें, फिर numbers compare करें। Date objects के लिए direct < > comparison काम करता है। timezone issues का ध्यान रखें। same day check के लिए, year/month/day को separately compare करें या toDateString() इस्तेमाल करें।

Marathi:
dates ला getTime() किंवा Date.now() ने timestamps मध्ये convert करा, नंतर numbers compare करा. Date objects साठी direct < > comparison काम करते. timezone issues लक्षात घ्या. same day check साठी, year/month/day ला separately compare करा किंवा toDateString() वापरा.

Analogy:
Like comparing heights: convert to inches (timestamps) for easy comparison, or stand back-to-back for direct visual comparison.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

---

✅ 89. How do you get the current timestamp in JavaScript?

English:
Use Date.now() for current timestamp in milliseconds since Unix epoch (January 1, 1970). Or new Date().getTime(). More precise than new Date() for timing. Use for performance measurements, caching, unique IDs.

Hindi:
Unix epoch (January 1, 1970) से milliseconds में current timestamp के लिए Date.now() इस्तेमाल करें। या

SKILLHUB

new Date().getTime() । timing के लिए new Date() से ज्यादा precise । performance measurements, caching, unique IDs के लिए इस्तेमाल करें।

Marathi:
Unix epoch (January 1, 1970) पासून milliseconds मध्ये current timestamp साठी Date.now() वापरा. किंवा new Date().getTime(). timing साठी new Date() पेक्षा अधिक precise. performance measurements, caching, unique IDs साठी वापरा.

Analogy:
Like a stopwatch: Date.now() gives you the exact millisecond when you pressed "start" for precise timing.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/now

---

✅ 90. How do you add or subtract days to a date in JavaScript?

English:
Use setDate() method: date.setDate(date.getDate() + days) to add/subtract days. Be careful with month boundaries (31st + 1 day = 1st of next month). For larger time spans, use setTime() with milliseconds, or libraries like date-fns for complex operations.

Hindi:
setDate() method इस्तेमाल करें: date.setDate(date.getDate() + days) days add/subtract करने के लिए। month boundaries से careful रहें (31st + 1 day = next month का 1st) । larger time spans के लिए, milliseconds के साथ setTime() इस्तेमाल करें, या complex operations के लिए date-fns जैसे libraries ।

Marathi:
setDate() method वापरा: date.setDate(date.getDate() + days) days add/subtract करण्यासाठी. month boundaries काळजीपूर्वक हाताळा (31st + 1 day = पुढच्या महिन्याचा 1st). larger time spans साठी, milliseconds सोबत setTime() वापरा, किंवा complex operations साठी date-fns सारखी libraries.

Analogy:
Like calendar math: adding 7 days moves you exactly one week forward, but watch for month transitions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setDate

---

✅ 91. what is iterator in javascript?

SKILLHUB

English:
Iterator is object with next() method that returns {value, done} objects. Implements iteration protocol for custom data structures. Used by for...of loops, spread operator, Array.from(). Create with Symbol.iterator. Enables lazy evaluation and infinite sequences.

Hindi:
iterator ऐसा object है जिसमें next() method है जो {value, done} objects रिटर्न करता है। custom data structures के लिए iteration protocol implement करता है। for...of loops, spread operator, Array.from() द्वारा इस्तेमाल किया जाता है। Symbol.iterator से create करें। lazy evaluation और infinite sequences को enable करता है।

Marathi:
iterator असा object आहे ज्यामध्ये next() method आहे जी {value, done} objects परत करते. custom data structures साठी iteration protocol implement करते. for...of loops, spread operator, Array.from() द्वारे वापरला जातो. Symbol.iterator ने create करा. lazy evaluation आणि infinite sequences सक्षम करते.

Analogy:
Like a bookmark in a book: iterator remembers where you left off and gives you the next item when asked.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Iteration_protocols

---

✅ 92. What is decorator in javascript?

English:
Decorators are functions that modify classes, methods, or properties at design time. ES2016+ feature, widely used in frameworks like Angular. @decorator syntax wraps target with additional behavior. Can add logging, validation, caching, etc. without modifying original code.

Hindi:
decorators ऐसे functions हैं जो design time पर classes, methods, या properties को modify करते हैं। ES2016+ feature, Angular जैसे frameworks में widely इस्तेमाल। @decorator syntax target को additional behavior से wrap करता है। original code को modify किए बिना logging, validation, caching, etc. add कर सकते हैं।

Marathi:
decorators असे functions आहेत जी design time वर classes, methods किंवा properties ला modify करतात. ES2016+ feature, Angular सारख्या frameworks मध्ये widely वापरले. @decorator syntax target ला additional behavior ने wrap करते. original code ला modify न करता logging, validation, caching, etc. add करू शकता.

SKILLHUB

Analogy:
Like gift wrapping: decorator adds beautiful packaging (extra functionality) around your present (original code) without changing what's inside.

Learn more:

- https://javascript.info/decorators

✅ 93. What is optional chaining?

English:
Optional chaining (?.) stops evaluation if reference is null/undefined, preventing errors. Instead of obj.prop.subprop (crashes if obj null), use obj?.prop?.subprop (returns undefined safely). Works with property access, method calls, array access. ES2020 feature.

Hindi:
optional chaining (?.) evaluation को stop करता है अगर reference null/undefined है, errors को prevent करता है। obj.prop.subprop (crashes if obj null) के बजाय obj?.prop?.subprop (safely undefined रिटर्न करता है) इस्तेमाल करें। property access, method calls, array access के साथ काम करता है। ES2020 feature।

Marathi:
optional chaining (?.) evaluation ला थांबवते जर reference null/undefined असेल, errors prevent करते. obj.prop.subprop (crashes if obj null) ऐवजी obj?.prop?.subprop (safely undefined परत करते) वापरा. property access, method calls, array access सोबत काम करते. ES2020 feature.

Analogy:
Like a safety net: optional chaining prevents falls (errors) by checking if each step exists before proceeding.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Optional_chaining

---

✅ 94. What is throttling?

English:
Throttling limits function execution to once per time window, regardless of calls. Unlike debouncing (waits for pause), throttling enforces maximum rate. Used for scroll handlers, resize events, API calls. Prevents excessive execution while maintaining responsiveness.

Hindi:
throttling function execution को time window में एक बार limit करता है, calls की संख्या की परवाह किए बिना। debouncing (pause का wait करता है) के विपरीत, throttling maximum rate enforce करता है। scroll

SKILLHUB

handlers, resize events, API calls के लिए इस्तेमाल किया जाता है। responsiveness बनाए रखते हुए excessive execution को prevent करता है।

Marathi:
throttling function execution ला time window मध्ये एकदा limit करते, calls च्या संख्येची पर्वा न करता. debouncing (pause साठी थांबते) च्या विपरीत, throttling maximum rate enforce करते. scroll handlers, resize events, API calls साठी वापरले जाते. responsiveness ठेवत excessive execution prevent करते.

Analogy:
Like traffic lights: throttling ensures cars (function calls) can only pass at controlled intervals, preventing traffic jams.

Learn more:

- https://css-tricks.com/debouncing-throttling-explained-examples/

✅ 95. What is babel?

English:
Babel is JavaScript transpiler that converts modern ES6+ code to ES5 for older browser compatibility. Transforms arrow functions, classes, modules, etc. to compatible syntax. Essential for modern development, configured with presets and plugins.

Hindi:
babel JavaScript transpiler है जो modern ES6+ code को older browser compatibility के लिए ES5 में convert करता है। arrow functions, classes, modules, etc. को compatible syntax में transform करता है। modern development के लिए essential, presets और plugins के साथ configured किया जाता है।

Marathi:
babel JavaScript transpiler आहे जे modern ES6+ code ला older browser compatibility साठी ES5 मध्ये convert करते. arrow functions, classes, modules, etc. ला compatible syntax मध्ये transform करते. modern development साठी essential, presets आणि plugins सोबत configured केले जाते.

Analogy:
Like a language translator: Babel converts modern JavaScript "dialect" to older browser "dialect" everyone can understand.

Learn more:

- https://babeljs.io/docs/

✅ 96. What is debouncing?

SKILLHUB

English:
Debouncing delays function execution until pause in calls, then executes once. Used for search inputs, window resize. Prevents excessive API calls while typing. Unlike throttling (fixed rate), debouncing waits for user to stop before executing.

Hindi:
debouncing calls में pause तक function execution को delay करता है, फिर एक बार execute करता है। search inputs, window resize के लिए इस्तेमाल किया जाता है। typing के दौरान excessive API calls को prevent करता है। throttling (fixed rate) के विपरीत, debouncing execute करने से पहले user के stop होने का wait करता है।

Marathi:
debouncing calls मध्ये pause पर्यंत function execution ला delay करते, नंतर एकदा execute करते. search inputs, window resize साठी वापरले जाते. typing दरम्यान excessive API calls prevent करते. throttling (fixed rate) च्या विपरीत, debouncing execute करण्यापूर्वी user थांबण्याची वाट पाहते.

Analogy:
Like elevator doors: debouncing waits for people to stop entering before closing (executing) the door.

Learn more:

- https://css-tricks.com/debouncing-throttling-explained-examples/

✅ 97. What is global execution context?

English:
Global execution context is JavaScript's base context created when script loads. Contains global object (window in browsers), this refers to global object. All global variables/functions become properties of global object. One per JavaScript environment.

Hindi:
global execution context JavaScript का base context है जो script load होने पर create होता है। global object (browsers में window) contain करता है, this global object को refer करता है। सभी global variables/functions global object के properties बन जाते हैं। JavaScript environment में एक।

Marathi:
global execution context JavaScript चा base context आहे जो script load झाल्यावर create होतो. global object (browsers मध्ये window) contain करते, this global object ला refer करते. सर्व global variables/functions global object ची properties बनतात. JavaScript environment मध्ये एक.

Analogy:
Like the main stage: global execution context is where the entire JavaScript performance happens, with global variables as the main actors.

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_context

---

SKILLHUB

JavaScript Modern Concepts by SKILLHUB - Day 9

✅ 98. What is function execution context?

English:
Function execution context created when function called, with its own scope, this binding, and variable environment. Contains function arguments, local variables, and reference to outer scope. Each function call creates new context pushed to call stack.

Hindi:
function execution context function call होने पर create होता है, अपने own scope, this binding, और variable environment के साथ। function arguments, local variables, और outer scope का reference contain करता है। हर function call new context create करता है जो call stack में push होता है।

Marathi:
function execution context function call झाल्यावर create होतो, स्वतःच्या scope, this binding आणि variable environment सोबत. function arguments, local variables आणि outer scope चा reference contain करते. प्रत्येक function call नवीन context create करते जो call stack मध्ये push होतो.

Analogy:
Like private rooms at a party: each function gets its own space with its own variables and access to the main party (global scope).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_context

---

✅ 99. What does the variable environment in JavaScript's function execution context contain, and what is its purpose?

English:
Variable environment stores function parameters, local variables, and function declarations. Purpose is to manage variable lifecycle within function scope. Variables hoisted to top, functions fully hoisted. Enables lexical scoping and proper variable resolution.

Hindi:
variable environment function parameters, local variables, और function declarations को store करता है।

SKILLHUB

purpose function scope के अंदर variable lifecycle manage करना है। variables top पर hoisted, functions fully hoisted। lexical scoping और proper variable resolution को enable करता है।

Marathi:
variable environment function parameters, local variables आणि function declarations store करते. purpose function scope च्या आत variable lifecycle manage करणे आहे. variables top वर hoisted, functions fully hoisted. lexical scoping आणि proper variable resolution सक्षम करते.

Analogy:
Like a function's personal locker: stores all its belongings (variables) and knows where everything goes.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_context

---

✅ 100. What is a scope chain in JavaScript's function execution context?

English:
Scope chain is hierarchy of scopes JavaScript searches for variable resolution, from innermost to outermost. When variable referenced, JavaScript checks current scope, then outer scope, until found or global scope reached. Enables closure and lexical scoping.

Hindi:
scope chain scopes की hierarchy है जिसमें JavaScript variable resolution के लिए search करता है, innermost से outermost तक। जब variable referenced होती है, तो JavaScript current scope check करता है, फिर outer scope, जब तक found नहीं या global scope तक नहीं पहुंच जाता। closure और lexical scoping को enable करता है।

Marathi:
scope chain scopes ची hierarchy आहे ज्यामध्ये JavaScript variable resolution साठी search करते, innermost पासून outermost पर्यंत. जेव्हा variable referenced केली जाते, तेव्हा JavaScript current scope check करते, नंतर outer scope, जोपर्यंत सापडत नाही किंवा global scope पर्यंत पोहोचत नाही. closure आणि lexical scoping सक्षम करते.

Analogy:
Like asking up the family tree: "Do you know where the keys are? No? Ask grandma (outer scope)."

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Scope

SKILLHUB

✅ 101. How do I modify the url without reloading the page?

English:
Use History API: history.pushState() adds entry to history, history.replaceState() replaces current entry. Or location.hash = "newhash" for hash changes. Enables SPA navigation without full page reloads. Listen to popstate events for back/forward button handling.

Hindi:
history API इस्तेमाल करें: history.pushState() history में entry add करता है, history.replaceState() current entry को replace करता है। या hash changes के लिए location.hash = "newhash"। full page reloads के बिना SPA navigation को enable करता है। back/forward button handling के लिए popstate events को listen करें।

Marathi:
history API वापरा: history.pushState() history मध्ये entry add करते, history.replaceState() current entry replace करते. किंवा hash changes साठी location.hash = "newhash". full page reloads शिवाय SPA navigation सक्षम करते. back/forward button handling साठी popstate events ला listen करा.

Analogy:
Like changing TV channels: pushState/replaceState changes the channel (URL) without turning off the TV (reloading the page).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/History_API

✅ 102. What are dynamic imports in javascript?

English:
Dynamic imports load modules asynchronously at runtime using import() function, returning Promise. Unlike static imports (top-level), dynamic imports can be conditional or loaded on-demand. Enables code splitting, lazy loading, and conditional module loading. ES2020 feature.

Hindi:
dynamic imports import() function का इस्तेमाल करके runtime पर asynchronously modules load करते हैं, Promise रिटर्न करते हैं। static imports (top-level) के विपरीत, dynamic imports conditional हो सकते हैं या on-demand load किए जा सकते हैं। code splitting, lazy loading, और conditional module loading को enable करते हैं। ES2020 feature।

Marathi:
dynamic imports import() function वापरून runtime वर asynchronously modules load करतात, Promise परत करतात. static imports (top-level) च्या विपरीत, dynamic imports conditional असू शकतात किंवा on-demand load केले जाऊ शकतात. code splitting, lazy loading आणि conditional module loading सक्षम करतात. ES2020 feature.

SKILLHUB

Analogy:
Like ordering food as needed: dynamic imports let you request modules only when you actually need them, not all at once.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/import#dynamic_imports

---

✅ 103. How do you empty an array?

English:
Multiple ways: arr.length = 0 (fastest, modifies original), arr.splice(0, arr.length), arr = [] (creates new reference), or arr.length = 0 while preserving reference. Choose based on whether you want to modify original array or create new reference.

Hindi:
multiple तरीके: arr.length = 0 (fastest, original को modify करता है), arr.splice(0, arr.length), arr = [] (new reference create करता है), या reference preserve करते हुए arr.length = 0। original array को modify करना चाहते हैं या new reference create करना चाहते हैं इसके आधार पर चुनें।

Marathi:
multiple पद्धती: arr.length = 0 (fastest, original ला modify करते), arr.splice(0, arr.length), arr = [] (new reference create करते), किंवा reference preserve करत arr.length = 0. original array ला modify करायचे आहे की new reference create करायचे आहे यावर अवलंबून निवडा.

Analogy:
Like cleaning a whiteboard: length = 0 erases everything, arr = [] gets a new whiteboard.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

---

✅ 104. What is a lambda function?

English:
Lambda function is anonymous function, often arrow function in JavaScript. Concise syntax, doesn't bind own this/arguments/super. Used for short operations, callbacks. In functional programming, refers to anonymous functions that can be passed around.

Hindi:
lambda function anonymous function है, JavaScript में often arrow function। concise syntax, अपना

SKILLHUB

this/arguments/super bind नहीं करता। short operations, callbacks के लिए इस्तेमाल किया जाता है। functional programming में, anonymous functions को refer करता है जो pass around किए जा सकते हैं।

Marathi:
lambda function anonymous function आहे, JavaScript मध्ये often arrow function. concise syntax, स्वतःचे this/arguments/super bind करत नाही. short operations, callbacks साठी वापरले जाते. functional programming मध्ये, anonymous functions ला refer करते जे pass around केले जाऊ शकतात.

Analogy:
Like disposable cutlery: lambda functions are quick, single-use functions you don't need to name.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

---

✅ 105. What is variable shadowing in javascript?

English:
Variable shadowing occurs when variable in inner scope has same name as outer scope variable, hiding outer variable. Common in functions with parameters named same as global variables. Can cause bugs if unintentional, but useful for encapsulation.

Hindi:
variable shadowing तब होता है जब inner scope का variable outer scope के variable से same name होता है, outer variable को hide करता है। functions में common है जहां parameters global variables से same named हैं। अगर unintentional है तो bugs का कारण बन सकता है, लेकिन encapsulation के लिए useful है।

Marathi:
variable shadowing जेव्हा inner scope चा variable outer scope च्या variable सारखे नाव असतो, outer variable ला hide करते. functions मध्ये common आहे जिथे parameters global variables सारखी नावे आहेत. जर unintentional असेल तर bugs चा कारण होऊ शकते, पण encapsulation साठी useful आहे.

Analogy:
Like wearing sunglasses inside: the room light (outer scope) is hidden by the dark lenses (inner scope variable).

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Scope

✅ 106. What is a rest operator in javascript?

SKILLHUB

English:
Rest operator (...) collects remaining arguments into array. In function parameters: function sum(...nums), in destructuring: [first, ...rest] = arr. Opposite of spread operator. Useful for variable argument functions and array destructuring.

Hindi:
rest operator (...) remaining arguments को array में collect करता है। function parameters में: function sum(...nums), destructuring में: [first, ...rest] = arr। spread operator का opposite। variable argument functions और array destructuring के लिए useful।

Marathi:
rest operator (...) remaining arguments ला array मध्ये collect करते. function parameters मध्ये: function sum(...nums), destructuring मध्ये: [first, ...rest] = arr. spread operator चे opposite. variable argument functions आणि array destructuring साठी useful.

Analogy:
Like a vacuum cleaner: rest operator sucks up all the remaining items into a container (array).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/rest_parameters

---

✅ 107. What is a spread operator?

English:
Spread operator (...) expands iterables into individual elements. For arrays: [...arr1, ...arr2] merges, for objects: {...obj1, ...obj2} merges properties. Opposite of rest operator. Used for copying, merging, and passing array elements as function arguments.

Hindi:
spread operator (...) iterables को individual elements में expand करता है। arrays के लिए: [...arr1, ...arr2] merge करता है, objects के लिए: {...obj1, ...obj2} properties merge करता है। rest operator का opposite। copying, merging, और function arguments के रूप में array elements pass करने के लिए इस्तेमाल किया जाता है।

Marathi:
spread operator (...) iterables ला individual elements मध्ये expand करते. arrays साठी: [...arr1, ...arr2] merge करते, objects साठी: {...obj1, ...obj2} properties merge करते. rest operator चे opposite. copying, merging आणि function arguments म्हणून array elements pass करण्यासाठी वापरले जाते.

Analogy:
Like unpacking a suitcase: spread operator takes out each item individually and lays them out.

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

---

✅ 108. What paradigm is Javascript?

English:
JavaScript is multi-paradigm language supporting object-oriented programming (prototypal inheritance, classes), functional programming (first-class functions, closures), and imperative programming. No single paradigm dominates - choose based on problem. Modern JS leans toward functional programming patterns.

Hindi:
JavaScript multi-paradigm language है जो object-oriented programming (prototypal inheritance, classes), functional programming (first-class functions, closures), और imperative programming support करता है। कोई single paradigm dominate नहीं करता - problem के आधार पर चुनें। modern JS functional programming patterns की ओर lean करता है।

Marathi:
JavaScript multi-paradigm language आहे जे object-oriented programming (prototypal inheritance, classes), functional programming (first-class functions, closures) आणि imperative programming support करते. कोणताही single paradigm dominate करत नाही - problem वर आधारित निवडा. modern JS functional programming patterns कडे lean करते.

Analogy:
Like a Swiss Army knife: JavaScript can approach problems from multiple angles - OOP for modeling, functional for data transformation, imperative for step-by-step procedures.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Introduction

---

✅ 109. How do you assign default values to variables?

English:
Use default parameter syntax: function greet(name = "Guest"). For variables: const name = value || "default". ES2020 nullish coalescing: const name = value ?? "default". Logical OR (||) treats falsy values as defaults, nullish (??) only null/undefined.

Hindi:
default parameter syntax इस्तेमाल करें: function greet(name = "Guest") । variables के लिए: const name = value || "default" । ES2020 nullish coalescing: const name = value ?? "default" । logical OR (||) falsy values को defaults के रूप में treat करता है, nullish (??) सिर्फ null/undefined को।

SKILLHUB

Marathi:
default parameter syntax वापरा: function greet(name = "Guest"). variables साठी: const name = value || "default". ES2020 nullish coalescing: const name = value ?? "default". logical OR (||) falsy values ला defaults म्हणून treat करते, nullish (??) फक्त null/undefined ला.

Analogy:
Like having backup plans: default values ensure you always have something to work with, even when input is missing.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Default_parameters

---

✅ 110. What's a typical use case for anonymous functions?

English:
Anonymous functions used for one-time operations: callbacks, event handlers, IIFEs, array methods. No need for naming when function used immediately or passed as argument. Arrow functions commonly used for concise anonymous functions in modern code.

Hindi:
anonymous functions एक-time operations के लिए इस्तेमाल किए जाते हैं: callbacks, event handlers, IIFEs, array methods। जब function immediately इस्तेमाल किया जाता है या argument के रूप में pass किया जाता है तब naming की need नहीं। modern code में concise anonymous functions के लिए arrow functions commonly इस्तेमाल किए जाते हैं।

Marathi:
anonymous functions एक-time operations साठी वापरले जातात: callbacks, event handlers, IIFEs, array methods. जेव्हा function immediately वापरला जातो किंवा argument म्हणून pass केला जातो तेव्हा naming ची गरज नाही. modern code मध्ये concise anonymous functions साठी arrow functions commonly वापरले जातात.

Analogy:
Like disposable cameras: anonymous functions are used once and thrown away, no need to keep track of them.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions

---

SKILLHUB

✅ 111. What is the difference between a prototype and an instance?

English:
Prototype is template object containing shared properties/methods for instances. Instance is individual object created from constructor/class. Changes to prototype affect all instances, changes to instance affect only that object. Prototypes enable memory-efficient inheritance.

Hindi:
prototype template object है जिसमें instances के लिए shared properties/methods होते हैं। instance individual object है जो constructor/class से create किया जाता है। prototype में changes सभी instances को affect करते हैं, instance में changes सिर्फ उस object को affect करते हैं। prototypes memory-efficient inheritance को enable करते हैं।

Marathi:
prototype template object आहे ज्यात instances साठी shared properties/methods असतात. instance individual object आहे जो constructor/class पासून create केला जातो. prototype मध्ये changes सर्व instances ला affect करतात, instance मध्ये changes फक्त त्या object ला affect करतात. prototypes memory-efficient inheritance सक्षम करतात.

Analogy:
Like a cookie cutter (prototype) vs. individual cookies (instances) - the cutter defines the shape, but each cookie is unique.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Inheritance_and_the_prototype_chain

---

✅ 112. What is a function expression?

English:
Function expression creates function as part of expression, can be anonymous or named. Assigned to variables: const func = function() {}. Not hoisted like function declarations. Used when function needed as value, in closures, or conditionally. More flexible than declarations.

Hindi:
function expression expression का part के रूप में function create करता है, anonymous या named हो सकता है। variables को assign किया जाता है: const func = function() {}। function declarations की तरह hoisted नहीं। जब function value के रूप में needed हो, closures में, या conditionally तब इस्तेमाल किया जाता है। declarations से ज्यादा flexible।

Marathi:
function expression expression चा भाग म्हणून function create करते, anonymous किंवा named असू शकते.

SKILLHUB

variables ला assign केले जाते: const func = function() {}. function declarations प्रमाणे hoisted नाही. जेव्हा function value म्हणून needed असेल, closures मध्ये किंवा conditionally तेव्हा वापरले जाते. declarations पेक्षा अधिक flexible.

Analogy:
Like takeout food: function expression is prepared fresh when ordered, not pre-made like function declarations.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/function

---

✅ 113. What is the difference between a module and a library?

English:
Module is single file/unit of code with specific functionality, library is collection of modules providing broader functionality. Modules are imported/exported, libraries are installed and used. Libraries often contain multiple modules working together for specific purpose (e.g., React library contains many modules).

Hindi:
module single file/unit of code है जिसमें specific functionality होती है, library modules का collection है जो broader functionality provide करती है। modules import/export किए जाते हैं, libraries install और use की जाती हैं। libraries often multiple modules contain करती हैं जो specific purpose (e.g., React library) के लिए साथ काम करते हैं।

Marathi:
module single file/unit of code आहे ज्यात specific functionality असते, library modules चा collection आहे जी broader functionality पुरवते. modules import/export केली जातात, libraries install आणि use केली जातात. libraries often multiple modules समाविष्ट करतात जी specific purpose (e.g., React library) साठी एकत्र काम करतात.

Analogy:
Like books vs. library: module is one book with specific knowledge, library is collection of many books for comprehensive learning.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

---

✅ 114. How are JavaScript and ECMA Script related?

SKILLHUB

English:
ECMAScript is the specification/standard that defines JavaScript language. JavaScript is implementation of ECMAScript specification. ECMAScript evolves through versions (ES5, ES6/ES2015, etc.), browsers implement these specs. JavaScript adds host-specific features (DOM, BOM) beyond core ECMAScript.

Hindi:
ECMAScript specification/standard है जो JavaScript language को define करता है। JavaScript ECMAScript specification की implementation है। ECMAScript versions (ES5, ES6/ES2015, etc.) के माध्यम से evolve करता है, browsers इन specs को implement करते हैं। JavaScript core ECMAScript के अलावा host-specific features (DOM, BOM) add करता है।

Marathi:
ECMAScript specification/standard आहे जे JavaScript language ला define करते. JavaScript ECMAScript specification ची implementation आहे. ECMAScript versions (ES5, ES6/ES2015, etc.) द्वारे evolve करते, browsers या specs ला implement करतात. JavaScript core ECMAScript व्यतिरिक्त host-specific features (DOM, BOM) add करते.

Analogy:
Like recipe vs. actual dish: ECMAScript is the recipe (specification), JavaScript is the cooked meal (implementation).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Language_Resources

---

✅ 115. What are the different ways to delete a variable in JavaScript?

English:
delete operator removes object properties, not variables. Variables go out of scope naturally. For global variables: delete window.varName. For object properties: delete obj.prop. Cannot delete var/let/const declared variables. Use undefined assignment or scope management instead.

Hindi:
delete operator object properties को remove करता है, variables को नहीं। variables naturally out of scope हो जाते हैं। global variables के लिए: delete window.varName। object properties के लिए: delete obj.prop। var/let/const declared variables को delete नहीं कर सकते। इसके बजाय undefined assignment या scope management इस्तेमाल करें।

Marathi:
delete operator object properties ला remove करते, variables ला नाही. variables naturally out of scope होतात. global variables साठी: delete window.varName. object properties साठी: delete obj.prop.

SKILLHUB

var/let/const declared variables ला delete करू शकत नाही. त्याऐवजी undefined assignment किंवा scope management वापरा.

Analogy:
Like erasing whiteboard vs. leaving the room: delete removes properties from objects, but variables disappear when their room (scope) ends.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/delete

---

✅ 116. What are the different types of errors in JavaScript?

English:
SyntaxError: invalid syntax. ReferenceError: undefined variable access. TypeError: operation on wrong type. RangeError: number out of range. EvalError: eval() misuse (rare). URIError: encodeURI/decodeURI misuse. Custom errors with Error constructor.

Hindi:
syntaxError: invalid syntax। referenceError: undefined variable access। typeError: wrong type पर operation। rangeError: number out of range। evalError: eval() misuse (rare)। uriError: encodeURI/decodeURI misuse। error constructor के साथ custom errors।

Marathi:
syntaxError: invalid syntax. referenceError: undefined variable access. typeError: wrong type वर operation. rangeError: number out of range. evalError: eval() misuse (rare). uriError: encodeURI/decodeURI misuse. error constructor सोबत custom errors.

Analogy:
Like different warning lights on a car dashboard: each error type signals a specific kind of problem in your code.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error

---

✅ 117. What is the use of the blur function?

English:
blur() method removes focus from element, triggering blur event. Opposite of focus(). Used for form validation (validate on blur), hiding dropdowns, closing modals. Also event handler for blur events when elements lose focus.

SKILLHUB

Hindi:
blur() method element से focus remove करता है, blur event trigger करता है। focus() का opposite। form validation (blur पर validate), dropdowns hide करने, modals close करने के लिए इस्तेमाल किया जाता है। जब elements focus lose करते हैं तब blur events का event handler भी है।

Marathi:
blur() method element पासून focus remove करते, blur event trigger करते. focus() चे opposite. form validation (blur वर validate), dropdowns hide करण्यासाठी, modals close करण्यासाठी वापरले जाते. जेव्हा elements focus lose करतात तेव्हा blur events चा event handler आहे.

Analogy:
Like looking away: blur() makes an element "look away" by removing focus, just like when you stop staring at something.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/blur

---

✅ 118. What is the difference between an alert box and a confirmation box?

English:
alert() shows message with OK button, blocks execution until clicked. confirm() shows message with OK/Cancel buttons, returns boolean. alert() for notifications, confirm() for yes/no decisions. Both block UI, use sparingly in production.

Hindi:
alert() OK button के साथ message show करता है, clicked होने तक execution block करता है। confirm() OK/cancel buttons के साथ message show करता है, boolean रिटर्न करता है। alert() notifications के लिए, confirm() yes/no decisions के लिए। दोनों UI को block करते हैं, production में sparingly इस्तेमाल करें।

Marathi:
alert() OK button सोबत message show करते, clicked होईपर्यंत execution block करते. confirm() OK/cancel buttons सोबत message show करते, boolean परत करते. alert() notifications साठी, confirm() yes/no decisions साठी. दोन्ही UI ला block करतात, production मध्ये sparingly वापरा.

Analogy:
Like different conversation styles: alert is "Here's information" (one-way), confirm is "Do you agree?" (two-way).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Window/alert
- https://developer.mozilla.org/en-US/docs/Web/API/Window/confirm

SKILLHUB

✅ 119. What is prompt() in javascript?

English:
prompt() shows dialog with text input field, OK/Cancel buttons. Returns entered string or null if canceled. Used for simple user input. Blocks execution until response. Limited validation - use with caution, better to use custom forms.

Hindi:
prompt() text input field, OK/cancel buttons के साथ dialog show करता है। entered string या अगर canceled तो null रिटर्न करता है। simple user input के लिए इस्तेमाल किया जाता है। response तक execution block करता है। limited validation - caution के साथ इस्तेमाल करें, custom forms better हैं।

Marathi:
prompt() text input field, OK/cancel buttons सोबत dialog show करते. entered string किंवा canceled असल्यास null परत करते. simple user input साठी वापरले जाते. response पर्यंत execution block करते. limited validation - caution सोबत वापरा, custom forms better आहेत.

Analogy:
Like asking for someone's name at the door: prompt() asks a question and waits for typed response.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/API/Window/prompt

---

✅ 120. When to Use Internal and External JavaScript Code?

English:
External JS: reusable across pages, better performance (cached), cleaner HTML. Use for libraries, large scripts. Internal JS: page-specific logic, small scripts, when external not needed. Inline JS (in HTML): only for event handlers, avoid for larger code.

Hindi:
external JS: pages में reusable, better performance (cached), cleaner HTML। libraries, large scripts के लिए इस्तेमाल करें। internal JS: page-specific logic, small scripts, जब external की need न हो। inline JS (HTML में): सिर्फ event handlers के लिए, larger code के लिए avoid करें।

Marathi:
external JS: pages मध्ये reusable, better performance (cached), cleaner HTML. libraries, large scripts साठी वापरा. internal JS: page-specific logic, small scripts, जेव्हा external ची गरज नसेल. inline JS (HTML मध्ये): फक्त event handlers साठी, larger code साठी avoid करा.

Analogy:
Like organizing books: external JS is library books (shared), internal JS is personal notes (page-

SKILLHUB

specific).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript

✅ 121. What is npm?

English:
npm (Node Package Manager) is package manager for JavaScript, installs/manages dependencies. Comes with Node.js. Uses package.json for project dependencies. Provides command-line tools for package installation, version management, script running. Largest software registry.

Hindi:
npm (Node Package Manager) JavaScript के लिए package manager है, dependencies को install/manage करता है। node.js के साथ आता है। project dependencies के लिए package.json इस्तेमाल करता है। package installation, version management, script running के लिए command-line tools provide करता है। largest software registry।

Marathi:
npm (Node Package Manager) JavaScript साठी package manager आहे, dependencies install/manage करते. node.js सोबत येते. project dependencies साठी package.json वापरते. package installation, version management, script running साठी command-line tools पुरवते. largest software registry.

Analogy:
Like a grocery delivery service: npm delivers and manages all the "ingredients" (packages) your JavaScript project needs.

Learn more:

- https://docs.npmjs.com/

✅ 122. What is the name of the file which npm uses to identify the project and its dependencies?

English:
package.json file defines project metadata, dependencies, scripts. Created by npm init. Contains name, version, description, main file, scripts object, dependencies/devDependencies. npm uses this to install correct packages and run scripts.

Hindi:
package.json file project metadata, dependencies, scripts को define करता है। npm init द्वारा created।

SKILLHUB

name, version, description, main file, scripts object, dependencies/devDependencies contain करता है। npm इसको correct packages install करने और scripts run करने के लिए इस्तेमाल करता है।

Marathi:
package.json file project metadata, dependencies, scripts define करते. npm init द्वारे created. name, version, description, main file, scripts object, dependencies/devDependencies contain करते. npm हे correct packages install करण्यासाठी आणि scripts run करण्यासाठी वापरते.

Analogy:
Like a shopping list with instructions: package.json tells npm exactly what to buy and how to prepare the meal.

Learn more:

- https://docs.npmjs.com/cli/v9/configuring-npm/package-json

✅ 123. What is the difference between dependencies and devDependencies?

English:
dependencies: packages required for application to run in production. Installed with npm install. Listed in dependencies object. devDependencies: packages only needed during development (testing, building). Installed with npm install --save-dev. Not included in production builds.

Hindi:
dependencies: application को production में run करने के लिए required packages। npm install के साथ installed। dependencies object में listed। devDependencies: सिर्फ development के दौरान needed packages (testing, building)। npm install --save-dev के साथ installed। production builds में included नहीं।

Marathi:
dependencies: application ला production मध्ये run करण्यासाठी required packages. npm install सोबत installed. dependencies object मध्ये listed. devDependencies: फक्त development दरम्यान needed packages (testing, building). npm install --save-dev सोबत installed. production builds मध्ये included नाहीत.

Analogy:
Like ingredients vs. kitchen tools: dependencies are the food items (needed to serve the meal), devDependencies are the pots and pans (needed to prepare the meal).

Learn more:

- https://docs.npmjs.com/specifying-dependencies-and-devdependencies-in-a-package-json-file

---

SKILLHUB

✅ 124. What is a non-blocking function?

English:
Non-blocking function doesn't wait for operation to complete, allows other code to execute. Uses callbacks/promises/async-await. Common in I/O operations (file reading, network requests). Opposite of blocking functions. Enables scalable, responsive applications.

Hindi:
non-blocking function operation complete होने का wait नहीं करता, अन्य code को execute करने की अनुमति देता है। callbacks/promises/async-await इस्तेमाल करता है। I/O operations (file reading, network requests) में common है। blocking functions का opposite। scalable, responsive applications को enable करता है।

Marathi:
non-blocking function operation complete होण्याची वाट पाहत नाही, इतर code ला execute करण्यास परवानगी देतो. callbacks/promises/async-await वापरतो. I/O operations (file reading, network requests) मध्ये common आहे. blocking functions चे opposite. scalable, responsive applications सक्षम करतो.

Analogy:
Like texting while cooking: non-blocking functions let you handle multiple tasks simultaneously without waiting.

Learn more:

- https://nodejs.org/en/docs/guides/blocking-vs-non-blocking/

✅ 125. What is a blocking function?

English:
Blocking function waits for operation to complete before proceeding, halting execution. Common in synchronous file operations or CPU-intensive tasks. Can make applications unresponsive. Avoid in main thread. Use non-blocking alternatives for I/O operations.

Hindi:
blocking function operation complete होने से पहले proceed नहीं करता, execution को halt करता है। synchronous file operations या CPU-intensive tasks में common है। applications को unresponsive बना सकता है। main thread में avoid करें। I/O operations के लिए non-blocking alternatives इस्तेमाल करें।

Marathi:
blocking function operation complete होण्यापूर्वी proceed करत नाही, execution ला halt करते. synchronous file operations किंवा CPU-intensive tasks मध्ये common आहे. applications ला unresponsive बनवू शकते. main thread मध्ये avoid करा. I/O operations साठी non-blocking alternatives वापरा.

Analogy:
Like waiting in a long line: blocking functions make everything stop and wait, one thing at a time.

SKILLHUB

Learn more:

- https://nodejs.org/en/docs/guides/blocking-vs-non-blocking/

✅ 126. What is Function Composition?

English:
Function composition combines multiple functions into one, where output of one becomes input of next. Enables building complex operations from simple functions. Pure functional programming concept. Can be implemented manually or with libraries like Ramda, Lodash.

Hindi:
function composition multiple functions को एक में combine करता है, जहां एक का output अगले का input बनता है। simple functions से complex operations build करने को enable करता है। pure functional programming concept। manually implement किया जा सकता है या Ramda, Lodash जैसे libraries के साथ।

Marathi:
function composition multiple functions ला एकात combine करते, जिथे एकाचे output पुढच्या चे input बनते. simple functions पासून complex operations build करण्यास सक्षम करतो. pure functional programming concept. manually implement केले जाऊ शकते किंवा Ramda, Lodash सारख्या libraries सोबत.

Analogy:
Like assembling a car on a production line: each function adds a component, passing the partially built car to the next station.

Learn more:

- https://medium.com/javascript-scene/master-the-javascript-interview-what-is-function-composition-20dfb109a1a0

✅ 127. What is heap?

English:
Heap is memory area for dynamic memory allocation in JavaScript runtime. Stores objects, arrays, functions. Managed by garbage collector. Unlike stack (fixed size, fast), heap is flexible but slower. Memory leaks occur when objects not properly garbage collected.

Hindi:
heap JavaScript runtime में dynamic memory allocation के लिए memory area है। objects, arrays, functions को store करता है। garbage collector द्वारा managed। stack (fixed size, fast) के विपरीत, heap flexible है लेकिन slower। जब objects properly garbage collected नहीं होते तब memory leaks होते हैं।

SKILLHUB

Marathi:
heap JavaScript runtime मध्ये dynamic memory allocation साठी memory area आहे. objects, arrays, functions store करते. garbage collector द्वारे managed. stack (fixed size, fast) च्या विपरीत, heap flexible आहे पण slower. जेव्हा objects properly garbage collected होत नाहीत तेव्हा memory leaks होतात.

Analogy:
Like a storage warehouse: heap is where all the "stuff" (objects) gets stored when you don't know how much space you'll need.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Memory_Management

---

✅ 128. What is typed array in javascript?

English:
Typed arrays provide array-like views of binary data buffers with specific numeric types. Fixed size, better performance for numeric computations. Types: Int8Array, Uint8Array, Float32Array, etc. Used for WebGL, Canvas manipulation, file processing, cryptographic operations.

Hindi:
typed arrays binary data buffers के array-like views provide करते हैं specific numeric types के साथ। fixed size, numeric computations के लिए better performance। types: Int8Array, Uint8Array, Float32Array, etc। WebGL, Canvas manipulation, file processing, cryptographic operations के लिए इस्तेमाल किए जाते हैं।

Marathi:
typed arrays binary data buffers चे array-like views specific numeric types सोबत पुरवतात. fixed size, numeric computations साठी better performance. types: Int8Array, Uint8Array, Float32Array, etc. WebGL, Canvas manipulation, file processing, cryptographic operations साठी वापरले जातात.

Analogy:
Like specialized containers: typed arrays are purpose-built boxes for specific types of data (integers, floats) with known performance characteristics.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Typed_arrays

---

✅ 129. What is nodejs?

English:
Node.js is JavaScript runtime built on Chrome's V8 engine, enables server-side JavaScript execution.

SKILLHUB

Uses event-driven, non-blocking I/O model. Includes npm package manager. Used for web servers, APIs, real-time applications, command-line tools, desktop apps (with Electron).

Hindi:
node.js Chrome के V8 engine पर built JavaScript runtime है, server-side JavaScript execution को enable करता है। event-driven, non-blocking I/O model इस्तेमाल करता है। npm package manager include करता है। web servers, APIs, real-time applications, command-line tools, desktop apps (electron के साथ) के लिए इस्तेमाल किया जाता है।

Marathi:
node.js Chrome च्या V8 engine वर built JavaScript runtime आहे, server-side JavaScript execution सक्षम करते. event-driven, non-blocking I/O model वापरते. npm package manager समाविष्ट करते. web servers, APIs, real-time applications, command-line tools, desktop apps (electron सोबत) साठी वापरले जाते.

Analogy:
Like JavaScript growing up: Node.js lets JavaScript leave the browser and work on servers, handling files, databases, and network operations.

Learn more:

- https://nodejs.org/en/about/

✅ 130. What is a PWA?

English:
Progressive Web App combines web and native app features. Installable, works offline via service workers, push notifications, responsive design. Uses web technologies (HTML, CSS, JS) but provides app-like experience. Can be installed from browser, appears in app drawer.

Hindi:
progressive web app web और native app features को combine करता है। installable, service workers के माध्यम से offline काम करता है, push notifications, responsive design। web technologies (HTML, CSS, JS) इस्तेमाल करता है लेकिन app-like experience provide करता है। browser से install किया जा सकता है, app drawer में appear होता है।

Marathi:
progressive web app web आणि native app features combine करते. installable, service workers द्वारे offline काम करते, push notifications, responsive design. web technologies (HTML, CSS, JS) वापरते पण app-like experience पुरवते. browser मधून install केले जाऊ शकते, app drawer मध्ये दिसते.

Analogy:
Like a website that graduates to become an app: PWAs take websites and give them superpowers like offline access and installation.

SKILLHUB

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

---

✅ 131. What is nullish coalescing operator (??)?

English:
?? operator returns right operand when left is null or undefined, otherwise returns left operand. Unlike || (treats falsy values as false), ?? only treats null/undefined as "nullish". Useful for default values when 0, false, empty string are valid values.

Hindi:
?? operator right operand रिटर्न करता है जब left null या undefined है, otherwise left operand रिटर्न करता है। || (falsy values को false treat करता है) के विपरीत, ?? सिर्फ null/undefined को "nullish" treat करता है। जब 0, false, empty string valid values हैं तब default values के लिए useful।

Marathi:
?? operator right operand परत करतो जेव्हा left null किंवा undefined असेल, otherwise left operand परत करतो. || (falsy values ला false treat करते) च्या विपरीत, ?? फक्त null/undefined ला "nullish" treat करते. जेव्हा 0, false, empty string valid values असतील तेव्हा default values साठी useful.

Analogy:
Like a smarter "or" operator: ?? only fills in the blanks when something is truly missing (null/undefined), not just falsy.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing_operator

---

✅ 132. How do you get the status of a checkbox with javascript?

English:
Use .checked property: checkbox.checked returns true/false. For multiple checkboxes, use querySelectorAll, loop and check .checked. Can also listen to 'change' events. Set with checkbox.checked = true/false. Works for radio buttons too.

Hindi:
.checked property इस्तेमाल करें: checkbox.checked true/false रिटर्न करता है। multiple checkboxes के लिए, querySelectorAll इस्तेमाल करें, loop करें और .checked check करें। 'change' events को भी listen कर सकते हैं। checkbox.checked = true/false से set करें। radio buttons के लिए भी काम करता है।

SKILLHUB

Marathi:
.checked property वापरा: checkbox.checked true/false परत करतो. multiple checkboxes साठी, querySelectorAll वापरा, loop करा आणि .checked check करा. 'change' events ला listen करू शकता. checkbox.checked = true/false ने set करा. radio buttons साठीही काम करतो.

Analogy:
Like checking light switches: .checked tells you whether each checkbox "switch" is turned on (true) or off (false).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input/checkbox

---

✅ 133. What is the purpose of double tilde operator?

English:
Double tilde (~~) is bitwise NOT operator twice, effectively truncates decimal part of numbers. Faster than Math.floor() for positive numbers, but less readable. ~~x is equivalent to Math.trunc(x) for positive numbers. Not recommended for production code due to poor readability.

Hindi:
double tilde (~~) bitwise NOT operator twice है, effectively numbers की decimal part को truncate करता है। positive numbers के लिए Math.floor() से faster, लेकिन less readable। positive numbers के लिए ~~x Math.trunc(x) के equivalent है। poor readability की वजह से production code में recommended नहीं।

Marathi:
double tilde (~~) bitwise NOT operator twice आहे, effectively numbers ची decimal part truncate करते. positive numbers साठी Math.floor() पेक्षा faster, पण less readable. positive numbers साठी ~~x Math.trunc(x) चे equivalent आहे. poor readability मुळे production code मध्ये recommended नाही.

Analogy:
Like chopping off decimals with scissors: ~~ quickly removes the fractional part, but not as clean as proper rounding functions.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Bitwise_NOT

✅ 134. What are the different ways to debug JavaScript code?

English:
Browser dev tools (Chrome/Firefox), console.log/debugger statements, breakpoints, watch

SKILLHUB

expressions, call stack inspection. Node.js debugging with --inspect flag. VS Code debugger. Error messages and stack traces. Unit testing with console.assert().

Hindi:
browser dev tools (Chrome/Firefox), console.log/debugger statements, breakpoints, watch expressions, call stack inspection। node.js debugging --inspect flag के साथ। VS Code debugger। error messages और stack traces। console.assert() के साथ unit testing।

Marathi:
browser dev tools (Chrome/Firefox), console.log/debugger statements, breakpoints, watch expressions, call stack inspection. node.js debugging --inspect flag सोबत. VS Code debugger. error messages आणि stack traces. console.assert() सोबत unit testing.

Analogy:
Like detective work: debugging tools are your magnifying glass, fingerprint kit, and lab equipment to solve code mysteries.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/debugger

---

✅ 135. What is the difference between undeclared & undefined?

English:
Undeclared variable never declared, accessing throws ReferenceError. Undefined variable declared but not assigned value, has undefined value. Use typeof to safely check undeclared variables. Always declare variables to avoid ReferenceError.

Hindi:
undeclared variable कभी declared नहीं, access करने पर ReferenceError throw करता है। undefined variable declared है लेकिन value assign नहीं, undefined value है। undeclared variables को safely check करने के लिए typeof इस्तेमाल करें। ReferenceError से बचने के लिए हमेशा variables declare करें।

Marathi:
undeclared variable कधीही declared नाही, access केल्यावर ReferenceError throw करते. undefined variable declared आहे पण value assign नाही, undefined value आहे. undeclared variables ला safely check करण्यासाठी typeof वापरा. ReferenceError टाळण्यासाठी नेहमी variables declare करा.

Analogy:
Like missing vs. empty: undeclared is "that person doesn't exist" (error), undefined is "that person exists but has no information."

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/undefined

---

✅ 136. What would be the result of 2+5+"3"?

English:
Result is "73". Left-to-right evaluation: 2+5=7 (number addition), 7+"3"="73" (string concatenation). JavaScript coerces types based on operator. + operator concatenates when either operand is string. Remember type coercion rules.

Hindi:
result "73" है। left-to-right evaluation: 2+5=7 (number addition), 7+"3"="73" (string concatenation)। JavaScript operator के आधार पर types coerce करता है। जब कोई operand string है तब + operator concatenate करता है। type coercion rules को याद रखें।

Marathi:
result "73" आहे. left-to-right evaluation: 2+5=7 (number addition), 7+"3"="73" (string concatenation). JavaScript operator वर आधारित types coerce करते. जेव्हा कोणताही operand string असेल तेव्हा + operator concatenate करते. type coercion rules लक्षात ठेवा.

Analogy:
Like cooking measurements: 2 cups + 5 cups = 7 cups (addition), but 7 cups + "3" = "73" (mixing numbers with text gives text).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Addition

---

✅ 137. What is statically typed and dynamically typed language and is javascript a statically typed or a dynamically typed language?

English:
Statically typed: types checked at compile time (Java, C++), errors caught early. Dynamically typed: types checked at runtime (JavaScript, Python), more flexible but runtime errors possible. JavaScript is dynamically typed - variables can hold any type, types determined at runtime.

Hindi:
statically typed: compile time पर types checked (Java, C++), errors early caught। dynamically typed: runtime पर types checked (JavaScript, Python), ज्यादा flexible लेकिन runtime errors possible। JavaScript dynamically typed है - variables कोई भी type hold कर सकते हैं, types runtime पर determined।

SKILLHUB

Marathi:
statically typed: compile time वर types checked (Java, C++), errors early caught. dynamically typed: runtime वर types checked (JavaScript, Python), अधिक flexible पण runtime errors possible. JavaScript dynamically typed आहे - variables कोणताही type hold करू शकतात, types runtime वर determined.

Analogy:
Like dress codes: statically typed languages enforce strict dress codes at the door (compile time), dynamically typed languages let you change clothes whenever you want (runtime).

Learn more:

- https://developer.mozilla.org/en-US/docs/Glossary/Dynamic_typing

---

✅ 138. What is function currying?

English:
Function currying transforms multi-argument function into sequence of single-argument functions. Each call returns new function until all arguments provided. Enables partial application, function composition. Example: add(2)(3)(4) instead of add(2,3,4).

Hindi:
function currying multi-argument function को single-argument functions की sequence में transform करता है। हर call new function रिटर्न करता है जब तक सभी arguments provided नहीं होते। partial application, function composition को enable करता है। example: add(2)(3)(4) बजाय add(2,3,4) का।

Marathi:
function currying multi-argument function ला single-argument functions च्या sequence मध्ये transform करते. प्रत्येक call new function परत करतो जोपर्यंत सर्व arguments पुरवली जात नाहीत. partial application, function composition सक्षम करते. example: add(2)(3)(4) ऐवजी add(2,3,4) चा.

Analogy:
Like building a sandwich step by step: currying lets you add ingredients one at a time, creating specialized sandwich builders.

Learn more:

- https://javascript.info/currying-partials

✅ 139. What does delete do in JavaScript?

English:
delete operator removes property from object, returns true if successful. Cannot delete inherited

SKILLHUB

properties or non-configurable properties. For variables declared with var/let/const, delete doesn't work - they remain in scope. Only removes object own properties.

Hindi:
delete operator object से property को remove करता है, successful होने पर true रिटर्न करता है। inherited properties या non-configurable properties को delete नहीं कर सकता। var/let/const से declared variables के लिए delete काम नहीं करता - वे scope में remain करते हैं। सिर्फ object की own properties को remove करता है।

Marathi:
delete operator object पासून property ला remove करते, successful झाल्यास true परत करते. inherited properties किंवा non-configurable properties ला delete करू शकत नाही. var/let/const ने declared variables साठी delete काम करत नाही - ते scope मध्ये राहतात. फक्त object च्या own properties ला remove करते.

Analogy:
Like erasing from a whiteboard: delete removes your own marks, but not the permanent ones written by others (inherited properties).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/delete

---

✅ 140. How does control flow function in js play a role in asynchronous operation in javascript?

English:
JavaScript single-threaded, control flow managed by event loop and call stack. Async operations (setTimeout, promises) moved to Web APIs, callbacks queued in callback queue. Event loop continuously checks stack, moves callbacks when empty. Enables non-blocking async operations.

Hindi:
JavaScript single-threaded है, control flow event loop और call stack द्वारा managed। async operations (setTimeout, promises) Web APIs में move होते हैं, callbacks callback queue में queued। event loop continuously stack check करता है, जब empty होता है तब callbacks move करता है। non-blocking async operations को enable करता है।

Marathi:
JavaScript single-threaded आहे, control flow event loop आणि call stack द्वारे managed. async operations (setTimeout, promises) Web APIs मध्ये move होतात, callbacks callback queue मध्ये queued. event loop continuously stack check करते, जेव्हा empty असते तेव्हा callbacks move करते. non-blocking async operations सक्षम करते.

SKILLHUB

Analogy:
Like an air traffic controller: event loop manages the flow of operations, ensuring async flights (callbacks) land only when the runway (call stack) is clear.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/EventLoop

---

✅ 141. How can you share code between files?

English:
ES6 modules: export/import statements. CommonJS: module.exports/require() (Node.js). AMD: define()/require() (browsers). Bundlers like Webpack/Rollup combine files. Libraries: global variables (not recommended). Choose based on environment and build setup.

Hindi:
ES6 modules: export/import statements। CommonJS: module.exports/require() (node.js)। AMD: define()/require() (browsers)। webpack/rollup जैसे bundlers files को combine करते हैं। libraries: global variables (recommended नहीं)। environment और build setup के आधार पर चुनें।

Marathi:
ES6 modules: export/import statements. CommonJS: module.exports/require() (node.js). AMD: define()/require() (browsers). webpack/rollup सारखे bundlers files combine करतात. libraries: global variables (recommended नाही). environment आणि build setup वर आधारित निवडा.

Analogy:
Like sharing recipes: modules are organized recipe cards you can import, bundlers combine them into a complete cookbook.

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

---

✅ 142. What does the instanceof operator do?

English:
instanceof checks if object is instance of specific constructor/class. Returns true if object's prototype chain contains constructor's prototype. Works with inheritance. More reliable than constructor property. Use cautiously with cross-realm objects.

Hindi:
instanceof check करता है कि object specific constructor/class का instance है या नहीं। अगर object's

SKILLHUB

prototype chain में constructor का prototype है तो true रिटर्न करता है। inheritance के साथ काम करता है। constructor property से ज्यादा reliable। cross-realm objects के साथ cautiously इस्तेमाल करें।

Marathi:
instanceof check करते की object specific constructor/class चा instance आहे की नाही. जर object's prototype chain मध्ये constructor चा prototype असेल तर true परत करते. inheritance सोबत काम करते. constructor property पेक्षा अधिक reliable. cross-realm objects सोबत cautiously वापरा.

Analogy:
Like checking family lineage: instanceof asks "is this person a member of the Johnson family?" by checking the family tree (prototype chain).

Learn more:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/instanceof

---

SKILLHUB

ReactJS Basics by SKILLHUB - Day 10

✅ 143. What is ReactJS?

English:
ReactJS is a popular open-source JavaScript library developed by Facebook for building user interfaces, particularly web applications. It focuses on the "View" layer of MVC architecture and allows developers to create reusable UI components. React uses a component-based architecture and virtual DOM for efficient rendering.

Hindi:
ReactJS Facebook द्वारा विकसित एक लोकप्रिय open-source JavaScript library है जो user interfaces बनाने के लिए, विशेष रूप से web applications के लिए। यह MVC architecture के "View" layer पर focus करता है और developers को reusable UI components बनाने की अनुमति देता है। React component-based architecture और efficient rendering के लिए virtual DOM का इस्तेमाल करता है।

Marathi:
ReactJS Facebook द्वारे विकसित एक लोकप्रिय open-source JavaScript library आहे जी user interfaces तयार करण्यासाठी, विशेषतः web applications साठी. हे MVC architecture च्या "View" layer वर focus करते आणि developers ला reusable UI components तयार करण्यास परवानगी देते. React component-based architecture आणि efficient rendering साठी virtual DOM वापरते.

Analogy:
Like LEGO blocks: React provides modular building blocks (components) that you can assemble to create complex user interfaces, just like connecting LEGO pieces to build structures.

Learn more:

- https://reactjs.org/docs/getting-started.html
- https://react.dev/

✅ 144. What are the key features of ReactJS?

English:
Key features include: Component-based architecture, Virtual DOM, JSX syntax, Unidirectional data flow, Declarative programming, Reusable components, Strong community support, and excellent developer tools. React also supports server-side rendering and has a rich ecosystem of libraries.

Hindi:
key features में शामिल हैं: Component-based architecture, Virtual DOM, JSX syntax, Unidirectional data

SKILLHUB

flow, Declarative programming, Reusable components, Strong community support, और excellent developer tools। React server-side rendering को भी support करता है और libraries का rich ecosystem है।

Marathi:
key features मध्ये समाविष्ट आहेत: Component-based architecture, Virtual DOM, JSX syntax, Unidirectional data flow, Declarative programming, Reusable components, Strong community support आणि excellent developer tools. React server-side rendering लाही support करते आणि libraries चा rich ecosystem आहे.

Analogy:
Like a Swiss Army knife for web development: React comes with multiple tools (features) that solve different aspects of building modern web applications efficiently.

Learn more:

- https://reactjs.org/docs/thinking-in-react.html

✅ 145. What is the difference between ReactJS and other JavaScript frameworks/libraries?

English:
React is a library focused only on the View layer, while frameworks like Angular provide complete MVC architecture. React uses virtual DOM for efficient updates, has a steeper learning curve initially due to JSX, but offers more flexibility. React has a larger community and ecosystem compared to some frameworks.

Hindi:
React सिर्फ View layer पर focused library है, जबकि Angular जैसे frameworks complete MVC architecture provide करते हैं। React efficient updates के लिए virtual DOM का इस्तेमाल करता है, JSX की वजह से initially steeper learning curve है, लेकिन ज्यादा flexibility देता है। React कुछ frameworks की तुलना में larger community और ecosystem है।

Marathi:
React फक्त View layer वर focused library आहे, तर Angular सारखे frameworks complete MVC architecture पुरवतात. React efficient updates साठी virtual DOM वापरते, JSX मुळे initially steeper learning curve आहे, पण अधिक flexibility देते. React काही frameworks पेक्षा मोठी community आणि ecosystem आहे.

Analogy:
Like choosing between a specialized tool vs. a complete toolkit: React is a precision View-building tool, while other frameworks are comprehensive development kits.

Learn more:

- https://reactjs.org/docs/comparison-with-other-frameworks.html

SKILLHUB

✅ 146. What is JSX in ReactJS?

English:
JSX (JavaScript XML) is a syntax extension that allows writing HTML-like code within JavaScript. It makes React components more readable and expressive. JSX gets transpiled to React.createElement() calls. While not required, it's the standard way to define React component structure.

Hindi:
JSX (JavaScript XML) एक syntax extension है जो JavaScript के अंदर HTML-like code लिखने की अनुमति देता है। यह React components को ज्यादा readable और expressive बनाता है। JSX React.createElement() calls में transpile होता है। हालांकि required नहीं है, लेकिन यह React component structure define करने का standard तरीका है।

Marathi:
JSX (JavaScript XML) एक syntax extension आहे जे JavaScript च्या आत HTML-like code लिहिण्यास परवानगी देते. हे React components ला अधिक readable आणि expressive बनवते. JSX React.createElement() calls मध्ये transpile होते. जरी required नसले तरी, हे React component structure define करण्याचा standard मार्ग आहे.

Analogy:
Like writing HTML directly in your JavaScript: JSX lets you describe what your UI should look like using familiar HTML syntax, right inside your JavaScript code.

Learn more:

- https://reactjs.org/docs/jsx-in-depth.html

✅ 147. Explain the concept of virtual DOM in ReactJS.

English:
Virtual DOM is a lightweight JavaScript representation of the actual DOM. React maintains a virtual copy and compares it with the real DOM to identify changes. Only necessary updates are applied to the real DOM, improving performance. This diffing process is called reconciliation.

Hindi:
virtual DOM actual DOM का lightweight JavaScript representation है। React virtual copy maintain करता है और real DOM से compare करके changes identify करता है। सिर्फ necessary updates real DOM में apply होते हैं, जिससे performance improve होती है। यह diffing process reconciliation कहलाता है।

Marathi:
virtual DOM actual DOM चे lightweight JavaScript representation आहे. React virtual copy maintain करते आणि real DOM सोबत compare करून changes identify करते. फक्त necessary updates real DOM मध्ये apply होतात, ज्यामुळे performance सुधारते. हे diffing process reconciliation म्हणून ओळखले जाते.

SKILLHUB

Analogy:
Like having a shopping list draft: you plan all your changes in the virtual list first, then make only the necessary trips to the actual store (real DOM) to execute them efficiently.

Learn more:

- https://reactjs.org/docs/faq-internals.html
- https://reactjs.org/docs/reconciliation.html

✅ 148. What are the components in ReactJS?

English:
Components are the building blocks of React applications, reusable pieces of code that return JSX to define UI structure. Components can be class-based or functional, accept props for customization, and manage their own state. They promote code reusability, maintainability, and separation of concerns.

Hindi:
components React applications के building blocks हैं, reusable pieces of code जो UI structure define करने के लिए JSX return करते हैं। components class-based या functional हो सकते हैं, customization के लिए props accept करते हैं, और अपना state manage करते हैं। वे code reusability, maintainability, और separation of concerns को promote करते हैं।

Marathi:
components React applications ची building blocks आहेत, reusable pieces of code जी UI structure define करण्यासाठी JSX return करतात. components class-based किंवा functional असू शकतात, customization साठी props accept करतात आणि स्वतःचा state manage करतात. ते code reusability, maintainability आणि separation of concerns ला promote करतात.

Analogy:
Like LEGO bricks in construction: each component is a specialized brick that connects with others to build complex user interfaces, and can be reused in different structures.

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 149. What is the significance of state in ReactJS?

English:
State represents data that can change over time and affects component rendering. It's managed

SKILLHUB

internally by components and triggers re-renders when updated. State enables dynamic, interactive UIs. In class components, state is an object; in functional components, it's managed via useState hook.

Hindi:
state ऐसे data को represent करता है जो time के साथ change हो सकता है और component rendering को affect करता है। यह components द्वारा internally managed किया जाता है और update होने पर re-renders trigger करता है। state dynamic, interactive UIs को enable करता है। class components में state object है; functional components में useState hook के माध्यम से managed किया जाता है।

Marathi:
state अशा data ला represent करते जे time सोबत change होऊ शकते आणि component rendering ला affect करते. हे components द्वारे internally managed केले जाते आणि update झाल्यावर re-renders trigger करते. state dynamic, interactive UIs सक्षम करते. class components मध्ये state object आहे; functional components मध्ये useState hook द्वारे managed केले जाते.

Analogy:
Like a component's memory: state stores information that changes based on user interactions, just like how your mood (state) changes throughout the day based on experiences.

Learn more:

- https://reactjs.org/docs/state-and-lifecycle.html

✅ 150. What is the difference between props and state?

English:
Props are read-only data passed from parent to child components, used for component communication. State is mutable data managed within a component, used for internal component logic. Props flow downward, state is local. Props changes trigger re-renders, state changes also trigger re-renders but are controlled by the component.

Hindi:
props parent से child components को pass किए गए read-only data हैं, component communication के लिए इस्तेमाल किए जाते हैं। state एक component के अंदर managed mutable data है, internal component logic के लिए इस्तेमाल किया जाता है। props downward flow करते हैं, state local है। props changes re-renders trigger करते हैं, state changes भी re-renders trigger करते हैं लेकिन component द्वारा controlled होते हैं।

Marathi:
props parent पासून child components ला pass केलेले read-only data आहेत, component communication साठी वापरले जातात. state एका component च्या आत managed mutable data आहे, internal component logic साठी वापरले जाते. props downward flow करतात, state local आहे. props changes re-renders trigger करतात, state changes ही re-renders trigger करतात पण component द्वारे controlled असतात.

SKILLHUB

Analogy:
Like receiving gifts vs. personal belongings: props are gifts you receive from others (can't change), state is your personal stuff (you can modify).

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 151. What is the purpose of the "render" method in ReactJS?

English:
The render() method returns the JSX that defines a component's UI structure. It's called whenever the component needs to update. In class components, it's required and should be pure (no side effects). React uses the returned JSX to create or update the virtual DOM.

Hindi:
render() method JSX return करता है जो component की UI structure define करता है। यह तब called होता है जब component को update की need होती है। class components में यह required है और pure होना चाहिए (no side effects)। React returned JSX का इस्तेमाल virtual DOM create या update करने के लिए करता है।

Marathi:
render() method JSX return करते जे component ची UI structure define करते. हे जेव्हा called होते जेव्हा component ला update ची गरज असते. class components मध्ये हे required आहे आणि pure असावे (no side effects). React returned JSX चा वापर virtual DOM create किंवा update करण्यासाठी करते.

Analogy:
Like a blueprint: the render method provides the architectural plan that React uses to construct and maintain the actual building (UI).

Learn more:

- https://reactjs.org/docs/react-component.html

✅ 152. What is a higher-order component (HOC) in ReactJS?

English:
A HOC is a function that takes a component and returns an enhanced component with additional functionality. HOCs enable code reuse, cross-cutting concerns like logging, authentication, or styling. They're pure functions that don't modify the original component but wrap it with new capabilities.

SKILLHUB

Hindi:
HOC एक function है जो component लेता है और additional functionality के साथ enhanced component return करता है। HOCs code reuse, cross-cutting concerns जैसे logging, authentication, या styling को enable करते हैं। वे pure functions हैं जो original component को modify नहीं करते लेकिन इसे new capabilities से wrap करते हैं।

Marathi:
HOC एक function आहे जे component घेते आणि additional functionality सोबत enhanced component return करते. HOCs code reuse, cross-cutting concerns जसे logging, authentication किंवा styling सक्षम करतात. ते pure functions आहेत जी original component ला modify करत नाहीत पण त्याला new capabilities ने wrap करतात.

Analogy:
Like adding superpowers to regular people: HOCs take a normal component and give it special abilities (like flying, invisibility) without changing the original person.

Learn more:

- https://reactjs.org/docs/higher-order-components.html

✅ 153. What is the purpose of the "key" prop in ReactJS?

English:
The key prop helps React identify which items in a list have changed, been added, or removed. It gives elements a stable identity across renders. Keys should be unique among siblings and stable (not array index). Proper keys enable efficient reconciliation and prevent UI bugs in dynamic lists.

Hindi:
key prop React को help करता है identify करने में कि list में कौन से items change हुए हैं, add हुए हैं, या remove हुए हैं। यह elements को renders के across stable identity देता है। keys siblings के बीच unique और stable होने चाहिए (array index नहीं)। proper keys efficient reconciliation को enable करते हैं और dynamic lists में UI bugs को prevent करते हैं।

Marathi:
key prop React ला help करते की list मध्ये कोणती items change झाली आहेत, add झाली आहेत किंवा remove झाली आहेत हे identify करण्यासाठी. हे elements ला renders च्या across stable identity देते. keys siblings दरम्यान unique आणि stable असावेत (array index नाही). proper keys efficient reconciliation सक्षम करतात आणि dynamic lists मध्ये UI bugs prevent करतात.

Analogy:
Like student ID cards: keys give each list item a unique identifier so React knows exactly who is who, even when the class roster changes.

Learn more:

SKILLHUB

- https://reactjs.org/docs/lists-and-keys.html

✅ 154. Explain the concept of controlled and uncontrolled components in ReactJS.

English:
Controlled components have their value controlled by React state, changes trigger state updates via onChange. Uncontrolled components maintain their own internal state, accessed via ref. Controlled components are preferred for validation and dynamic behavior, uncontrolled for simple cases or integrating with non-React code.

Hindi:
controlled components का value React state द्वारा controlled होता है, changes state updates trigger करते हैं onChange के माध्यम से। uncontrolled components अपना internal state maintain करते हैं, ref के माध्यम से accessed किए जाते हैं। controlled components validation और dynamic behavior के लिए preferred हैं, uncontrolled simple cases या non-React code के साथ integration के लिए।

Marathi:
controlled components चे value React state द्वारे controlled असते, changes state updates trigger करतात onChange द्वारे. uncontrolled components स्वतःचे internal state maintain करतात, ref द्वारे accessed केले जातात. controlled components validation आणि dynamic behavior साठी preferred आहेत, uncontrolled simple cases किंवा non-React code सोबत integration साठी.

Analogy:
Like parenting styles: controlled components are like helicopter parents (React controls everything), uncontrolled components are like free-range kids (manage themselves, React just checks in occasionally).

Learn more:

- https://reactjs.org/docs/forms.html

✅ 155. What is the purpose of the "setState" method in ReactJS and how does it work?

English:
setState updates a component's state and triggers re-rendering. It can take an object or function (for state based on previous state). React batches multiple setState calls for performance. It's asynchronous, so don't rely on immediate state access. For functional components, useState hook provides similar functionality.

SKILLHUB

Hindi:
setState component के state को update करता है और re-rendering trigger करता है। यह object या function (previous state पर based state के लिए) ले सकता है। React performance के लिए multiple setState calls को batch करता है। यह asynchronous है, इसलिए immediate state access पर rely न करें। functional components के लिए, useState hook similar functionality provide करता है।

Marathi:
setState component च्या state ला update करते आणि re-rendering trigger करते. हे object किंवा function घेऊ शकते (previous state वर based state साठी). React performance साठी multiple setState calls ला batch करते. हे asynchronous आहे, म्हणून immediate state access वर rely करू नका. functional components साठी, useState hook similar functionality पुरवते.

Analogy:
Like updating a shopping list: setState tells React "hey, the list changed, please update the display" and React handles the rest asynchronously.

Learn more:

- https://reactjs.org/docs/state-and-lifecycle.html

✅ 156. What is the significance of the "React.Fragment" component?

English:
React.Fragment allows grouping multiple elements without adding extra DOM nodes. It prevents unnecessary wrapper divs that can break CSS layouts. Fragments can have keys for lists. It's lighter than div wrappers and improves semantic structure.

Hindi:
React.Fragment extra DOM nodes add किए बिना multiple elements को group करने की अनुमति देता है। यह unnecessary wrapper divs को prevent करता है जो CSS layouts को break कर सकते हैं। Fragments lists के लिए keys have कर सकते हैं। यह div wrappers से lighter है और semantic structure को improve करता है।

Marathi:
React.Fragment extra DOM nodes add न करता multiple elements ला group करण्यास परवानगी देतो. हे unnecessary wrapper divs prevent करते जी CSS layouts ला break करू शकतात. Fragments lists साठी keys have करू शकतात. हे div wrappers पेक्षा lighter आहे आणि semantic structure सुधारते.

Analogy:
Like invisible packaging: Fragment groups items together for React's purposes but leaves no visible trace in the final DOM, unlike a wrapper div.

Learn more:

SKILLHUB

- https://reactjs.org/docs/fragments.html

✅ 157. How does React handle event handling?

English:
React uses synthetic events (wrapper around native events) for cross-browser compatibility. Events are camelCase (onClick vs onclick), passed as functions. React automatically binds this in class components. Events are pooled for performance. Use preventDefault() to stop default behavior, stopPropagation() to prevent bubbling.

Hindi:
React cross-browser compatibility के लिए synthetic events (native events का wrapper) इस्तेमाल करता है। events camelCase हैं (onClick vs onclick), functions के रूप में passed किए जाते हैं। React class components में automatically this bind करता है। events performance के लिए pooled हैं। default behavior stop करने के लिए preventDefault() इस्तेमाल करें, bubbling prevent करने के लिए stopPropagation()।

Marathi:
React cross-browser compatibility साठी synthetic events (native events चा wrapper) वापरते. events camelCase आहेत (onClick vs onclick), functions म्हणून passed केले जातात. React class components मध्ये automatically this bind करते. events performance साठी pooled आहेत. default behavior stop करण्यासाठी preventDefault() वापरा, bubbling prevent करण्यासाठी stopPropagation().

Analogy:
Like a translator at a conference: React's synthetic events translate browser-specific event handling into a consistent, cross-browser language that works everywhere.

Learn more:

- https://reactjs.org/docs/handling-events.html

SKILLHUB

ReactJS Hooks & State by SKILLHUB - Day 11

✅ 158. What is the concept of "everything is a component" in ReactJS?

English:
React follows a component-based architecture where UI is broken into reusable, composable pieces. Everything from buttons to entire pages can be components. Components can contain other components, creating a tree structure. This promotes reusability, maintainability, and separation of concerns.

Hindi:
React component-based architecture follow करता है जहां UI reusable, composable pieces में break होता है। buttons से entire pages तक सब कुछ components हो सकते हैं। components दूसरे components contain कर सकते हैं, tree structure create करते हैं। यह reusability, maintainability, और separation of concerns को promote करता है।

Marathi:
React component-based architecture अनुसरण करते जेथे UI reusable, composable pieces मध्ये break होते. buttons पासून entire pages पर्यंत सर्व काही components असू शकतात. components इतर components contain करू शकतात, tree structure तयार करतात. हे reusability, maintainability आणि separation of concerns ला promote करते.

Analogy:
Like Russian nesting dolls: everything is a component, and components can contain smaller components, creating a hierarchical structure just like nested dolls.

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 159. What is React Router and why is it used?

English:
React Router is a library for handling navigation and routing in React applications. It enables single-page application (SPA) navigation without full page reloads. It provides components like BrowserRouter, Route, Link, and Switch for declarative routing. It synchronizes UI with URL, enabling bookmarking and browser history.

SKILLHUB

Hindi:
React Router React applications में navigation और routing handle करने के लिए एक library है। यह full page reloads के बिना single-page application (SPA) navigation को enable करता है। यह declarative routing के लिए BrowserRouter, Route, Link, और Switch जैसे components provide करता है। यह UI को URL के साथ synchronize करता है, bookmarking और browser history को enable करता है।

Marathi:
React Router React applications मध्ये navigation आणि routing handle करण्यासाठी एक library आहे. हे full page reloads शिवाय single-page application (SPA) navigation सक्षम करते. हे declarative routing साठी BrowserRouter, Route, Link आणि Switch सारखे components पुरवते. हे UI ला URL सोबत synchronize करते, bookmarking आणि browser history सक्षम करते.

Analogy:
Like a GPS for your app: React Router helps navigate between different "locations" (routes) in your application without leaving the app, just like GPS guides you through different places.

Learn more:

- https://reactrouter.com/

✅ 160. What is a Pure Component in ReactJS?

English:
PureComponent is a class component that implements shouldComponentUpdate with shallow prop and state comparison. It prevents unnecessary re-renders when props/state haven't changed. It's a performance optimization. For functional components, React.memo provides similar functionality.

Hindi:
PureComponent एक class component है जो shallow prop और state comparison के साथ shouldComponentUpdate implement करता है। यह unnecessary re-renders को prevent करता है जब props/state change नहीं हुए हैं। यह एक performance optimization है। functional components के लिए, React.memo similar functionality provide करता है।

Marathi:
PureComponent एक class component आहे जे shallow prop आणि state comparison सोबत shouldComponentUpdate implement करते. हे unnecessary re-renders prevent करते जेव्हा props/state change झाले नाहीत. हे एक performance optimization आहे. functional components साठी, React.memo similar functionality पुरवते.

Analogy:
Like a smart thermostat: PureComponent only "reacts" (re-renders) when there's an actual change in temperature (props/state), not for every minor fluctuation.

Learn more:

SKILLHUB

- https://reactjs.org/docs/react-api.html

✅ 161. What are React Hooks and why were they introduced?

English:
Hooks are functions that let you use state and other React features in functional components. Introduced in React 16.8, they solve issues like code reuse, complex class components, and understanding "this" binding. Common hooks include useState, useEffect, useContext, useReducer, and custom hooks.

Hindi:
Hooks functions हैं जो आपको functional components में state और other React features use करने देते हैं। React 16.8 में introduced, वे code reuse, complex class components, और "this" binding को समझने जैसे issues को solve करते हैं। common hooks में useState, useEffect, useContext, useReducer, और custom hooks शामिल हैं।

Marathi:
Hooks functions आहेत जी तुम्हाला functional components मध्ये state आणि other React features वापरण्यास परवानगी देतात. React 16.8 मध्ये introduced, ते code reuse, complex class components आणि "this" binding समजून घेण्यासारख्या issues सोडवतात. common hooks मध्ये useState, useEffect, useContext, useReducer आणि custom hooks समाविष्ट आहेत.

Analogy:
Like adding superpowers to functional components: Hooks give functional components the abilities (state, lifecycle) that were previously only available to class components.

Learn more:

- https://reactjs.org/docs/hooks-intro.html

✅ 162. Can JSX be directly readable by browsers?

English:
No, browsers cannot directly read JSX. JSX must be transpiled to JavaScript using tools like Babel. Babel converts JSX syntax into React.createElement() calls that browsers understand. This transpilation happens during the build process before deployment.

Hindi:
नहीं, browsers JSX को directly read नहीं कर सकते। JSX को Babel जैसे tools का इस्तेमाल करके JavaScript में transpile करना होता है। Babel JSX syntax को React.createElement() calls में convert करता है जिन्हें browsers समझते हैं। यह transpilation deployment से पहले build process के दौरान होती है।

SKILLHUB

Marathi:
नाही, browsers JSX ला directly read करू शकत नाहीत. JSX ला Babel सारख्या tools वापरून JavaScript मध्ये transpile करावे लागते. Babel JSX syntax ला React.createElement() calls मध्ये convert करते जे browsers समजतात. हे transpilation deployment आधी build process दरम्यान होते.

Analogy:
Like translating a foreign language: JSX is a language browsers don't understand, so Babel translates it to JavaScript (a language browsers do understand) before it reaches the browser.

Learn more:

- https://reactjs.org/docs/jsx-in-depth.html

✅ 163. What are the naming conventions for React components?

English:
React components should use PascalCase (capitalized first letter). File names typically match component names. Examples: MyComponent, UserProfile, NavigationBar. This convention helps distinguish components from regular functions and variables, which use camelCase.

Hindi:
React components को PascalCase (capitalized first letter) use करना चाहिए। file names typically component names से match करते हैं। examples: MyComponent, UserProfile, NavigationBar। यह convention components को regular functions और variables से distinguish करने में help करता है, जो camelCase use करते हैं।

Marathi:
React components ला PascalCase (capitalized first letter) वापरावे. file names typically component names शी match करतात. examples: MyComponent, UserProfile, NavigationBar. हे convention components ला regular functions आणि variables पासून distinguish करण्यात help करते, जे camelCase वापरतात.

Analogy:
Like naming conventions for people: React components are like proper nouns (capitalized), while regular functions are like common nouns (camelCase).

Learn more:

- https://reactjs.org/docs/jsx-in-depth.html

✅ 164. What is useEffect hook in ReactJS?

SKILLHUB

English:
useEffect handles side effects in functional components, like data fetching, subscriptions, or DOM manipulation. It runs after render and can return a cleanup function. It accepts a dependency array to control when it runs. Empty array means run once on mount, no array means run on every render.

Hindi:
useEffect functional components में side effects handle करता है, जैसे data fetching, subscriptions, या DOM manipulation। यह render के बाद run होता है और cleanup function return कर सकता है। यह dependency array accept करता है जो control करता है कि यह कब run होगा। empty array का मतलब mount पर एक बार run, no array का मतलब हर render पर run।

Marathi:
useEffect functional components मध्ये side effects handle करते, जसे data fetching, subscriptions किंवा DOM manipulation. हे render नंतर run होते आणि cleanup function return करू शकते. हे dependency array accept करते जे control करते की ते कधी run होईल. empty array चा अर्थ mount वर एकदा run, no array चा अर्थ प्रत्येक render वर run.

Analogy:
Like a housekeeper: useEffect runs after you've set up the room (render), handles maintenance tasks (side effects), and can clean up when you leave (cleanup function).

Learn more:

- https://reactjs.org/docs/hooks-effect.html

✅ 165. How does setState work in ReactJS?

English:
setState updates component state and schedules a re-render. It can accept an object or a function (for state based on previous state). React batches multiple setState calls for performance. It's asynchronous, so state updates may not be immediate. In functional components, useState provides similar functionality.

Hindi:
setState component state को update करता है और re-render schedule करता है। यह object या function (previous state पर based state के लिए) accept कर सकता है। React performance के लिए multiple setState calls को batch करता है। यह asynchronous है, इसलिए state updates immediate नहीं हो सकते। functional components में, useState similar functionality provide करता है।

Marathi:
setState component state ला update करते आणि re-render schedule करते. हे object किंवा function (previous state वर based state साठी) accept करू शकते. React performance साठी multiple setState calls

SKILLHUB

ला batch करते. हे asynchronous आहे, म्हणून state updates immediate असू शकत नाहीत. functional components मध्ये, useState similar functionality पुरवते.

Analogy:
Like updating a shopping list: setState tells React "update this item" and React batches all updates together before making a single trip to the store (re-render).

Learn more:

- https://reactjs.org/docs/state-and-lifecycle.html

✅ 166. What is the "children" prop in ReactJS?

English:
The children prop is a special prop that contains content passed between opening and closing component tags. It allows components to wrap and render arbitrary content. Children can be strings, elements, arrays, or functions. It enables composition patterns and flexible component APIs.

Hindi:
children prop एक special prop है जिसमें opening और closing component tags के बीच passed content होता है। यह components को wrap करने और arbitrary content render करने की अनुमति देता है। children strings, elements, arrays, या functions हो सकते हैं। यह composition patterns और flexible component APIs को enable करता है।

Marathi:
children prop एक special prop आहे ज्यामध्ये opening आणि closing component tags दरम्यान passed content असते. हे components ला wrap करण्यास आणि arbitrary content render करण्यास परवानगी देते. children strings, elements, arrays किंवा functions असू शकतात. हे composition patterns आणि flexible component APIs सक्षम करते.

Analogy:
Like a gift box: the children prop is whatever you put inside the box (component), and the component decides how to display or use that content.

Learn more:

- https://reactjs.org/docs/composition-vs-inheritance.html

✅ 167. What is prop drilling in ReactJS?

English:
Prop drilling occurs when props are passed through multiple component layers that don't need them, just to reach a deeply nested component. It makes code harder to maintain and creates unnecessary

SKILLHUB

coupling. Solutions include Context API, state management libraries (Redux), or component composition.

Hindi:
prop drilling तब होता है जब props multiple component layers के through pass किए जाते हैं जिन्हें उनकी जरूरत नहीं है, सिर्फ deeply nested component तक पहुंचने के लिए। यह code को maintain करना harder बनाता है और unnecessary coupling create करता है। solutions में Context API, state management libraries (Redux), या component composition शामिल हैं।

Marathi:
prop drilling तेव्हा होते जेव्हा props multiple component layers द्वारे pass केले जातात ज्यांना त्यांची गरज नसते, फक्त deeply nested component पर्यंत पोहोचण्यासाठी. हे code ला maintain करणे कठीण बनवते आणि unnecessary coupling तयार करते. solutions मध्ये Context API, state management libraries (Redux) किंवा component composition समाविष्ट आहेत.

Analogy:
Like passing a message through many people: prop drilling is like telling person A to tell person B to tell person C... just to deliver a message to person Z, when you could have just told person Z directly.

Learn more:

- https://reactjs.org/docs/context.html

✅ 168. What is the difference between class components and functional components?

English:
Class components use ES6 classes, have lifecycle methods, use this keyword, and manage state with this.state. Functional components are simpler functions that return JSX, use hooks for state/lifecycle, and are easier to test and understand. Functional components are now preferred, but class components still work.

Hindi:
class components ES6 classes use करते हैं, lifecycle methods have करते हैं, this keyword use करते हैं, और this.state के साथ state manage करते हैं। functional components simpler functions हैं जो JSX return करते हैं, state/lifecycle के लिए hooks use करते हैं, और test और understand करना easier हैं। functional components अब preferred हैं, लेकिन class components अभी भी work करते हैं।

Marathi:
class components ES6 classes वापरतात, lifecycle methods असतात, this keyword वापरतात आणि this.state सोबत state manage करतात. functional components simpler functions आहेत जी JSX return करतात, state/lifecycle साठी hooks वापरतात आणि test आणि understand करणे सोपे आहे. functional components आता preferred आहेत, पण class components अजूनही work करतात.

SKILLHUB

Analogy:
Like different writing styles: class components are like formal essays (structured, verbose), functional components are like concise notes (simple, direct).

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 169. How can you optimize React component performance?

English:
Optimization techniques include: React.memo for functional components, PureComponent for class components, useMemo and useCallback hooks, code splitting, lazy loading, virtualizing long lists, avoiding inline functions/objects in render, and proper key usage. Profile with React DevTools to identify bottlenecks.

Hindi:
optimization techniques में शामिल हैं: functional components के लिए React.memo, class components के लिए PureComponent, useMemo और useCallback hooks, code splitting, lazy loading, long lists को virtualizing, render में inline functions/objects avoid करना, और proper key usage। bottlenecks identify करने के लिए React DevTools के साथ profile करें।

Marathi:
optimization techniques मध्ये समाविष्ट आहेत: functional components साठी React.memo, class components साठी PureComponent, useMemo आणि useCallback hooks, code splitting, lazy loading, long lists ला virtualizing, render मध्ये inline functions/objects avoid करणे आणि proper key usage. bottlenecks identify करण्यासाठी React DevTools सोबत profile करा.

Analogy:
Like tuning a car: performance optimization involves multiple tweaks (memoization, code splitting) to make your React app run faster and smoother.

Learn more:

- https://reactjs.org/docs/optimizing-performance.html

✅ 170. When should you use class components instead of functional components?

English:
Class components are still valid but generally not recommended for new code. Use them if: you need

SKILLHUB

error boundaries (currently only class components), working with legacy codebases, or specific lifecycle methods not available in hooks. Otherwise, prefer functional components with hooks.

Hindi:
class components अभी भी valid हैं लेकिन generally new code के लिए recommended नहीं हैं। उनका इस्तेमाल करें अगर: आपको error boundaries की जरूरत है (currently only class components), legacy codebases के साथ काम कर रहे हैं, या specific lifecycle methods hooks में available नहीं हैं। otherwise, hooks के साथ functional components prefer करें।

Marathi:
class components अजूनही valid आहेत पण generally new code साठी recommended नाहीत. ते वापरा जर: तुम्हाला error boundaries ची गरज आहे (currently only class components), legacy codebases सोबत काम करत आहात किंवा specific lifecycle methods hooks मध्ये available नाहीत. otherwise, hooks सोबत functional components prefer करा.

Analogy:
Like choosing between old and new tools: class components are like older, heavier tools that still work, but functional components are the modern, lighter alternatives that do the same job better.

Learn more:

- https://reactjs.org/docs/react-component.html

✅ 171. What is form handling in ReactJS?

English:
Form handling involves managing form inputs, validation, and submission. Controlled components use state to manage input values via onChange handlers. Uncontrolled components use refs. Common patterns include validation, error messages, disabled states, and handling submission. Libraries like Formik or React Hook Form can simplify form management.

Hindi:
form handling form inputs, validation, और submission manage करने को involve करता है। controlled components state use करते हैं onChange handlers के माध्यम से input values manage करने के लिए। uncontrolled components refs use करते हैं। common patterns में validation, error messages, disabled states, और submission handling शामिल हैं। Formik या React Hook Form जैसी libraries form management को simplify कर सकती हैं।

Marathi:
form handling form inputs, validation आणि submission manage करणे समाविष्ट करते. controlled components state वापरतात onChange handlers द्वारे input values manage करण्यासाठी. uncontrolled components refs वापरतात. common patterns मध्ये validation, error messages, disabled states आणि

SKILLHUB

submission handling समाविष्ट आहेत. Formik किंवा React Hook Form सारख्या libraries form management सुलभ करू शकतात.

Analogy:
Like managing a restaurant order form: form handling involves collecting customer input, validating it (correct items, quantities), and processing the order (submission).

Learn more:

- https://reactjs.org/docs/forms.html

✅ 172. What is the Flux architectural pattern in ReactJS?

English:
Flux is a unidirectional data flow pattern for React applications. Data flows in one direction: Actions → Dispatcher → Store → View. It helps manage complex state and makes data flow predictable. Redux is a popular implementation of Flux principles. It's useful for large applications with complex state management needs.

Hindi:
Flux React applications के लिए unidirectional data flow pattern है। data एक direction में flow करता है: Actions → Dispatcher → Store → View। यह complex state manage करने में help करता है और data flow को predictable बनाता है। Redux Flux principles का popular implementation है। यह complex state management needs के साथ large applications के लिए useful है।

Marathi:
Flux React applications साठी unidirectional data flow pattern आहे. data एका दिशेने flow करते: Actions → Dispatcher → Store → View. हे complex state manage करण्यात help करते आणि data flow ला predictable बनवते. Redux Flux principles चे popular implementation आहे. हे complex state management needs सोबत large applications साठी उपयुक्त आहे.

Analogy:
Like a one-way street: Flux ensures data flows in one direction only, making it easier to track where data comes from and where it goes, preventing traffic jams (state conflicts).

Learn more:

- https://reactjs.org/blog/2014/05/06/flux.html

✅ 173. What is Context in ReactJS and when should it be used?

SKILLHUB

English:
Context provides a way to pass data through the component tree without prop drilling. It's useful for global data like themes, user authentication, or language preferences. Use Context for data that many components need, but avoid overusing it as it can make components harder to test and reason about.

Hindi:
Context prop drilling के बिना component tree के through data pass करने का तरीका provide करता है। यह global data जैसे themes, user authentication, या language preferences के लिए useful है। Context का इस्तेमाल उस data के लिए करें जिसकी जरूरत many components को है, लेकिन इसे overuse करने से avoid करें क्योंकि यह components को test और reason करना harder बना सकता है।

Marathi:
Context prop drilling शिवाय component tree द्वारे data pass करण्याचा मार्ग पुरवते. हे global data जसे themes, user authentication किंवा language preferences साठी उपयुक्त आहे. Context चा वापर अशा data साठी करा ज्याची गरज अनेक components ला आहे, पण ते overuse करण्यापासून टाळा कारण हे components ला test आणि reason करणे कठीण बनवू शकते.

Analogy:
Like a public announcement system: Context broadcasts information to all components that need it, without having to pass messages through every intermediate person (component).

Learn more:

- https://reactjs.org/docs/context.html

✅ 174. What are Portals in ReactJS?

English:
Portals allow rendering children into a DOM node outside the parent component's hierarchy. Useful for modals, tooltips, or dropdowns that need to break out of container overflow/z-index constraints. Created using ReactDOM.createPortal(child, container). The component tree structure remains, but DOM placement changes.

Hindi:
Portals parent component की hierarchy के बाहर DOM node में children render करने की अनुमति देते हैं। modals, tooltips, या dropdowns के लिए useful जिन्हें container overflow/z-index constraints से break out करने की जरूरत है। ReactDOM.createPortal(child, container) का इस्तेमाल करके created। component tree structure बनी रहती है, लेकिन DOM placement change होता है।

Marathi:
Portals parent component च्या hierarchy च्या बाहेर DOM node मध्ये children render करण्यास परवानगी देतात. modals, tooltips किंवा dropdowns साठी उपयुक्त ज्यांना container overflow/z-index constraints पासून

SKILLHUB

break out करण्याची गरज आहे. ReactDOM.createPortal(child, container) वापरून created. component tree structure राहते, पण DOM placement बदलते.

Analogy:
Like a teleportation device: Portals let you render a component in one place in your code structure, but display it somewhere else entirely in the DOM, like teleporting an object to a different location.

Learn more:

- https://reactjs.org/docs/portals.html

✅ 175. What is Lazy Loading in ReactJS?

English:
Lazy loading defers loading components until they're needed, improving initial load time. Implemented using React.lazy() with Suspense. Components are code-split and loaded on demand. Useful for large applications with many routes or heavy components. Reduces bundle size and improves performance.

Hindi:
lazy loading components को तब तक defer करता है जब तक उनकी जरूरत न हो, initial load time को improve करता है। React.lazy() को Suspense के साथ implement किया जाता है। components code-split होते हैं और on demand loaded होते हैं। many routes या heavy components के साथ large applications के लिए useful। bundle size को reduce करता है और performance को improve करता है।

Marathi:
lazy loading components ला तोपर्यंत defer करते जोपर्यंत त्यांची गरज नाही, initial load time सुधारते. React.lazy() ला Suspense सोबत implement केले जाते. components code-split होतात आणि on demand loaded होतात. many routes किंवा heavy components सोबत large applications साठी उपयुक्त. bundle size कमी करते आणि performance सुधारते.

Analogy:
Like loading books on demand: instead of carrying all books (components) at once, lazy loading loads them only when you need to read them, reducing the initial weight (bundle size).

Learn more:

- https://reactjs.org/docs/code-splitting.html

✅ 176. What is useReducer hook in ReactJS?

English:
useReducer is a hook for managing complex state logic, similar to Redux's reducer pattern. It takes a

SKILLHUB

reducer function and initial state, returns current state and dispatch function. Useful when state logic is complex or when next state depends on previous state. Provides more predictable state updates than useState for complex scenarios.

Hindi:
useReducer complex state logic manage करने के लिए एक hook है, Redux के reducer pattern के similar। यह reducer function और initial state लेता है, current state और dispatch function return करता है। useful जब state logic complex है या जब next state previous state पर depend करता है। complex scenarios के लिए useState से ज्यादा predictable state updates provide करता है।

Marathi:
useReducer complex state logic manage करण्यासाठी एक hook आहे, Redux च्या reducer pattern सारखे. हे reducer function आणि initial state घेते, current state आणि dispatch function return करते. उपयुक्त जेव्हा state logic complex आहे किंवा जेव्हा next state previous state वर depend करते. complex scenarios साठी useState पेक्षा अधिक predictable state updates पुरवते.

Analogy:
Like a state machine: useReducer provides a structured way to manage state transitions, like a vending machine that follows specific rules (reducer) for each button press (action).

Learn more:

- https://reactjs.org/docs/hooks-reference.html

✅ 177. What is useRef hook in ReactJS?

English:
useRef returns a mutable ref object that persists across renders. It can hold any value and doesn't cause re-renders when changed. Common uses: accessing DOM elements, storing previous values, keeping mutable values without triggering re-renders. The .current property holds the current value.

Hindi:
useRef mutable ref object return करता है जो renders के across persist करता है। यह कोई भी value hold कर सकता है और change होने पर re-renders cause नहीं करता। common uses: DOM elements access करना, previous values store करना, re-renders trigger किए बिना mutable values keep करना। .current property current value hold करती है।

Marathi:
useRef mutable ref object return करते जे renders च्या across persist करते. हे कोणतीही value hold करू शकते आणि change झाल्यावर re-renders cause करत नाही. common uses: DOM elements access करणे, previous values store करणे, re-renders trigger न करता mutable values keep करणे. .current property current value hold करते.

SKILLHUB

Analogy:
Like a sticky note that persists: useRef gives you a reference that stays the same across renders, like a sticky note that doesn't disappear when you refresh the page.

Learn more:

- https://reactjs.org/docs/hooks-reference.html

✅ 178. What is useMemo hook in ReactJS?

English:
useMemo memoizes expensive computations, returning cached result when dependencies haven't changed. It prevents unnecessary recalculations on every render. Takes a function and dependency array. Only recomputes when dependencies change. Useful for expensive calculations, object/array creation in render.

Hindi:
useMemo expensive computations को memoize करता है, cached result return करता है जब dependencies change नहीं हुई हैं। यह हर render पर unnecessary recalculations को prevent करता है। function और dependency array लेता है। सिर्फ dependencies change होने पर recompute करता है। expensive calculations, render में object/array creation के लिए useful।

Marathi:
useMemo expensive computations ला memoize करते, cached result return करते जेव्हा dependencies change झाल्या नाहीत. हे प्रत्येक render वर unnecessary recalculations prevent करते. function आणि dependency array घेते. फक्त dependencies change झाल्यावर recompute करते. expensive calculations, render मध्ये object/array creation साठी उपयुक्त.

Analogy:
Like remembering a calculation: useMemo stores the result of an expensive calculation and only recalculates when inputs change, like remembering the answer to a math problem instead of solving it every time.

Learn more:

- https://reactjs.org/docs/hooks-reference.html

✅ 179. What is useCallback hook in ReactJS?

English:
useCallback memoizes functions, returning the same function reference when dependencies haven't changed. Prevents unnecessary function recreation on every render. Useful when passing functions as

SKILLHUB

props to memoized children, or as dependencies to other hooks. Takes a function and dependency array.

Hindi:
useCallback functions को memoize करता है, same function reference return करता है जब dependencies change नहीं हुई हैं। हर render पर unnecessary function recreation को prevent करता है। memoized children को functions को props के रूप में pass करते समय useful, या other hooks के dependencies के रूप में। function और dependency array लेता है।

Marathi:
useCallback functions ला memoize करते, same function reference return करते जेव्हा dependencies change झाल्या नाहीत. प्रत्येक render वर unnecessary function recreation prevent करते. memoized children ला functions ला props म्हणून pass करताना उपयुक्त किंवा other hooks च्या dependencies म्हणून. function आणि dependency array घेते.

Analogy:
Like keeping the same tool: useCallback ensures you're using the same function reference unless its dependencies change, preventing unnecessary tool replacements.

Learn more:

- https://reactjs.org/docs/hooks-reference.html

✅ 180. What is the difference between stateless and stateful components?

English:
Stateless (functional) components don't manage state, receive data via props, and are pure functions. Stateful components manage their own state using useState/useReducer (functional) or this.state (class). Stateless components are simpler and easier to test, while stateful components handle dynamic behavior.

Hindi:
stateless (functional) components state manage नहीं करते, props के माध्यम से data receive करते हैं, और pure functions हैं। stateful components useState/useReducer (functional) या this.state (class) का इस्तेमाल करके अपना state manage करते हैं। stateless components simpler और test करना easier हैं, जबकि stateful components dynamic behavior handle करते हैं।

Marathi:
stateless (functional) components state manage करत नाहीत, props द्वारे data receive करतात आणि pure functions आहेत. stateful components useState/useReducer (functional) किंवा this.state (class) वापरून स्वतःचा state manage करतात. stateless components सोपे आणि test करणे सोपे आहेत, तर stateful components dynamic behavior handle करतात.

SKILLHUB

Analogy:
Like calculators vs. computers: stateless components are like calculators (input → output, no memory), stateful components are like computers (can remember and change state).

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 181. What is the impact of using indexes as keys in React lists?

English:
Using array indexes as keys can cause bugs when list items are reordered, added, or removed. React may incorrectly reuse components, leading to state bugs, lost focus, or incorrect rendering. Use stable, unique identifiers (like IDs) as keys instead. Indexes are acceptable only for static, never-changing lists.

Hindi:
array indexes को keys के रूप में use करना bugs cause कर सकता है जब list items reordered, added, या removed होते हैं। React incorrectly components reuse कर सकता है, state bugs, lost focus, या incorrect rendering की ओर ले जाता है। stable, unique identifiers (जैसे IDs) को keys के रूप में use करें। indexes सिर्फ static, never-changing lists के लिए acceptable हैं।

Marathi:
array indexes ला keys म्हणून वापरणे bugs cause करू शकते जेव्हा list items reordered, added किंवा removed होतात. React incorrectly components reuse करू शकते, state bugs, lost focus किंवा incorrect rendering कडे नेत. stable, unique identifiers (जसे IDs) ला keys म्हणून वापरा. indexes फक्त static, never-changing lists साठी acceptable आहेत.

Analogy:
Like using seat numbers vs. passenger names: using indexes is like assigning seats by position number—when passengers move, the numbers don't match anymore. Using IDs is like using passenger names—always accurate.

Learn more:

- https://reactjs.org/docs/lists-and-keys.html

SKILLHUB

ReactJS Advanced Patterns by SKILLHUB - Day 12

✅ 182. How do you organize a React application?

English:
Common organization patterns include: feature-based (group by features), component-based (group by component type), or hybrid approach. Structure typically includes: components/, containers/, hooks/, utils/, services/, assets/, and tests/. Use consistent naming, clear folder hierarchy, and separate concerns. Tools like Create React App provide default structure.

Hindi:
common organization patterns में शामिल हैं: feature-based (features के अनुसार group), component-based (component type के अनुसार group), या hybrid approach। structure typically includes: components/, containers/, hooks/, utils/, services/, assets/, और tests/। consistent naming, clear folder hierarchy, और separate concerns use करें। Create React App जैसे tools default structure provide करते हैं।

Marathi:
common organization patterns मध्ये समाविष्ट आहेत: feature-based (features नुसार group), component-based (component type नुसार group) किंवा hybrid approach. structure typically includes: components/, containers/, hooks/, utils/, services/, assets/ आणि tests/. consistent naming, clear folder hierarchy आणि separate concerns वापरा. Create React App सारख्या tools default structure पुरवतात.

Analogy:
Like organizing a library: you can organize by genre (feature-based), by book type (component-based), or a mix. The goal is to find books (code) quickly and maintain order.

Learn more:

- https://reactjs.org/docs/faq-structure.html

✅ 183. What are the main challenges in React development?

English:
Common challenges include: prop drilling, state management complexity, performance optimization, component re-rendering, code organization at scale, testing, bundle size, SEO for SPAs, learning curve, and keeping up with frequent updates. Solutions include Context API, state management libraries, code splitting, and best practices.

SKILLHUB

Hindi:
common challenges में शामिल हैं: prop drilling, state management complexity, performance optimization, component re-rendering, scale पर code organization, testing, bundle size, SPAs के लिए SEO, learning curve, और frequent updates के साथ keeping up। solutions में Context API, state management libraries, code splitting, और best practices शामिल हैं।

Marathi:
common challenges मध्ये समाविष्ट आहेत: prop drilling, state management complexity, performance optimization, component re-rendering, scale वर code organization, testing, bundle size, SPAs साठी SEO, learning curve आणि frequent updates सोबत keeping up. solutions मध्ये Context API, state management libraries, code splitting आणि best practices समाविष्ट आहेत.

Analogy:
Like managing a growing city: as React apps grow, challenges like traffic (prop drilling), infrastructure (state management), and efficiency (performance) need solutions to keep everything running smoothly.

Learn more:

- https://reactjs.org/docs/thinking-in-react.html

✅ 184. What is Next.js and how does it relate to React?

English:
Next.js is a React framework that adds server-side rendering (SSR), static site generation (SSG), routing, API routes, and optimizations. It builds on React and provides production-ready features out of the box. It simplifies React development with conventions, automatic code splitting, and built-in optimizations.

Hindi:
Next.js एक React framework है जो server-side rendering (SSR), static site generation (SSG), routing, API routes, और optimizations add करता है। यह React पर build करता है और production-ready features out of the box provide करता है। यह conventions, automatic code splitting, और built-in optimizations के साथ React development को simplify करता है।

Marathi:
Next.js एक React framework आहे जे server-side rendering (SSR), static site generation (SSG), routing, API routes आणि optimizations add करते. हे React वर build करते आणि production-ready features out of the box पुरवते. हे conventions, automatic code splitting आणि built-in optimizations सोबत React development सुलभ करते.

Analogy:
Like React being a car engine and Next.js being the complete car: React provides the power, Next.js

SKILLHUB

adds the body, wheels, and features to make it road-ready.

Learn more:

- https://nextjs.org/docs

✅ 185. What is StrictMode in ReactJS?

English:
StrictMode is a development tool that highlights potential problems in applications. It doesn't render visible UI but activates additional checks and warnings. It helps identify unsafe lifecycles, legacy API usage, unexpected side effects, and deprecated features. Wrap your app or components with <React.StrictMode>.

Hindi:
StrictMode एक development tool है जो applications में potential problems को highlight करता है। यह visible UI render नहीं करता लेकिन additional checks और warnings activate करता है। यह unsafe lifecycles, legacy API usage, unexpected side effects, और deprecated features identify करने में help करता है। अपने app या components को <React.StrictMode> से wrap करें।

Marathi:
StrictMode एक development tool आहे जे applications मध्ये potential problems highlight करते. हे visible UI render करत नाही पण additional checks आणि warnings activate करते. हे unsafe lifecycles, legacy API usage, unexpected side effects आणि deprecated features identify करण्यात help करते. तुमच्या app किंवा components ला <React.StrictMode> ने wrap करा.

Analogy:
Like a spell-checker for code: StrictMode doesn't change your code but highlights potential issues, helping you write better React code.

Learn more:

- https://reactjs.org/docs/strict-mode.html

✅ 186. What are the pitfalls of using Context API?

English:
Common pitfalls include: overusing Context (causing unnecessary re-renders), creating too many contexts, not splitting contexts by concern, using Context for frequently changing data, and making components harder to test. Best practice: use Context for truly global, rarely changing data, and split contexts logically.

SKILLHUB

Hindi:
common pitfalls में शामिल हैं: Context को overuse करना (unnecessary re-renders cause करना), too many contexts create करना, concern के अनुसार contexts को split न करना, frequently changing data के लिए Context use करना, और components को test करना harder बनाना। best practice: truly global, rarely changing data के लिए Context use करें, और logically contexts को split करें।

Marathi:
common pitfalls मध्ये समाविष्ट आहेत: Context ला overuse करणे (unnecessary re-renders cause करणे), too many contexts create करणे, concern नुसार contexts ला split न करणे, frequently changing data साठी Context वापरणे आणि components ला test करणे कठीण बनवणे. best practice: truly global, rarely changing data साठी Context वापरा आणि logically contexts ला split करा.

Analogy:
Like overusing a megaphone: Context is powerful but using it for everything causes noise (re-renders). Use it selectively for important announcements (global data).

Learn more:

- https://reactjs.org/docs/context.html

✅ 187. How does React handle re-rendering?

English:
React re-renders when: state changes, props change, parent re-renders, or context value changes. React uses virtual DOM diffing to determine what actually changed. Only changed parts are updated in the real DOM. React.memo, useMemo, and useCallback can prevent unnecessary re-renders. React batches updates for performance.

Hindi:
React re-render करता है जब: state change होता है, props change होते हैं, parent re-render होता है, या context value change होता है। React virtual DOM diffing का इस्तेमाल करता है यह determine करने के लिए कि actually क्या change हुआ। सिर्फ changed parts real DOM में update होते हैं। React.memo, useMemo, और useCallback unnecessary re-renders को prevent कर सकते हैं। React performance के लिए updates को batch करता है।

Marathi:
React re-render करते जेव्हा: state change होते, props change होतात, parent re-render होते किंवा context value change होते. React virtual DOM diffing वापरते काय actually change झाले हे determine करण्यासाठी. फक्त changed parts real DOM मध्ये update होतात. React.memo, useMemo आणि useCallback unnecessary re-renders prevent करू शकतात. React performance साठी updates ला batch करते.

Analogy:
Like updating a painting: React doesn't repaint the entire canvas, it only touches up the parts that

SKILLHUB

actually changed, making updates efficient.

Learn more:

- https://reactjs.org/docs/reconciliation.html

✅ 188. When should you use useMemo and useReducer?

English:
Use useMemo for expensive computations that shouldn't run on every render, or when creating objects/arrays that are passed as props to memoized children. Use useReducer for complex state logic, when next state depends on previous state, or when managing multiple related state values. Both optimize performance and manage complexity.

Hindi:
useMemo का इस्तेमाल expensive computations के लिए करें जो हर render पर run नहीं होनी चाहिए, या objects/arrays create करते समय जो memoized children को props के रूप में pass किए जाते हैं। useReducer का इस्तेमाल complex state logic के लिए करें, जब next state previous state पर depend करता है, या multiple related state values manage करते समय। दोनों performance को optimize करते हैं और complexity को manage करते हैं।

Marathi:
useMemo चा वापर expensive computations साठी करा जे प्रत्येक render वर run होऊ नयेत किंवा objects/arrays create करताना जे memoized children ला props म्हणून pass केले जातात. useReducer चा वापर complex state logic साठी करा, जेव्हा next state previous state वर depend करते किंवा multiple related state values manage करताना. दोन्ही performance optimize करतात आणि complexity manage करतात.

Analogy:
Like choosing tools: useMemo is like caching a calculation result, useReducer is like having a rulebook for complex state changes—both make your app more efficient.

Learn more:

- https://reactjs.org/docs/hooks-reference.html

✅ 189. What are styled components in ReactJS?

English:
Styled Components is a CSS-in-JS library that lets you write CSS directly in JavaScript components. Styles are scoped to components, support props for dynamic styling, and eliminate class name conflicts. It provides a way to write component-level styles with JavaScript template literals and CSS syntax.

SKILLHUB

Hindi:
Styled Components एक CSS-in-JS library है जो आपको JavaScript components में directly CSS write करने देती है। styles components तक scoped हैं, dynamic styling के लिए props support करते हैं, और class name conflicts को eliminate करते हैं। यह JavaScript template literals और CSS syntax के साथ component-level styles write करने का तरीका provide करता है।

Marathi:
Styled Components एक CSS-in-JS library आहे जी तुम्हाला JavaScript components मध्ये directly CSS लिहिण्यास परवानगी देते. styles components पर्यंत scoped आहेत, dynamic styling साठी props support करतात आणि class name conflicts eliminate करतात. हे JavaScript template literals आणि CSS syntax सोबत component-level styles लिहिण्याचा मार्ग पुरवते.

Analogy:
Like having a personal stylist for each component: styled components give each component its own styling wardrobe that's perfectly tailored and doesn't conflict with others.

Learn more:

- https://styled-components.com/

✅ 190. Why should props be immutable in ReactJS?

English:
Props should be immutable because React relies on prop changes to determine when to re-render. Mutating props breaks React's unidirectional data flow, makes debugging difficult, and can cause unexpected behavior. Props flow down from parent to child, and children shouldn't modify them. Use state or callbacks for changes.

Hindi:
props immutable होने चाहिए क्योंकि React re-render करने का निर्णय लेने के लिए prop changes पर rely करता है। props को mutate करना React के unidirectional data flow को break करता है, debugging को difficult बनाता है, और unexpected behavior cause कर सकता है। props parent से child तक flow करते हैं, और children को उन्हें modify नहीं करना चाहिए। changes के लिए state या callbacks use करें।

Marathi:
props immutable असावेत कारण React re-render करण्याचा निर्णय घेण्यासाठी prop changes वर rely करते. props ला mutate करणे React च्या unidirectional data flow ला break करते, debugging कठीण बनवते आणि unexpected behavior cause करू शकते. props parent पासून child पर्यंत flow करतात आणि children ने त्यांना modify करू नये. changes साठी state किंवा callbacks वापरा.

Analogy:
Like a one-way street: props flow in one direction (parent → child), and trying to change them is like driving the wrong way—it breaks the system and causes problems.

SKILLHUB

Learn more:

- https://reactjs.org/docs/components-and-props.html

✅ 191. What is a dispatcher in ReactJS?

English:
A dispatcher is part of the Flux architecture pattern. It's a central hub that receives actions and dispatches them to registered stores. In Redux (Flux implementation), the dispatcher is abstracted. Actions are dispatched to reducers, which update the store. It ensures actions are processed in order and stores are updated correctly.

Hindi:
dispatcher Flux architecture pattern का हिस्सा है। यह एक central hub है जो actions receive करता है और registered stores को dispatch करता है। Redux (Flux implementation) में, dispatcher abstracted है। actions reducers को dispatch होती हैं, जो store को update करते हैं। यह ensure करता है कि actions order में process होती हैं और stores correctly update होते हैं।

Marathi:
dispatcher Flux architecture pattern चा भाग आहे. हे एक central hub आहे जे actions receive करते आणि registered stores ला dispatch करते. Redux (Flux implementation) मध्ये, dispatcher abstracted आहे. actions reducers ला dispatch होतात, जे store ला update करतात. हे ensure करते की actions order मध्ये process होतात आणि stores correctly update होतात.

Analogy:
Like a post office dispatcher: the dispatcher receives mail (actions) and routes them to the correct departments (stores) in an organized manner.

Learn more:

- https://reactjs.org/blog/2014/05/06/flux.html

✅ 192. What are default props in ReactJS?

English:
Default props provide fallback values when props aren't passed to a component. In class components, use static defaultProps. In functional components, use default parameters in function signature or || operator. Default props ensure components work even when optional props are missing, improving component robustness.

Hindi:
default props fallback values provide करते हैं जब props component को pass नहीं किए जाते हैं। class

SKILLHUB

components में, static defaultProps use करें। functional components में, function signature में default parameters या || operator use करें। default props ensure करते हैं कि components work करते हैं even when optional props missing हैं, component robustness को improve करते हैं।

Marathi:
default props fallback values पुरवतात जेव्हा props component ला pass केले जात नाहीत. class components मध्ये, static defaultProps वापरा. functional components मध्ये, function signature मध्ये default parameters किंवा || operator वापरा. default props ensure करतात की components work करतात even when optional props missing आहेत, component robustness सुधारतात.

Analogy:
Like having backup plans: default props are like having a backup plan when something isn't provided, ensuring the component always has what it needs to function.

Learn more:

- https://reactjs.org/docs/typechecking-with-proptypes.html

✅ 193. What are inline conditional expressions in ReactJS?

English:
Inline conditionals allow rendering content based on conditions directly in JSX. Common patterns: ternary operator (condition ? true : false), logical AND (condition && element), or IIFE for complex logic. They provide concise conditional rendering without separate if-else blocks, making JSX more readable.

Hindi:
inline conditionals JSX में directly conditions पर based content render करने की अनुमति देते हैं। common patterns: ternary operator (condition ? true : false), logical AND (condition && element), या complex logic के लिए IIFE। वे separate if-else blocks के बिना concise conditional rendering provide करते हैं, JSX को ज्यादा readable बनाते हैं।

Marathi:
inline conditionals JSX मध्ये directly conditions वर based content render करण्यास परवानगी देतात. common patterns: ternary operator (condition ? true : false), logical AND (condition && element) किंवा complex logic साठी IIFE. ते separate if-else blocks शिवाय concise conditional rendering पुरवतात, JSX ला अधिक readable बनवतात.

Analogy:
Like quick decision-making: inline conditionals are like making quick decisions on the spot ("if hungry, eat; else wait") rather than writing a full decision tree.

Learn more:

SKILLHUB

- https://reactjs.org/docs/conditional-rendering.html

✅ 194. What is the difference between React and React Native?

English:
React is for web development (DOM), React Native is for mobile app development (native components). React uses HTML elements, React Native uses native mobile components. React Native compiles to native code, React renders to web browsers. Both share the same component model and JSX syntax, but target different platforms.

Hindi:
React web development (DOM) के लिए है, React Native mobile app development (native components) के लिए है। React HTML elements use करता है, React Native native mobile components use करता है। React Native native code में compile होता है, React web browsers में render होता है। दोनों same component model और JSX syntax share करते हैं, लेकिन different platforms target करते हैं।

Marathi:
React web development (DOM) साठी आहे, React Native mobile app development (native components) साठी आहे. React HTML elements वापरते, React Native native mobile components वापरते. React Native native code मध्ये compile होते, React web browsers मध्ये render होते. दोन्ही same component model आणि JSX syntax share करतात, पण different platforms target करतात.

Analogy:
Like the same language (React) spoken in different countries: React is for web (like English in UK), React Native is for mobile (like English in USA)—same language, different environments.

Learn more:

- https://reactnative.dev/

✅ 195. Is React a library or a framework?

English:
React is a library, not a framework. It focuses only on the View layer and doesn't provide routing, state management, or HTTP client out of the box. Frameworks (like Angular) provide complete solutions. React's library nature gives developers flexibility to choose other tools, but requires more decisions.

Hindi:
React एक library है, framework नहीं है। यह सिर्फ View layer पर focus करता है और routing, state management, या HTTP client out of the box provide नहीं करता। frameworks (जैसे Angular) complete

SKILLHUB

solutions provide करते हैं। React की library nature developers को flexibility देती है other tools choose करने के लिए, लेकिन ज्यादा decisions require करती है।

Marathi:
React एक library आहे, framework नाही. हे फक्त View layer वर focus करते आणि routing, state management किंवा HTTP client out of the box पुरवत नाही. frameworks (जसे Angular) complete solutions पुरवतात. React ची library nature developers ला flexibility देते other tools निवडण्यासाठी, पण अधिक decisions आवश्यक करते.

Analogy:
Like a tool vs. a toolkit: React is a specific tool (hammer) for building UIs, while frameworks are complete toolkits (toolbox with everything). You choose other tools with React, frameworks come with everything.

Learn more:

- https://reactjs.org/docs/getting-started.html

✅ 196. What are nested components in ReactJS?

English:
Nested components are components that contain other components as children. This creates a component hierarchy or tree structure. Parent components render child components, passing props down. This composition pattern allows building complex UIs from simpler, reusable components. It's a fundamental React pattern.

Hindi:
nested components components हैं जो other components को children के रूप में contain करते हैं। यह component hierarchy या tree structure create करता है। parent components child components render करते हैं, props को नीचे pass करते हैं। यह composition pattern simpler, reusable components से complex UIs build करने की अनुमति देता है। यह एक fundamental React pattern है।

Marathi:
nested components components आहेत जे other components ला children म्हणून contain करतात. हे component hierarchy किंवा tree structure तयार करते. parent components child components render करतात, props खाली pass करतात. हे composition pattern simpler, reusable components पासून complex UIs build करण्यास परवानगी देते. हे एक fundamental React pattern आहे.

Analogy:
Like Russian nesting dolls: nested components are components inside components, creating a hierarchical structure just like nested dolls fit inside each other.

Learn more:

SKILLHUB

- https://reactjs.org/docs/composition-vs-inheritance.html

✅ 197. How do you access sibling component state in ReactJS?

English:
Sibling components can't directly access each other's state. Common solutions: lift state up to a common parent, use Context API for shared state, use state management library (Redux), or use custom events. The most common pattern is lifting state to the nearest common ancestor and passing it down as props.

Hindi:
sibling components directly एक दूसरे के state access नहीं कर सकते। common solutions: state को common parent तक lift करना, shared state के लिए Context API use करना, state management library (Redux) use करना, या custom events use करना। most common pattern state को nearest common ancestor तक lift करना है और इसे props के रूप में नीचे pass करना है।

Marathi:
sibling components directly एकमेकांचा state access करू शकत नाहीत. common solutions: state ला common parent पर्यंत lift करणे, shared state साठी Context API वापरणे, state management library (Redux) वापरणे किंवा custom events वापरणे. most common pattern state ला nearest common ancestor पर्यंत lift करणे आहे आणि ते props म्हणून खाली pass करणे आहे.

Analogy:
Like siblings asking parents: sibling components can't talk directly to each other, so they communicate through their parent (lifted state) or a shared system (Context/Redux).

Learn more:

- https://reactjs.org/docs/lifting-state-up.html

✅ 198. What are error boundaries in ReactJS?

English:
Error boundaries are React components that catch JavaScript errors in their child component tree, log them, and display fallback UI. They catch errors during rendering, lifecycle methods, and constructors. Created using class components with componentDidCatch or static getDerivedStateFromError. They prevent entire app crashes.

Hindi:
error boundaries React components हैं जो अपने child component tree में JavaScript errors catch करते हैं, log करते हैं, और fallback UI display करते हैं। वे rendering, lifecycle methods, और constructors के दौरान

SKILLHUB

errors catch करते हैं। componentDidCatch या static getDerivedStateFromError के साथ class components का इस्तेमाल करके created। वे entire app crashes को prevent करते हैं।

Marathi:
error boundaries React components आहेत जे स्वतःच्या child component tree मध्ये JavaScript errors catch करतात, log करतात आणि fallback UI display करतात. ते rendering, lifecycle methods आणि constructors दरम्यान errors catch करतात. componentDidCatch किंवा static getDerivedStateFromError सोबत class components वापरून created. ते entire app crashes prevent करतात.

Analogy:
Like safety nets: error boundaries catch errors (falls) in parts of your app, preventing the entire app from crashing, just like safety nets catch performers but don't stop the whole show.

Learn more:

- https://reactjs.org/docs/error-boundaries.html

✅ 199. What is React DevTools?

English:
React DevTools is a browser extension for debugging React applications. It allows inspecting component hierarchy, viewing props and state, profiling performance, and tracking component updates. Available for Chrome, Firefox, and Edge. It's an essential tool for React development and debugging.

Hindi:
React DevTools React applications को debug करने के लिए एक browser extension है। यह component hierarchy inspect करने, props और state view करने, performance profile करने, और component updates track करने की अनुमति देता है। Chrome, Firefox, और Edge के लिए available। यह React development और debugging के लिए एक essential tool है।

Marathi:
React DevTools React applications ला debug करण्यासाठी एक browser extension आहे. हे component hierarchy inspect करणे, props आणि state view करणे, performance profile करणे आणि component updates track करण्यास परवानगी देते. Chrome, Firefox आणि Edge साठी available. हे React development आणि debugging साठी एक essential tool आहे.

Analogy:
Like X-ray vision for React apps: React DevTools lets you see inside components, inspect their structure, and understand what's happening under the hood.

Learn more:

- https://reactjs.org/blog/2019/08/15/new-react-devtools.html

SKILLHUB

✅ 200. What is React Node?

English:
A React Node (ReactNode) is a type that represents anything React can render: strings, numbers, elements, fragments, portals, null, undefined, or arrays of these. It's the return type of component render methods and children props. It's a flexible type that covers all possible renderable content.

Hindi:
React Node (ReactNode) एक type है जो किसी भी चीज़ को represent करता है जिसे React render कर सकता है: strings, numbers, elements, fragments, portals, null, undefined, या इनके arrays। यह component render methods और children props का return type है। यह एक flexible type है जो सभी possible renderable content को cover करता है।

Marathi:
React Node (ReactNode) एक type आहे जे काहीही represent करते जे React render करू शकते: strings, numbers, elements, fragments, portals, null, undefined किंवा यांचे arrays. हे component render methods आणि children props चे return type आहे. हे एक flexible type आहे जे सर्व possible renderable content cover करते.

Analogy:
Like a universal container: ReactNode is like a box that can hold anything React can display—text, numbers, components, or nothing at all.

Learn more:

- https://reactjs.org/docs/typechecking-with-proptypes.html

✅ 201. Is React Router necessary for React applications?

English:
React Router is not necessary but highly recommended for multi-page applications. React itself doesn't include routing. For single-page applications with navigation, React Router provides declarative routing, browser history, and URL synchronization. For simple single-page apps without routing needs, it's optional.

Hindi:
React Router necessary नहीं है लेकिन multi-page applications के लिए highly recommended है। React itself routing include नहीं करता। navigation के साथ single-page applications के लिए, React Router declarative routing, browser history, और URL synchronization provide करता है। routing needs के बिना simple single-page apps के लिए, यह optional है।

SKILLHUB

Marathi:
React Router आवश्यक नाही पण multi-page applications साठी highly recommended आहे. React itself routing include करत नाही. navigation सोबत single-page applications साठी, React Router declarative routing, browser history आणि URL synchronization पुरवते. routing needs शिवाय simple single-page apps साठी, हे optional आहे.

Analogy:
Like GPS for navigation: React Router isn't required to drive (build React apps), but it's essential if you want to navigate between different destinations (routes) efficiently.

Learn more:

- https://reactrouter.com/

SKILLHUB

ReactJS Performance & Architecture by SKILLHUB - Day 13

✅ 196. Explain the role of Reducer.

English:
Reducer is pure function that takes current state and action, returns new state. Used in useReducer hook and Redux. Must be pure (no side effects), predictable. Actions describe what happened, reducer specifies how state changes. Enables time-travel debugging and predictable state updates.

Hindi:
reducer pure function है जो current state और action लेता है, new state return करता है। useReducer hook और redux में इस्तेमाल किया जाता है। pure होना चाहिए (no side effects), predictable। actions बताते हैं कि क्या हुआ, reducer specify करता है कि state कैसे changes। time-travel debugging और predictable state updates को enable करता है।

Marathi:
reducer pure function आहे जे current state आणि action घेते, new state return करते. useReducer hook आणि redux मध्ये वापरले जाते. pure असावे (no side effects), predictable. actions सांगतात की काय झाले, reducer specify करतो की state कसे changes. time-travel debugging आणि predictable state updates सक्षम करतो.

Analogy:
Like a bank teller processing transactions: reducer takes your current balance (state) and a deposit/withdrawal slip (action), then returns your new balance.

Learn more:

- https://redux.js.org/tutorials/fundamentals/part-3-state-actions-reducers

✅ 197. What is the use of React.cloneElement?

English:
React.cloneElement clones and returns a new React element with additional props/children. Preserves original element's props while adding new ones. Useful for higher-order components that modify children. More controlled than directly modifying props. Maintains element's key and ref.

Hindi:
react.cloneElement additional props/children के साथ new react element को clone करता है और return

SKILLHUB

करता है। original element के props को preserve करता है जबकि new ones add करता है। children को modify करने वाले higher-order components के लिए useful। directly props modify करने से ज्यादा controlled। element के key और ref को maintain करता है।

Marathi:
react.cloneElement additional props/children सोबत नवीन react element ला clone करतो आणि return करतो. original element चे props preserve करतो तर new ones add करतो. children ला modify करणाऱ्या higher-order components साठी useful. directly props modify करण्यापेक्षा अधिक controlled. element चा key आणि ref maintain करतो.

Analogy:
Like making a copy of a painting with extra details: cloneElement takes an existing component and adds new "brush strokes" (props) without changing the original.

Learn more:

- https://reactjs.org/docs/react-api.html#cloneelement

✅ 198. Can you explain React's "lifting state up" concept and why it is important?

English:
Lifting state up moves state to common ancestor when multiple components need same data. Siblings share state through parent. Enables coordination between components. Important for single source of truth, predictable data flow, easier testing. Alternative to prop drilling with Context API.

Hindi:
lifting state up तब किया जाता है जब multiple components को same data की need होती है तो state को common ancestor में move करें। siblings parent के माध्यम से state share करते हैं। components के बीच coordination को enable करता है। single source of truth, predictable data flow, easier testing के लिए important। context API के साथ prop drilling का alternative।

Marathi:
lifting state up जेव्हा multiple components ला समान data ची गरज असते तेव्हा state ला common ancestor मध्ये move करा. siblings parent द्वारे state share करतात. components दरम्यान coordination सक्षम करतो. single source of truth, predictable data flow, easier testing साठी important. context API सोबत prop drilling चा alternative.

Analogy:
Like moving family valuables to a safe deposit box: lifting state up puts shared data in a secure, accessible location (parent component) that all family members (child components) can reach.

Learn more:

SKILLHUB

- https://reactjs.org/docs/lifting-state-up.html

✅ 199. What is a ref in React and what is its purpose?

English:
Ref provides direct access to DOM element or React component instance. Created with React.createRef() or useRef hook. Bypasses React's declarative paradigm for imperative operations. Used for focus management, media playback control, integrating non-React libraries, measuring DOM elements.

Hindi:
ref DOM element या react component instance तक direct access provide करता है। react.createRef() या useRef hook से created। imperative operations के लिए react के declarative paradigm को bypass करता है। focus management, media playback control, non-react libraries के साथ integration, DOM elements को measure करने के लिए इस्तेमाल किया जाता है।

Marathi:
ref DOM element किंवा react component instance पर्यंत direct access पुरवतो. react.createRef() किंवा useRef hook ने created. imperative operations साठी react च्या declarative paradigm ला bypass करतो. focus management, media playback control, non-react libraries सोबत integration, DOM elements measure करण्यासाठी वापरला जातो.

Analogy:
Like a secret backdoor: refs let you access the "real" DOM element directly when React's normal "front door" (props/state) isn't enough.

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

✅ 200. How does using refs differ from traditional DOM manipulation?

English:
Refs provide React-managed way to access DOM, safer than document.getElementById. Refs work with virtual DOM, update during React's render cycle. Traditional DOM manipulation can conflict with React's updates. Refs are React's way of saying "I know what I'm doing" for imperative operations.

Hindi:
refs DOM access करने का react-managed तरीका provide करते हैं, document.getElementById से safer। refs virtual DOM के साथ work करते हैं, react के render cycle के दौरान update होते हैं। traditional DOM

SKILLHUB

manipulation react के updates से conflict कर सकता है। refs imperative operations के लिए react का "I know what I'm doing" तरीका है।

Marathi:
refs DOM access करण्याचा react-managed मार्ग पुरवतात, document.getElementById पेक्षा safer. refs virtual DOM सोबत work करतात, react च्या render cycle दरम्यान update होतात. traditional DOM manipulation react च्या updates सोबत conflict करू शकते. refs imperative operations साठी react चा "I know what I'm doing" मार्ग आहे.

Analogy:
Like asking permission vs. sneaking in: refs are React's approved way to touch the DOM, while traditional manipulation is like going behind React's back.

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

✅ 201. What is the main difference between "string refs" and "callback refs" in React.

English:
String refs (deprecated) use string to identify DOM element, accessed via this.refs.stringName. Callback refs pass function that receives DOM element as parameter. Callback refs are preferred - more predictable, work with dynamic refs, don't conflict with React's future changes.

Hindi:
string refs (deprecated) DOM element identify करने के लिए string इस्तेमाल करते हैं, this.refs.stringName के माध्यम से accessed। callback refs function pass करते हैं जो DOM element को parameter के रूप में receive करता है। callback refs preferred हैं - ज्यादा predictable, dynamic refs के साथ work करते हैं, react के future changes से conflict नहीं करते।

Marathi:
string refs (deprecated) DOM element identify करण्यासाठी string वापरतात, this.refs.stringName द्वारे accessed. callback refs function pass करतात जी DOM element ला parameter म्हणून receive करते. callback refs preferred आहेत - अधिक predictable, dynamic refs सोबत work करतात, react च्या future changes सोबत conflict करत नाहीत.

Analogy:
Like naming pets vs. training them to come when called: string refs use names (static), callback refs use behaviors (dynamic functions).

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

SKILLHUB

✅ 202. How do refs affect the component lifecycle in React?

English:
Refs are set after component mounts (componentDidMount in class, useEffect in functional). Not available during initial render. When component unmounts, refs become null. Refs don't trigger re-renders when changed. Updating ref values is side effect, not state change.

Hindi:
refs component mount होने के बाद set होते हैं (class में componentDidMount, functional में useEffect)। initial render के दौरान available नहीं। जब component unmount होता है, तब refs null हो जाते हैं। changed होने पर refs re-renders trigger नहीं करते। ref values को update करना side effect है, state change नहीं।

Marathi:
refs component mount झाल्यानंतर set होतात (class मध्ये componentDidMount, functional मध्ये useEffect). initial render दरम्यान available नाहीत. जेव्हा component unmount होते, तेव्हा refs null होतात. changed झाल्यावर refs re-renders trigger करत नाहीत. ref values update करणे side effect आहे, state change नाही.

Analogy:
Like getting a library card after joining: refs become available only after the component has "joined" (mounted) the React app.

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

✅ 203. What are the benefits of using callback refs over string refs?

English:
Callback refs are more flexible, work immediately when component mounts. Can be dynamic, passed as props. Don't create string-based dependencies. Safer with minification. React can optimize callback refs better. No deprecated warnings. Work consistently across different React versions.

Hindi:
callback refs ज्यादा flexible हैं, component mount होते ही work करते हैं। dynamic हो सकते हैं, props के रूप में pass किए जा सकते हैं। string-based dependencies create नहीं करते। minification के साथ safer। react callback refs को better optimize कर सकता है। कोई deprecated warnings नहीं। different react versions में consistently work करते हैं।

Marathi:
callback refs अधिक flexible आहेत, component mount झाल्याबरोबर work करतात. dynamic असू शकतात, props म्हणून pass केले जाऊ शकतात. string-based dependencies create करत नाहीत. minification सोबत

SKILLHUB

safer. react callback refs ला better optimize करू शकते. कोणतेही deprecated warnings नाहीत. different react versions मध्ये consistently work करतात.

Analogy:
Like smart locks vs. labeled keys: callback refs adapt to any situation, while string refs are rigid and predictable.

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

✅ 204. Can we use refs with functional components in React? If yes, how?

English:
Yes, use useRef hook in functional components. const myRef = useRef(initialValue). Returns mutable ref object with .current property. Use for DOM access, storing mutable values, or instance-like variables. Functional components can't have string refs, only callback or useRef.

Hindi:
हां, functional components में useRef hook इस्तेमाल करें। const myRef = useRef(initialValue)। .current property वाला mutable ref object return करता है। DOM access, mutable values store करने, या instance-like variables के लिए इस्तेमाल करें। functional components में string refs नहीं हो सकते, सिर्फ callback या useRef।

Marathi:
होय, functional components मध्ये useRef hook वापरा. const myRef = useRef(initialValue). .current property असलेला mutable ref object return करतो. DOM access, mutable values store करण्यासाठी किंवा instance-like variables साठी वापरा. functional components मध्ये string refs असू शकत नाहीत, फक्त callback किंवा useRef.

Analogy:
Like getting a locker in a new school: useRef gives functional components their own storage space, just like class components have instance variables.

Learn more:

- https://reactjs.org/docs/hooks-reference.html#useref

✅ 205. What are the limitations or caveats of using refs in React?

SKILLHUB

English:
Refs break React's declarative nature, should be last resort. Can cause memory leaks if not cleaned up. Don't overuse for state that should trigger renders. String refs deprecated. Can't use refs conditionally. May not work with some advanced patterns. Prefer state/props over refs.

Hindi:
refs react की declarative nature को break करते हैं, last resort होना चाहिए। अगर cleanup नहीं किया गया तो memory leaks का कारण बन सकते हैं। renders trigger करने वाले state के लिए overuse न करें। string refs deprecated। conditionally refs इस्तेमाल नहीं कर सकते। कुछ advanced patterns के साथ work नहीं कर सकते। refs के बजाय state/props prefer करें।

Marathi:
refs react च्या declarative nature ला break करतात, last resort असावा. जर cleanup केले नाही तर memory leaks चा कारण होऊ शकतात. renders trigger करणाऱ्या state साठी overuse करू नका. string refs deprecated. conditionally refs वापरू शकत नाही. काही advanced patterns सोबत work करू शकत नाही. refs ऐवजी state/props prefer करा.

Analogy:
Like using duct tape on everything: refs are powerful but can create messy, hard-to-maintain code if overused instead of proper React patterns.

Learn more:

- https://reactjs.org/docs/refs-and-the-dom.html

✅ 206. What is the difference between Element and Component in React?

English:
Component is function/class that accepts props and returns React element. Element is plain object describing what to render (type, props, children). Components are factories that produce elements. Elements are immutable descriptions, components can be reused. JSX compiles to React.createElement() calls that create elements.

Hindi:
component function/class है जो props accept करता है और react element return करता है। element plain object है जो बताता है कि क्या render करना है (type, props, children)। components ऐसे factories हैं जो elements produce करते हैं। elements immutable descriptions हैं, components reused हो सकते हैं। JSX react.createElement() calls में compile होता है जो elements create करते हैं।

Marathi:
component function/class आहे जे props accept करते आणि react element return करते. element plain object आहे जे सांगते की काय render करायचे (type, props, children). components असे factories आहेत जी

SKILLHUB

elements produce करतात. elements immutable descriptions आहेत, components reused होऊ शकतात. JSX react.createElement() calls मध्ये compile होते जे elements create करतात.

Analogy:
Like recipes vs. meals: components are recipes (reusable instructions), elements are the actual prepared meals (immutable results).

Learn more:

- https://reactjs.org/blog/2015/12/18/react-components-elements-and-instances.html

---

✅ 207. How to apply validation on props in React?

English:
Use PropTypes library for runtime type checking. Define propTypes object on component. Validates prop types and marks as required/optional. Shows warnings in development. For TypeScript, use interface/type definitions. Better development experience and catches bugs early.

Hindi:
react में props पर validation के लिए propTypes library इस्तेमाल करें। component पर propTypes object define करें। prop types validate करता है और required/optional mark करता है। development में warnings show करता है। typeScript के लिए, interface/type definitions इस्तेमाल करें। better development experience और bugs को early catch करता है।

Marathi:
react मध्ये props वर validation साठी propTypes library वापरा. component वर propTypes object define करा. prop types validate करते आणि required/optional mark करते. development मध्ये warnings show करते. typeScript साठी, interface/type definitions वापरा. better development experience आणि bugs ला early catch करते.

Analogy:
Like quality control inspectors: prop validation checks incoming "materials" (props) to ensure they meet the required specifications before use.

Learn more:

- https://reactjs.org/docs/typechecking-with-proptypes.html

✅ 208. Why is a React component declarative?

English:
Declarative programming describes what should happen, not how. React components declare UI structure and state, React handles DOM manipulation. Unlike imperative jQuery where you manually

SKILLHUB

update DOM. Declarative code is more predictable, easier to debug, and React optimizes the "how" automatically.

Hindi:
declarative programming बताता है कि क्या होना चाहिए, कैसे नहीं। react components UI structure और state declare करते हैं, react DOM manipulation handle करता है। imperative jQuery के विपरीत जहां आप manually DOM update करते हैं। declarative code ज्यादा predictable, debug करने में easy, और react "how" को automatically optimize करता है।

Marathi:
declarative programming सांगते की काय व्हायला हवे, कसे नाही. react components UI structure आणि state declare करतात, react DOM manipulation handle करते. imperative jQuery च्या विपरीत जिथे आप manually DOM update करता. declarative code अधिक predictable, debug करण्यास easy आणि react "how" ला automatically optimize करते.

Analogy:
Like ordering food vs. cooking: declarative React is "I want spaghetti" (what), imperative vanilla JS is "boil water, add pasta, stir..." (how).

Learn more:

- https://reactjs.org/docs/thinking-in-react.html

✅ 209. What is ReactDOM package?

English:
ReactDOM renders React components to the DOM. Provides methods like render() to mount components, unmountComponentAtNode() to unmount. Separate from React core for different rendering targets (React Native uses different renderer). Handles browser-specific DOM operations.

Hindi:
reactDOM react components को DOM में render करता है। render() जैसे methods provide करता है components को mount करने के लिए, unmountComponentAtNode() unmount करने के लिए। react core से separate है different rendering targets के लिए (react native different renderer इस्तेमाल करता है)। browser-specific DOM operations handle करता है।

Marathi:
reactDOM react components ला DOM मध्ये render करते. render() सारखे methods पुरवते components mount करण्यासाठी, unmountComponentAtNode() unmount करण्यासाठी. react core पासून separate आहे different rendering targets साठी (react native different renderer वापरते). browser-specific DOM operations handle करते.

Analogy:
Like a specialized delivery service: ReactDOM takes React's packages (components) and delivers them

SKILLHUB

specifically to the browser's warehouse (DOM).

Learn more:

- https://reactjs.org/docs/react-dom.html

✅ 210. how is React different from Angular?

English:
React is library for UI components, Angular is full framework. React uses JSX and one-way data binding, Angular uses templates and two-way binding. React is more flexible, Angular provides more built-in features. React has larger ecosystem, Angular has better TypeScript integration. Different learning curves and use cases.

Hindi:
react UI components के लिए library है, angular full framework है। react JSX और one-way data binding इस्तेमाल करता है, angular templates और two-way binding। react ज्यादा flexible है, angular ज्यादा built-in features provide करता है। react का larger ecosystem है, angular का better typescript integration। different learning curves और use cases।

Marathi:
react UI components साठी library आहे, angular full framework आहे. react JSX आणि one-way data binding वापरते, angular templates आणि two-way binding. react अधिक flexible आहे, angular अधिक built-in features पुरवते. react चा larger ecosystem आहे, angular चा better typescript integration. different learning curves आणि use cases.

Analogy:
Like choosing between a customizable sports car (React) vs. a fully-loaded SUV (Angular) - both get you there, but with different capabilities and driving experiences.

Learn more:

- https://reactjs.org/docs/comparison-with-other-frameworks.html

---

✅ 211. Explain the use of CSS modules in React.

English:
CSS modules scope CSS locally to components, preventing global conflicts. Import CSS as object: import styles from './Button.module.css'. Use with className={styles.className}. CSS is compiled with unique class names. Enables component-scoped styling without global pollution.

Hindi:
css modules css को components तक locally scope करते हैं, global conflicts को prevent करते हैं। css को

SKILLHUB

object के रूप में import करें: import styles from './Button.module.css'। className={styles.className} के साथ इस्तेमाल करें। css unique class names के साथ compiled होता है। global pollution के बिना component-scoped styling को enable करता है।

Marathi:
css modules css ला components पर्यंत locally scope करतात, global conflicts prevent करतात. css ला object म्हणून import करा: import styles from './Button.module.css'. className={styles.className} सोबत वापरा. css unique class names सोबत compiled होते. global pollution शिवाय component-scoped styling सक्षम करतो.

Analogy:
Like having private closets for each room: CSS modules give each component its own styling space, preventing clothes (styles) from getting mixed up.

Learn more:

- https://github.com/css-modules/css-modules

✅ 212. Can you explain what custom hooks are in React and how they differ from regular hooks?

English:
Custom hooks are reusable functions that use built-in hooks and encapsulate related logic. Created by developers, not React. Follow naming convention useXXX. Can call other hooks, manage state, side effects. Regular hooks (useState, useEffect) are provided by React. Custom hooks promote code reuse and cleaner components.

Hindi:
custom hooks reusable functions हैं जो built-in hooks इस्तेमाल करते हैं और related logic को encapsulate करते हैं। developers द्वारा created, react द्वारा नहीं। useXXX naming convention follow करते हैं। अन्य hooks call कर सकते हैं, state manage कर सकते हैं, side effects। regular hooks (useState, useEffect) react द्वारा provided। custom hooks code reuse और cleaner components को promote करते हैं।

Marathi:
custom hooks reusable functions आहेत जी built-in hooks वापरतात आणि related logic ला encapsulate करतात. developers द्वारे created, react द्वारे नाही. useXXX naming convention follow करतात. इतर hooks call करू शकतात, state manage करू शकतात, side effects. regular hooks (useState, useEffect) react द्वारे provided. custom hooks code reuse आणि cleaner components ला promote करतात.

Analogy:
Like creating your own recipes using basic ingredients: custom hooks combine React's built-in hooks to create specialized functionality you can reuse.

Learn more:

SKILLHUB

- https://reactjs.org/docs/hooks-custom.html

✅ 213. What are the benefits of using custom hooks in React? Can you provide an example of a scenario where a custom hook would be useful?

English:
Benefits: code reuse, separation of concerns, cleaner components, easier testing, logic abstraction. Useful scenario: data fetching hook that handles loading states, errors, and caching across multiple components. Or form validation logic shared between forms. Makes components focus on presentation.

Hindi:
benefits: code reuse, separation of concerns, cleaner components, easier testing, logic abstraction। useful scenario: data fetching hook जो loading states, errors, और caching को multiple components में handle करता है। या forms के बीच shared form validation logic। components को presentation पर focus करने देता है।

Marathi:
benefits: code reuse, separation of concerns, cleaner components, easier testing, logic abstraction. useful scenario: data fetching hook जो loading states, errors आणि caching ला multiple components मध्ये handle करतो. किंवा forms दरम्यान shared form validation logic. components ला presentation वर focus करण्यास परवानगी देतो.

Analogy:
Like having specialized kitchen gadgets: custom hooks are tools that handle specific cooking tasks (like a garlic press for data fetching), so you can focus on the overall meal (UI).

Learn more:

- https://reactjs.org/docs/hooks-custom.html

✅ 214. How do you create a custom hook in React?

English:
Create function starting with "use" (useCustomHook). Can use built-in hooks inside. Return values/state that components need. Follow rules of hooks. Place in separate file, import where needed. Should be pure functions. Can accept parameters for customization.

Hindi:
"use" से शुरू होने वाला function create करें (useCustomHook)। अंदर built-in hooks इस्तेमाल कर सकते हैं।

SKILLHUB

components को needed values/state return करें। hooks के rules follow करें। separate file में place करें, जहां needed हो वहां import करें। pure functions होने चाहिए। customization के लिए parameters accept कर सकते हैं।

Marathi:
"use" ने सुरू होणारा function create करा (useCustomHook). आत built-in hooks वापरू शकता. components ला needed values/state return करा. hooks च्या rules follow करा. separate file मध्ये place करा, जिथे needed असेल तिथे import करा. pure functions असावेत. customization साठी parameters accept करू शकतात.

Analogy:
Like writing a new recipe: custom hooks combine existing ingredients (built-in hooks) into a new dish (reusable functionality) following the cookbook rules.

Learn more:

- https://reactjs.org/docs/hooks-custom.html

✅ 215. Are there any limitations or considerations when using custom hooks in React? Are there any performance implications or potential pitfalls that developers should be aware of?

English:
Limitations: must follow rules of hooks, can't be called conditionally. Performance: hooks are called on every render, so expensive operations should be memoized. Pitfalls: stale closures, dependency arrays. Should not be overused - sometimes regular functions are better. Testing custom hooks requires special setup.

Hindi:
limitations: hooks के rules follow करना जरूरी, conditionally call नहीं किए जा सकते। performance: hooks हर render पर called होते हैं, इसलिए expensive operations को memoized किया जाना चाहिए। pitfalls: stale closures, dependency arrays। overused नहीं होना चाहिए - कभी-कभी regular functions better होते हैं। custom hooks को test करने के लिए special setup चाहिए।

Marathi:
limitations: hooks च्या rules follow करणे आवश्यक, conditionally call करू शकत नाही. performance: hooks प्रत्येक render वर called होतात, म्हणून expensive operations ला memoized केले पाहिजे. pitfalls: stale closures, dependency arrays. overused नसावे - कधीकधी regular functions better असतात. custom hooks test करण्यासाठी special setup हवी.

Analogy:
Like power tools: custom hooks are powerful but require careful handling - misuse can cause injury (performance issues) or create safety hazards (bugs).

SKILLHUB

Learn more:

- https://reactjs.org/docs/hooks-rules.html

✅ 216. What is Formik, and why would you use it in a React application?

English:
Formik is popular React form library that simplifies form handling. Manages form state, validation, submission. Provides form, field, error components. Reduces boilerplate code for complex forms. Handles form validation with Yup integration. Makes forms more maintainable and testable.

Hindi:
formik popular react form library है जो form handling को simplify करता है। form state, validation, submission manage करता है। form, field, error components provide करता है। complex forms के लिए boilerplate code कम करता है। yup integration के साथ form validation handle करता है। forms को ज्यादा maintainable और testable बनाता है।

Marathi:
formik popular react form library आहे जे form handling simplify करते. form state, validation, submission manage करते. form, field, error components पुरवते. complex forms साठी boilerplate code कमी करते. yup integration सोबत form validation handle करते. forms ला अधिक maintainable आणि testable बनवते.

Analogy:
Like having a personal assistant for paperwork: Formik handles all the tedious form management tasks so you can focus on the important business logic.

Learn more:

- https://formik.org/

✅ 217. What are the advantages of using Formik over traditional form handling in React?

English:
Reduces boilerplate code, handles form state automatically, provides built-in validation, better error handling, easier testing, handles form submission, supports complex forms. Traditional React forms require manual state management, validation logic, error handling. Formik provides higher-level abstraction.

SKILLHUB

Hindi:
boilerplate code कम करता है, form state automatically handle करता है, built-in validation provide करता है, better error handling, easier testing, form submission handle करता है, complex forms support करता है। traditional react forms manual state management, validation logic, error handling require करते हैं। formik higher-level abstraction provide करता है।

Marathi:
boilerplate code कमी करतो, form state automatically handle करतो, built-in validation पुरवतो, better error handling, easier testing, form submission handle करतो, complex forms support करतो. traditional react forms manual state management, validation logic, error handling require करतात. formik higher-level abstraction पुरवतो.

Analogy:
Like using a word processor vs. handwriting: Formik automates the formatting, spell-checking, and layout so you can focus on content.

Learn more:

- https://formik.org/docs/overview

✅ 218. What is Axios, and what problem does it solve in JavaScript or React development?

English:
Axios is HTTP client library for making API requests. Solves XMLHttpRequest complexity, provides promise-based API, automatic JSON transformation, request/response interceptors, error handling. Better than fetch() for complex applications. Handles timeouts, cancellations, progress tracking.

Hindi:
axios API requests बनाने के लिए HTTP client library है। xmlHttpRequest complexity को solve करता है, promise-based API provide करता है, automatic JSON transformation, request/response interceptors, error handling। complex applications के लिए fetch() से better। timeouts, cancellations, progress tracking handle करता है।

Marathi:
axios API requests तयार करण्यासाठी HTTP client library आहे. xmlHttpRequest complexity solve करते, promise-based API पुरवते, automatic JSON transformation, request/response interceptors, error handling. complex applications साठी fetch() पेक्षा better. timeouts, cancellations, progress tracking handle करते.

Analogy:
Like having a reliable delivery service: Axios handles all the complexities of shipping packages (API requests) so you don't have to worry about traffic, weather, or customs.

SKILLHUB

Learn more:

- https://axios-http.com/

✅ 219. What are the advantages of using Axios over the built-in fetch API in JavaScript?

English:
Better error handling, automatic JSON parsing, request/response interceptors, timeout support, request cancellation, progress tracking, browser compatibility, more intuitive API. Fetch requires manual error checking, JSON parsing, has no timeout by default. Axios has more features out-of-the-box.

Hindi:
better error handling, automatic JSON parsing, request/response interceptors, timeout support, request cancellation, progress tracking, browser compatibility, more intuitive API। fetch manual error checking, JSON parsing require करता है, default में कोई timeout नहीं। axios में out-of-the-box ज्यादा features हैं।

Marathi:
better error handling, automatic JSON parsing, request/response interceptors, timeout support, request cancellation, progress tracking, browser compatibility, more intuitive API. fetch manual error checking, JSON parsing require करतो, default मध्ये कोणताही timeout नाही. axios मध्ये out-of-the-box अधिक features आहेत.

Analogy:
Like choosing between a basic bicycle vs. an e-bike: fetch is functional but requires effort, Axios comes with motor assistance (extra features).

Learn more:

- https://axios-http.com/docs/intro

✅ 220. What is Material-UI, and what is its purpose in React development?

English:
Material-UI is React component library implementing Google Material Design. Provides pre-built, customizable components like buttons, cards, dialogs. Ensures consistent design, faster development, responsive components. Follows Material Design principles. Reduces need for custom CSS and design work.

SKILLHUB

Hindi:
material-ui google material design implement करने वाला react component library है। buttons, cards, dialogs जैसे pre-built, customizable components provide करता है। consistent design ensure करता है, faster development, responsive components। material design principles follow करता है। custom css और design work की need कम करता है।

Marathi:
material-ui google material design implement करणारा react component library आहे. buttons, cards, dialogs सारखे pre-built, customizable components पुरवतो. consistent design ensure करतो, faster development, responsive components. material design principles follow करतो. custom css आणि design work ची गरज कमी करतो.

Analogy:
Like using pre-cut lumber for construction: Material-UI provides ready-made, professionally designed building blocks so you can focus on architecture rather than carpentry.

Learn more:

- https://mui.com/

✅ 221. What is render hijacking in react?

English:
Render hijacking occurs when HOC or component modifies or replaces another component's render output. Using React.cloneElement to modify props/children of rendered elements. Can be used for theming, logging, or conditional rendering. But can make code complex and break component expectations.

Hindi:
render hijacking तब होता है जब hoc या component दूसरे component के render output को modify या replace करता है। rendered elements के props/children को modify करने के लिए react.cloneElement इस्तेमाल करना। theming, logging, या conditional rendering के लिए इस्तेमाल किया जा सकता है। लेकिन code को complex बना सकता है और component expectations को break कर सकता है।

Marathi:
render hijacking जेव्हा hoc किंवा component दुसऱ्या component च्या render output ला modify किंवा replace करतो तेव्हा होते. rendered elements चे props/children modify करण्यासाठी react.cloneElement वापरणे. theming, logging किंवा conditional rendering साठी वापरले जाऊ शकते. पण code ला complex बनवू शकते आणि component expectations ला break करू शकते.

Analogy:
Like intercepting mail: render hijacking "opens" and modifies what a component sends to the screen before it gets there.

SKILLHUB

Learn more:

- https://reactjs.org/docs/higher-order-components.html

✅ 222. What are Keyed Fragments?

English:
Keyed fragments are React fragments with key prop: <React.Fragment key={item.id}>. Used when rendering lists of fragments. Keys help React identify which fragments changed, added, or removed. Necessary when fragments are returned from map() or in arrays.

Hindi:
keyed fragments key prop वाले react fragments हैं: <react.fragment key={item.id}> । fragments की lists render करने के लिए इस्तेमाल किए जाते हैं। keys react को help करते हैं identify करने में कि कौन से fragments changed, added, या removed। जब fragments map() से returned होते हैं या arrays में तब necessary।

Marathi:
keyed fragments key prop असलेले react fragments आहेत: <react.fragment key={item.id}>. fragments च्या lists render करण्यासाठी वापरले जातात. keys react ला help करतात identify करण्यासाठी की कोणते fragments changed, added किंवा removed. जेव्हा fragments map() पासून returned असतात किंवा arrays मध्ये असतात तेव्हा necessary.

Analogy:
Like labeled boxes in storage: keyed fragments help React keep track of which "box" (fragment) goes where in the list.

Learn more:

- https://reactjs.org/docs/fragments.html

✅ 223. What is suspense component?

English:
Suspense enables components to "wait" for something before rendering. Used with React.lazy for code splitting, or data fetching libraries. Shows fallback UI while async operation completes. Wraps lazy-loaded components or components that suspend. Improves perceived performance.

Hindi:
suspense components को कुछ के लिए "wait" करने की अनुमति देता है rendering से पहले। code splitting के लिए react.lazy के साथ इस्तेमाल किया जाता है, या data fetching libraries। async operation complete होने तक fallback UI show करता है। lazy-loaded components या suspend करने वाले components को wrap करता है। perceived performance improve करता है।

SKILLHUB

Marathi:
suspense components ला rendering पूर्वी कशासाठी तरी "wait" करण्यास परवानगी देतो. code splitting साठी react.lazy सोबत वापरला जातो, किंवा data fetching libraries. async operation complete होईपर्यंत fallback UI show करतो. lazy-loaded components किंवा suspend करणारे components wrap करतो. perceived performance सुधारतो.

Analogy:
Like a waiting room: Suspense shows you a magazine (fallback UI) while you wait for your appointment (async operation) to be ready.

Learn more:

- https://reactjs.org/docs/code-splitting.html

✅ 224. Is it possible to use react without JSX?

English:
Yes, React can be used without JSX by using React.createElement(). JSX is syntactic sugar that compiles to createElement calls. Some developers prefer JSX for readability, others prefer pure JavaScript. Both approaches work identically - JSX is optional but widely used.

Hindi:
हां, react.createElement() इस्तेमाल करके react को jsx के बिना इस्तेमाल किया जा सकता है। jsx syntactic sugar है जो createElement calls में compile होता है। कुछ developers readability के लिए jsx prefer करते हैं, अन्य pure javascript। दोनों approaches identically work करते हैं - jsx optional है लेकिन widely इस्तेमाल किया जाता है।

Marathi:
होय, react.createElement() वापरून react ला jsx शिवाय वापरले जाऊ शकते. jsx syntactic sugar आहे जे createElement calls मध्ये compile होते. काही developers readability साठी jsx prefer करतात, इतर pure javascript. दोन्ही approaches identically work करतात - jsx optional आहे पण widely वापरले जाते.

Analogy:
Like writing in longhand vs. shorthand: you can write React without JSX (longhand with createElement), but JSX makes it more readable and concise.

Learn more:

- https://reactjs.org/docs/react-without-jsx.html

✅ 225. Why does strict mode render twice in React?

SKILLHUB

English:
StrictMode intentionally double-invokes functions during development to detect side effects. Includes render functions, constructors, state updaters, useEffect. Helps catch bugs from impure functions. Only in development, not production. Ensures components are resilient to multiple renders.

Hindi:
strictMode development में side effects detect करने के लिए intentionally functions को double-invoke करता है। render functions, constructors, state updaters, useEffect include करता है। impure functions से bugs catch करने में help करता है। सिर्फ development में, production में नहीं। components को multiple renders के लिए resilient बनाता है।

Marathi:
strictMode development मध्ये side effects detect करण्यासाठी intentionally functions ला double-invoke करतो. render functions, constructors, state updaters, useEffect include करतो. impure functions पासून bugs catch करण्यास help करतो. फक्त development मध्ये, production मध्ये नाही. components ला multiple renders साठी resilient बनवतो.

Analogy:
Like doing a double-take: StrictMode runs things twice to make sure they're behaving properly and not causing unexpected side effects.

Learn more:

- https://reactjs.org/docs/strict-mode.html

✅ 226. What is React Fiber?

English:
React Fiber is complete rewrite of React's reconciliation algorithm. Enables incremental rendering, pause/resume rendering, prioritize updates. Better performance for complex UIs, smoother animations. Introduced in React 16. Enables concurrent features like Suspense. Backward compatible.

Hindi:
react fiber react के reconciliation algorithm का complete rewrite है। incremental rendering enable करता है, rendering को pause/resume करता है, updates को prioritize करता है। complex uis के लिए better performance, smoother animations। react 16 में introduced। suspense जैसे concurrent features enable करता है। backward compatible।

Marathi:
react fiber react च्या reconciliation algorithm चा complete rewrite आहे. incremental rendering enable करतो, rendering ला pause/resume करतो, updates ला prioritize करतो. complex uis साठी better performance, smoother animations. react 16 मध्ये introduced. suspense सारखे concurrent features enable करतो. backward compatible.

SKILLHUB

Analogy:
Like upgrading from a single-lane road to a multi-lane highway: Fiber allows React to handle traffic (updates) more efficiently with better flow control.

Learn more:

- https://github.com/acdlite/react-fiber-architecture

✅ 227. How does React Fiber improve the overall performance of React applications?

English:
Fiber enables incremental rendering - break work into chunks, pause/resume as needed. Prioritizes urgent updates (user input) over less important ones. Better memory usage with work cancellation. Enables concurrent features like Suspense. Smoother animations and transitions. More responsive UIs.

Hindi:
fiber incremental rendering enable करता है - work को chunks में break करता है, needed के अनुसार pause/resume करता है। urgent updates (user input) को less important ones से prioritize करता है। work cancellation के साथ better memory usage। suspense जैसे concurrent features enable करता है। smoother animations और transitions। more responsive uis।

Marathi:
fiber incremental rendering enable करतो - work ला chunks मध्ये break करतो, needed प्रमाणे pause/resume करतो. urgent updates (user input) ला less important ones पेक्षा prioritize करतो. work cancellation सोबत better memory usage. suspense सारखे concurrent features enable करतो. smoother animations आणि transitions. अधिक responsive uis.

Analogy:
Like a smart task manager: Fiber breaks big jobs into smaller pieces and handles urgent tasks first, making everything feel more responsive.

Learn more:

- https://reactjs.org/docs/concurrent-mode-intro.html

✅ 228. What is Babel in React js?

English:
Babel is JavaScript transpiler that converts modern ES6+ code to ES5 for browser compatibility. Essential for React development to transform JSX. Configurable with presets and plugins. Handles TypeScript, converts arrow functions, classes, etc. Build tools like Create React App include Babel.

SKILLHUB

Hindi:
babel javascript transpiler है जो modern es6+ code को browser compatibility के लिए es5 में convert करता है। react development के लिए jsx transform करने के लिए essential। presets और plugins के साथ configurable। typescript handle करता है, arrow functions, classes, etc convert करता है। create react app जैसे build tools babel include करते हैं।

Marathi:
babel javascript transpiler आहे जे modern es6+ code ला browser compatibility साठी es5 मध्ये convert करते. react development साठी jsx transform करण्यासाठी essential. presets आणि plugins सोबत configurable. typescript handle करतो, arrow functions, classes, etc convert करतो. create react app सारखे build tools babel include करतात.

Analogy:
Like a language interpreter at the UN: Babel translates modern JavaScript "dialects" into a common language all browsers understand.

Learn more:

- https://babeljs.io/docs/en/babel-preset-react

✅ 229. What is a wrapper component in react?

English:
Wrapper component is higher-order component or container that wraps other components. Provides additional functionality like styling, data fetching, error handling. Can be HOC, render prop component, or component composition. Keeps wrapped components clean by handling cross-cutting concerns.

Hindi:
wrapper component higher-order component या container है जो अन्य components को wrap करता है। styling, data fetching, error handling जैसे additional functionality provide करता है। hoc, render prop component, या component composition हो सकता है। cross-cutting concerns handle करके wrapped components को clean रखता है।

Marathi:
wrapper component higher-order component किंवा container आहे जे इतर components ला wrap करते. styling, data fetching, error handling सारखी additional functionality पुरवते. hoc, render prop component किंवा component composition असू शकते. cross-cutting concerns handle करून wrapped components ला clean ठेवते.

Analogy:
Like gift wrapping: wrapper components add decorative layers (functionality) around the main present (component) without changing what's inside.

SKILLHUB

Learn more:

- https://reactjs.org/docs/higher-order-components.html

✅ 230. Explain DOM diffing?

English:
DOM diffing is React's algorithm comparing old virtual DOM with new virtual DOM to find changes. Only updates actual DOM for differences found. Uses efficient heuristics based on component types and keys. Enables React's performance by minimizing expensive DOM operations.

Hindi:
dom diffing react का algorithm है जो old virtual dom को new virtual dom से compare करता है changes find करने के लिए। सिर्फ differences के लिए actual dom को update करता है। component types और keys पर based efficient heuristics इस्तेमाल करता है। expensive dom operations को minimize करके react की performance enable करता है।

Marathi:
dom diffing react चा algorithm आहे जो old virtual dom ला new virtual dom सोबत compare करतो changes find करण्यासाठी. फक्त differences साठी actual dom update करतो. component types आणि keys वर based efficient heuristics वापरतो. expensive dom operations minimize करून react ची performance enable करतो.

Analogy:
Like proofreading: DOM diffing compares the old manuscript (virtual DOM) with the new one, marking only the changed words instead of rewriting the entire book.

Learn more:

- https://reactjs.org/docs/reconciliation.html

✅ 231. What are the dependencies in the dependency array of the useEffect hook? How do they affect the behavior of the hook?

English:
Dependencies are values useEffect watches for changes. When dependencies change, effect re-runs. Empty array [] runs once on mount. No array runs on every render. Including wrong dependencies causes infinite loops or stale closures. Must include all values used inside effect.

Hindi:
dependencies वे values हैं जिनको useEffect changes के लिए watch करता है। जब dependencies change होते

SKILLHUB

हैं, तो effect re-runs। empty array [] mount पर एक बार run होता है। no array हर render पर run होता है। wrong dependencies include करने से infinite loops या stale closures होते हैं। effect के अंदर इस्तेमाल किए गए सभी values को include करना जरूरी है।

Marathi:
dependencies ती values आहेत ज्या useEffect changes साठी watch करतो. जेव्हा dependencies change होतात, तेव्हा effect re-runs. empty array [] mount वर एकदा run होते. no array प्रत्येक render वर run होते. wrong dependencies include केल्याने infinite loops किंवा stale closures होतात. effect च्या आत वापरलेल्या सर्व values ला include करणे आवश्यक आहे.

Analogy:
Like a security system with motion sensors: dependencies tell useEffect "watch these specific areas (values), and alert (re-run effect) when anything moves (changes)."

Learn more:

- https://reactjs.org/docs/hooks-effect.html

✅ 232. What are some common use cases for the useEffect hook?

English:
Data fetching on component mount, subscribing to external data sources, setting up timers/intervals, manual DOM manipulation, cleaning up resources, syncing with external APIs, handling window events. Runs after render, perfect for side effects that shouldn't block UI.

Hindi:
component mount पर data fetching, external data sources को subscribe करना, timers/intervals setup करना, manual dom manipulation, resources clean up करना, external apis के साथ syncing, window events handle करना। render के बाद runs करता है, side effects के लिए perfect जो ui को block नहीं करते।

Marathi:
component mount वर data fetching, external data sources ला subscribe करणे, timers/intervals setup करणे, manual dom manipulation, resources clean up करणे, external apis सोबत syncing, window events handle करणे. render नंतर runs करतो, side effects साठी perfect जे ui ला block करत नाहीत.

Analogy:
Like a personal assistant who handles errands: useEffect takes care of background tasks (data fetching, subscriptions) after the main work (rendering) is done.

Learn more:

- https://reactjs.org/docs/hooks-effect.html

SKILLHUB

✅ 233. Can you explain the concept of multiple useEffect hooks in a single component? How do they interact with each other?

English:
Multiple useEffect hooks run independently in order declared. Each has own dependency array and cleanup function. Don't share state between effects - use multiple useState instead. Effects run in sequence but don't block each other. Useful for separating concerns (data fetching, event listeners, etc.).

Hindi:
multiple useEffect hooks declared order में independently run करते हैं। हर effect का अपना dependency array और cleanup function होता है। effects के बीच state share न करें - इसके बजाय multiple useState इस्तेमाल करें। effects sequence में run करते हैं लेकिन एक दूसरे को block नहीं करते। concerns separate करने के लिए useful (data fetching, event listeners, etc.)।

Marathi:
multiple useEffect hooks declared order मध्ये independently run करतात. प्रत्येक effect चा स्वतःचा dependency array आणि cleanup function असतो. effects दरम्यान state share करू नका - त्याऐवजी multiple useState वापरा. effects sequence मध्ये run करतात पण एकमेकांना block करत नाहीत. concerns separate करण्यासाठी useful (data fetching, event listeners, etc.).

Analogy:
Like having multiple personal assistants: each useEffect handles their own specialty (data, events, cleanup) without interfering with others' work.

Learn more:

- https://reactjs.org/docs/hooks-effect.html

✅ 234. What is the purpose of the cleanup function returned by the useEffect hook? How can you utilize it effectively?

English:
Cleanup function runs when component unmounts or before effect re-runs. Used to prevent memory leaks by cleaning subscriptions, timers, event listeners. Return function from useEffect callback. Essential for performance and preventing bugs. Runs synchronously before new effect.

Hindi:
cleanup function तब runs करता है जब component unmount होता है या effect re-runs से पहले। subscriptions, timers, event listeners को clean करके memory leaks prevent करने के लिए इस्तेमाल किया

SKILLHUB

जाता है। useEffect callback से function return करें। performance और bugs prevent करने के लिए essential। new effect से पहले synchronously runs करता है।

Marathi:
cleanup function जेव्हा component unmount होते किंवा effect re-runs पूर्वी runs करते. subscriptions, timers, event listeners clean करून memory leaks prevent करण्यासाठी वापरले जाते. useEffect callback पासून function return करा. performance आणि bugs prevent करण्यासाठी essential. new effect पूर्वी synchronously runs करतो.

Analogy:
Like cleaning up after a party: the cleanup function tidies up (removes listeners, cancels timers) before leaving or starting fresh.

Learn more:

- https://reactjs.org/docs/hooks-effect.html

✅ 235. What are the potential pitfalls or common mistakes when using the useEffect hook? How can you avoid them?

English:
Infinite loops from missing/incorrect dependencies, stale closures, forgetting cleanup functions, running effects on every render, overusing for state updates. Avoid by including all dependencies, using useCallback for functions, proper cleanup, using correct dependency arrays, preferring useState for state updates.

Hindi:
missing/incorrect dependencies से infinite loops, stale closures, cleanup functions भूलना, हर render पर effects run करना, state updates के लिए overusing। सभी dependencies include करके, functions के लिए useCallback इस्तेमाल करके, proper cleanup, correct dependency arrays, state updates के लिए useState prefer करके avoid करें।

Marathi:
missing/incorrect dependencies पासून infinite loops, stale closures, cleanup functions विसरणे, प्रत्येक render वर effects run करणे, state updates साठी overusing. सर्व dependencies include करून, functions साठी useCallback वापरून, proper cleanup, correct dependency arrays, state updates साठी useState prefer करून avoid करा.

Analogy:
Like avoiding kitchen disasters: careful ingredient measuring (dependencies), proper cleanup (cleanup functions), and not leaving the stove on (infinite loops).

Learn more:

SKILLHUB

- https://reactjs.org/docs/hooks-effect.html

SKILLHUB

ReactJS SSR & Advanced Topics by SKILLHUB - Day 14

✅ 236. What is server-side rendering (SSR) in React.js?

English:
SSR renders React components on server, sends HTML to client. Client then hydrates with JavaScript for interactivity. Improves initial page load, SEO, social sharing. Requires server setup (Node.js with Express). More complex than client-side rendering but better for performance and SEO.

Hindi:
ssr react components को server पर render करता है, html को client को send करता है। client फिर javascript के साथ hydrate करता है interactivity के लिए। initial page load, seo, social sharing improve करता है। server setup require करता है (node.js with express)। client-side rendering से ज्यादा complex लेकिन performance और seo के लिए better।

Marathi:
ssr react components ला server वर render करते, html ला client ला send करते. client नंतर javascript सोबत hydrate करते interactivity साठी. initial page load, seo, social sharing सुधारते. server setup require करते (node.js with express). client-side rendering पेक्षा अधिक complex पण performance आणि seo साठी better.

Analogy:
Like baking a cake in the oven (server) vs. mixing ingredients in front of guests (client): SSR serves a complete, ready-to-eat cake immediately.

Learn more:

- https://reactjs.org/docs/react-dom-server.html

✅ 237. Why would you choose to use server-side rendering instead of client-side rendering in React.js?

English:
Better SEO (search engines see content immediately), faster initial page loads, better social media sharing (crawlers see content), improved performance on slow devices/networks, better accessibility. Essential for content-heavy sites, e-commerce, blogs. Client-side rendering has slower initial loads.

Hindi:
better seo (search engines content immediately देखते हैं), faster initial page loads, better social media

SKILLHUB

sharing (crawlers content देखते हैं), slow devices/networks पर improved performance, better accessibility। content-heavy sites, e-commerce, blogs के लिए essential। client-side rendering में slower initial loads।

Marathi:
better seo (search engines content immediately पाहतात), faster initial page loads, better social media sharing (crawlers content पाहतात), slow devices/networks वर improved performance, better accessibility. content-heavy sites, e-commerce, blogs साठी essential. client-side rendering मध्ये slower initial loads.

Analogy:
Like serving a complete meal vs. giving guests raw ingredients: SSR provides immediately consumable content, while CSR requires preparation time.

Learn more:

- https://developers.google.com/web/updates/2019/02/rendering-on-the-web

---

✅ 238. How does server-side rendering differ from client-side rendering in React.js?

English:
SSR: components render on server, HTML sent to browser, then hydrated with JS. CSR: HTML shell sent first, JS downloads/executes, then renders. SSR shows content faster but requires server. CSR more interactive initially but slower first load. SSR better for SEO, CSR better for SPAs.

Hindi:
ssr: components server पर render होते हैं, html browser को send किया जाता है, फिर js के साथ hydrated। csr: पहले html shell send किया जाता है, js downloads/executes, फिर renders। ssr content को faster show करता है लेकिन server require करता है। csr initially ज्यादा interactive लेकिन slower first load। ssr seo के लिए better, csr spas के लिए better।

Marathi:
ssr: components server वर render होतात, html browser ला send केले जाते, नंतर js सोबत hydrated. csr: पहिले html shell send केले जाते, js downloads/executes, नंतर renders. ssr content ला faster show करतो पण server require करतो. csr initially अधिक interactive पण slower first load. ssr seo साठी better, csr spas साठी better.

Analogy:
SSR is like getting a fully decorated cake (ready content), CSR is like getting an undecorated cake that you assemble yourself (builds interactivity).

Learn more:

SKILLHUB

- https://developers.google.com/web/updates/2019/02/rendering-on-the-web

---

✅ 239. Explain the benefits and drawbacks of server-side rendering in React.js.

English:
Benefits: Better SEO, faster initial loads, improved social sharing, better performance on slow connections. Drawbacks: Server load/cost, complexity, slower TTFB for dynamic content, hydration mismatch issues, limited browser APIs during render. Requires careful state management.

Hindi:
benefits: better seo, faster initial loads, improved social sharing, slow connections पर better performance। drawbacks: server load/cost, complexity, dynamic content के लिए slower ttfb, hydration mismatch issues, render के दौरान limited browser apis। careful state management require करता है।

Marathi:
benefits: better seo, faster initial loads, improved social sharing, slow connections वर better performance. drawbacks: server load/cost, complexity, dynamic content साठी slower ttfb, hydration mismatch issues, render दरम्यान limited browser apis. careful state management require करतो.

Analogy:
Like owning a restaurant vs. catering: SSR provides consistent quality (SEO, performance) but requires infrastructure (servers, complexity).

Learn more:

- https://web.dev/rendering-on-the-web/

✅ 240. What are the performance implications of server-side rendering in React.js?

English:
Better initial load performance, improved Core Web Vitals. Server processing adds latency. Hydration can cause layout shifts. Bundle size still matters. Better caching strategies possible. Trade-off between server load and improved user experience. Memory usage on server vs. client.

Hindi:
better initial load performance, improved core web vitals। server processing latency add करता है। hydration layout shifts का कारण बन सकता है। bundle size अभी भी matters। better caching strategies possible। server load और improved user experience के बीच trade-off। server vs client पर memory usage।

SKILLHUB

Marathi:
better initial load performance, improved core web vitals. server processing latency add करतो. hydration layout shifts चा कारण होऊ शकतो. bundle size अजूनही matters. better caching strategies possible. server load आणि improved user experience दरम्यान trade-off. server vs client वर memory usage.

Analogy:
Like express delivery vs. standard shipping: SSR gets content to users faster initially, but requires more infrastructure overhead.

Learn more:

- https://web.dev/rendering-on-the-web/

✅ 241. How can you implement server-side rendering with React.js without using frameworks like Next.js?

English:
Use ReactDOMServer.renderToString() or renderToNodeStream() on server. Set up Express server, handle routing manually. Share components between client/server. Handle data fetching on server. Use hydrate() on client. Manage CSS/styling carefully. Implement proper error handling and caching.

Hindi:
server पर reactdomserver.rendertostring() या rendertonodestream() इस्तेमाल करें। express server set up करें, routing manually handle करें। client/server के बीच components share करें। server पर data fetching handle करें। client पर hydrate() इस्तेमाल करें। css/styling को carefully manage करें। proper error handling और caching implement करें।

Marathi:
server वर reactdomserver.rendertostring() किंवा rendertonodestream() वापरा. express server set up करा, routing manually handle करा. client/server दरम्यान components share करा. server वर data fetching handle करा. client वर hydrate() वापरा. css/styling carefully manage करा. proper error handling आणि caching implement करा.

Analogy:
Like building a house without pre-fab materials: you assemble all the pieces manually, giving you complete control but requiring more work.

Learn more:

- https://reactjs.org/docs/react-dom-server.html

SKILLHUB

✅ 242. Describe the steps involved in setting up server-side rendering with React.js from scratch.

English:

1. Set up Node.js/Express server. 2. Install React/ReactDOM. 3. Create shared components. 4. Set up client/server routing. 5. Implement data fetching on server. 6. Use renderToString() for HTML generation. 7. Send HTML with initial data. 8. Hydrate on client. 9. Handle CSS and assets. 10. Test and optimize.

Hindi:

1. node.js/express server set up करें। 2. react/reactdom install करें। 3. shared components create करें। 4. client/server routing set up करें। 5. server पर data fetching implement करें। 6. html generation के लिए rendertostring() इस्तेमाल करें। 7. initial data के साथ html send करें। 8. client पर hydrate करें। 9. css और assets handle करें। 10. test और optimize करें।

Marathi:

1. node.js/express server set up करा. 2. react/reactdom install करा. 3. shared components create करा. 4. client/server routing set up करा. 5. server वर data fetching implement करा. 6. html generation साठी rendertostring() वापरा. 7. initial data सोबत html send करा. 8. client वर hydrate करा. 9. css आणि assets handle करा. 10. test आणि optimize करा.

Analogy:
Like building a custom car from parts: each step requires careful assembly of components to create a working SSR system.

Learn more:

- https://github.com/ReactTraining/react-router/blob/master/packages/react-router-dom/docs/guides/server-rendering.md

---

✅ 243. What libraries or tools can be used to perform server-side rendering with React.js?

English:
Next.js, Gatsby, Razzle, After.js for frameworks. Express, Koa, Hapi for servers. React Router for routing. Styled Components, Emotion for styling. Redux, MobX for state. Webpack for bundling. Loadable Components for code splitting. React Helmet for head management.

Hindi:
frameworks के लिए next.js, gatsby, razzle, after.js। servers के लिए express, koa, hapi। routing के लिए

SKILLHUB

react router। styling के लिए styled components, emotion। state के लिए redux, mobx। bundling के लिए webpack। code splitting के लिए loadable components। head management के लिए react helmet।

Marathi:
frameworks साठी next.js, gatsby, razzle, after.js. servers साठी express, koa, hapi. routing साठी react router. styling साठी styled components, emotion. state साठी redux, mobx. bundling साठी webpack. code splitting साठी loadable components. head management साठी react helmet.

Analogy:
Like a mechanic's toolbox: each tool (library) serves a specific purpose in building and maintaining the SSR engine.

Learn more:

- https://reactjs.org/docs/react-dom-server.html

✅ 244. How can you handle data fetching and asynchronous operations during server-side rendering in React.js?

English:
Fetch data on server before rendering, pass as props to components. Use getServerSideProps in Next.js or custom solution. Handle promises carefully. Avoid direct API calls in render. Use suspense for async components. Handle errors gracefully. Consider caching strategies.

Hindi:
render से पहले server पर data fetch करें, components को props के रूप में pass करें। next.js में getserversideprops इस्तेमाल करें या custom solution। promises को carefully handle करें। render में direct api calls avoid करें। async components के लिए suspense इस्तेमाल करें। errors को gracefully handle करें। caching strategies consider करें।

Marathi:
render पूर्वी server वर data fetch करा, components ला props म्हणून pass करा. next.js मध्ये getserversideprops वापरा किंवा custom solution. promises carefully handle करा. render मध्ये direct api calls avoid करा. async components साठी suspense वापरा. errors gracefully handle करा. caching strategies consider करा.

Analogy:
Like preparing ingredients before cooking: gather all data on the server before rendering, so the component gets everything it needs immediately.

Learn more:

- https://nextjs.org/docs/basic-features/data-fetching

SKILLHUB

✅ 245. What considerations should you keep in mind when implementing server-side rendering for a large-scale React.js application?

English:
Server costs and scaling, caching strategies, bundle size optimization, memory usage, error handling, SEO requirements, performance monitoring, state management complexity, third-party library compatibility, development vs production differences, security implications.

Hindi:
server costs और scaling, caching strategies, bundle size optimization, memory usage, error handling, seo requirements, performance monitoring, state management complexity, third-party library compatibility, development vs production differences, security implications।

Marathi:
server costs आणि scaling, caching strategies, bundle size optimization, memory usage, error handling, seo requirements, performance monitoring, state management complexity, third-party library compatibility, development vs production differences, security implications.

Analogy:
Like planning a large banquet: SSR for big applications requires careful consideration of resources, logistics, and scalability.

Learn more:

- https://developers.google.com/web/updates/2019/02/rendering-on-the-web

---

✅ 246. Can you explain the concept of code splitting and how it relates to server-side rendering in React.js?

English:
Code splitting breaks bundle into smaller chunks loaded on demand. Improves initial load time. In SSR, load only critical code initially, lazy load rest. Use React.lazy() and Suspense. Related to SSR through improved performance and caching. Enables better SEO and user experience.

Hindi:
code splitting bundle को smaller chunks में break करता है जो on demand load होते हैं। initial load time improve करता है। ssr में, सिर्फ critical code initially load करें, rest को lazy load करें। react.lazy() और suspense इस्तेमाल करें। improved performance और caching के माध्यम से ssr से related। better seo और user experience enable करता है।

Marathi:
code splitting bundle ला smaller chunks मध्ये break करते जे on demand load होतात. initial load time

SKILLHUB

सुधारते. ssr मध्ये, फक्त critical code initially load करा, rest ला lazy load करा. react.lazy() आणि suspense वापरा. improved performance आणि caching द्वारे ssr शी related. better seo आणि user experience enable करते.

Analogy:
Like serving courses separately: code splitting delivers the essential "appetizer" (critical code) first, then brings additional "courses" as needed.

Learn more:

- https://reactjs.org/docs/code-splitting.html

✅ 247. How can you optimize server-side rendered React.js applications for search engine optimization (SEO)?

English:
Ensure all content renders on server, meta tags set correctly, structured data implemented, proper heading hierarchy, fast loading times, mobile-friendly design, clean URLs, sitemap submission, avoid client-side only content, use semantic HTML.

Hindi:
सुनिश्चित करें कि सभी content server पर render होता है, meta tags correctly set हैं, structured data implemented है, proper heading hierarchy, fast loading times, mobile-friendly design, clean urls, sitemap submission, client-side only content avoid करें, semantic html इस्तेमाल करें।

Marathi:
सुनिश्चित करा की सर्व content server वर render होते, meta tags correctly set आहेत, structured data implemented आहे, proper heading hierarchy, fast loading times, mobile-friendly design, clean urls, sitemap submission, client-side only content avoid करा, semantic html वापरा.

Analogy:
Like preparing a house for real estate photos: SSR ensures search engines see the "perfectly staged" version of your content immediately.

Learn more:

- https://developers.google.com/search/docs/beginner/seo-starter-guide

---

✅ 248. What are some common challenges or pitfalls associated with server-side rendering in React.js, and how can you address them?

SKILLHUB

English:
Hydration mismatches, slower TTFB, server costs, memory leaks, state management complexity, third-party script issues, CSS-in-JS problems, debugging difficulties. Address with proper error boundaries, caching, monitoring, careful state handling, isomorphic libraries.

Hindi:
hydration mismatches, slower ttfb, server costs, memory leaks, state management complexity, third-party script issues, css-in-js problems, debugging difficulties। proper error boundaries, caching, monitoring, careful state handling, isomorphic libraries के साथ address करें।

Marathi:
hydration mismatches, slower ttfb, server costs, memory leaks, state management complexity, third-party script issues, css-in-js problems, debugging difficulties. proper error boundaries, caching, monitoring, careful state handling, isomorphic libraries सोबत address करा.

Analogy:
Like sailing challenges: SSR has its rough waters (complexity, costs), but with good navigation (proper setup, monitoring) you can sail smoothly.

Learn more:

- https://developers.google.com/web/updates/2019/02/rendering-on-the-web

---

✅ 249. How does server-side rendering impact the development and debugging process compared to client-side rendering?

English:
More complex setup, harder debugging (server logs vs browser dev tools), development/production parity issues, build process complexity, testing challenges. But better for SEO/performance testing. Requires different debugging approaches, careful logging, isomorphic code.

Hindi:
more complex setup, debugging harder (server logs vs browser dev tools), development/production parity issues, build process complexity, testing challenges। लेकिन seo/performance testing के लिए better। different debugging approaches require करता है, careful logging, isomorphic code।

Marathi:
more complex setup, debugging harder (server logs vs browser dev tools), development/production parity issues, build process complexity, testing challenges. पण seo/performance testing साठी better. different debugging approaches require करतो, careful logging, isomorphic code.

SKILLHUB

Analogy:
Like debugging a car vs. an airplane: SSR development is more complex (multiple environments), but the results are worth the extra effort.

Learn more:

- https://developers.google.com/web/tools/chrome-devtools

✅ 250. Can you explain the concept of hydration in the context of server-side rendering in React.js?

English:
Hydration attaches event handlers to server-rendered HTML, making it interactive. React compares server HTML with client component tree, attaches listeners without re-rendering. If mismatch, shows warning. Essential for SSR to work. Happens after initial HTML load.

Hindi:
hydration server-rendered html को event handlers attach करता है, इसे interactive बनाता है। react server html को client component tree से compare करता है, re-rendering के बिना listeners attach करता है। अगर mismatch तो warning show करता है। ssr के work करने के लिए essential। initial html load के बाद होता है।

Marathi:
hydration server-rendered html ला event handlers attach करते, त्याला interactive बनवते. react server html ला client component tree सोबत compare करते, re-rendering शिवाय listeners attach करते. जर mismatch तर warning show करते. ssr च्या work करण्यासाठी essential. initial html load नंतर होते.

Analogy:
Like bringing a mannequin to life: hydration takes the static "dummy" (server HTML) and connects it with the nervous system (event handlers) to make it fully functional.

Learn more:

- https://reactjs.org/docs/react-dom.html#hydrate

✅ 251. What are the best practices for organizing and structuring a React application?

English:
Use feature-based folder structure, separate components/containers, use index.js for clean imports, implement proper naming conventions, use custom hooks for shared logic, keep components small and focused, implement proper error boundaries, use TypeScript for large apps.

SKILLHUB

Hindi:
feature-based folder structure इस्तेमाल करें, components/containers को separate करें, clean imports के लिए index.js इस्तेमाल करें, proper naming conventions implement करें, shared logic के लिए custom hooks इस्तेमाल करें, components को small और focused रखें, proper error boundaries implement करें, large apps के लिए typescript इस्तेमाल करें।

Marathi:
feature-based folder structure वापरा, components/containers ला separate करा, clean imports साठी index.js वापरा, proper naming conventions implement करा, shared logic साठी custom hooks वापरा, components ला small आणि focused ठेवा, proper error boundaries implement करा, large apps साठी typescript वापरा.

Analogy:
Like organizing a well-stocked kitchen: everything has its place, ingredients are grouped logically, and you can find what you need quickly.

Learn more:

- https://reactjs.org/docs/thinking-in-react.html

✅ 252. What are the challenges you have faced working with React?

English:
Learning curve with JSX/hooks, state management complexity, performance optimization, debugging async code, keeping up with frequent updates, integrating with legacy code, testing component interactions, bundle size management, SEO challenges with SPAs.

Hindi:
jsx/hooks के साथ learning curve, state management complexity, performance optimization, async code debugging, frequent updates के साथ keep up करना, legacy code के साथ integration, component interactions testing, bundle size management, spas के साथ seo challenges।

Marathi:
jsx/hooks सोबत learning curve, state management complexity, performance optimization, async code debugging, frequent updates सोबत keep up करणे, legacy code सोबत integration, component interactions testing, bundle size management, spas सोबत seo challenges.

Analogy:
Like learning to drive: React has its challenging moments (steep learning curve, complex state), but mastery brings great freedom and capability.

Learn more:

SKILLHUB

- https://reactjs.org/docs/getting-started.html

✅ 253. Why do we need Nextjs?

English:
Next.js provides SSR, static site generation, API routes, built-in optimizations. Better SEO/performance than CRA. File-based routing, image optimization, zero config. Perfect for production React apps. Reduces boilerplate for common features.

Hindi:
next.js ssr, static site generation, api routes, built-in optimizations provide करता है। cra से better seo/performance। file-based routing, image optimization, zero config। production react apps के लिए perfect। common features के लिए boilerplate कम करता है।

Marathi:
next.js ssr, static site generation, api routes, built-in optimizations पुरवते. cra पेक्षा better seo/performance. file-based routing, image optimization, zero config. production react apps साठी perfect. common features साठी boilerplate कमी करते.

Analogy:
Like having a personal chef: Next.js handles all the complex cooking (SSR, routing, optimization) so you can focus on creating great dishes (your app).

Learn more:

- https://nextjs.org/docs/getting-started

✅ 256. What is the difference between React.memo, useMemo, and useCallback?

English:
React.memo is a higher-order component that memoizes entire components, preventing re-renders when props haven't changed. useMemo memoizes expensive computations, recalculating only when dependencies change. useCallback memoizes functions to prevent unnecessary re-renders of child components receiving them as props. All prevent unnecessary work but serve different purposes.

Hindi:
react.memo एक higher-order component है जो entire components को memoize करता है, जब props change नहीं हुए हैं तब re-renders prevent करता है। usememo expensive computations को memoize करता है, सिर्फ तब recalculate करता है जब dependencies change होते हैं। usecallback functions को memoize करता है child components के unnecessary re-renders को prevent करने के लिए जो उन्हें props के रूप में receive करते हैं। सभी unnecessary work को prevent करते हैं लेकिन different purposes serve करते हैं।

SKILLHUB

Marathi:
react.memo एक higher-order component आहे जे entire components ला memoize करते, जेव्हा props change झालेले नाहीत तेव्हा re-renders prevent करते. usememo expensive computations ला memoize करते, फक्त तेव्हा recalculate करते जेव्हा dependencies change होतात. usecallback functions ला memoize करते child components च्या unnecessary re-renders prevent करण्यासाठी जी त्यांना props म्हणून receive करतात. सर्व unnecessary work prevent करतात पण different purposes serve करतात.

Analogy:
Like different types of caching: React.memo caches entire pages, useMemo caches calculations, useCallback caches the buttons that trigger actions.

Learn more:

- https://reactjs.org/docs/react-api.html#reactmemo
- https://reactjs.org/docs/hooks-reference.html#usememo
- https://reactjs.org/docs/hooks-reference.html#usecallback

✅ 257. How does React's reconciliation algorithm work and how can it be optimized?

English:
React's reconciliation compares virtual DOM with real DOM using diffing algorithm. It identifies changes through component types, keys, and props. Optimization techniques include using keys for dynamic lists, implementing shouldComponentUpdate, using React.memo, and structuring components to minimize cascading updates. The algorithm aims to minimize expensive DOM operations.

Hindi:
react की reconciliation virtual dom को real dom से diffing algorithm का इस्तेमाल करके compare करती है। यह component types, keys, और props के माध्यम से changes identify करती है। optimization techniques में dynamic lists के लिए keys इस्तेमाल करना, shouldcomponentupdate implement करना, react.memo इस्तेमाल करना, और cascading updates को minimize करने के लिए components को structure करना शामिल हैं। algorithm expensive dom operations को minimize करने का aim रखता है।

Marathi:
react ची reconciliation virtual dom ला real dom सोबत diffing algorithm वापरून compare करते. हे component types, keys आणि props द्वारे changes identify करते. optimization techniques मध्ये dynamic lists साठी keys वापरणे, shouldcomponentupdate implement करणे, react.memo वापरणे आणि cascading updates minimize करण्यासाठी components ला structure करणे समाविष्ट आहे. algorithm expensive dom operations minimize करण्याचा aim घेते.

SKILLHUB

Analogy:
Like a smart proofreader: reconciliation carefully compares two versions of a document, marking only the specific words that changed rather than rewriting the entire page.

Learn more:

- https://reactjs.org/docs/reconciliation.html

✅ 258. What are the most effective ways to implement code splitting in React applications?

English:
Use React.lazy() for component-based splitting, dynamic imports for route-based splitting, and third-party libraries like Loadable Components for more control. Implement error boundaries around lazy components. Use webpack magic comments for chunk naming. Consider bundle analyzer to optimize split points. Preload critical chunks for better UX.

Hindi:
component-based splitting के लिए react.lazy() इस्तेमाल करें, route-based splitting के लिए dynamic imports, और more control के लिए loadable components जैसे third-party libraries। lazy components के around error boundaries implement करें। chunk naming के लिए webpack magic comments इस्तेमाल करें। split points को optimize करने के लिए bundle analyzer consider करें। better ux के लिए critical chunks को preload करें।

Marathi:
component-based splitting साठी react.lazy() वापरा, route-based splitting साठी dynamic imports आणि अधिक control साठी loadable components सारखी third-party libraries. lazy components च्या around error boundaries implement करा. chunk naming साठी webpack magic comments वापरा. split points optimize करण्यासाठी bundle analyzer consider करा. better ux साठी critical chunks preload करा.

Analogy:
Like serving a multi-course meal: code splitting delivers the essential appetizer first, then brings additional courses as guests request them, keeping everyone satisfied without overwhelming the kitchen.

Learn more:

- https://reactjs.org/docs/code-splitting.html

✅ 259. How can you optimize bundle size in React applications?

SKILLHUB

English:
Use webpack bundle analyzer to identify large dependencies. Implement tree shaking to remove unused code. Use dynamic imports for code splitting. Optimize images and assets. Choose smaller alternatives for libraries. Implement lazy loading for routes and components. Use compression and minification. Monitor bundle size with size limits.

Hindi:
large dependencies identify करने के लिए webpack bundle analyzer इस्तेमाल करें। unused code remove करने के लिए tree shaking implement करें। code splitting के लिए dynamic imports इस्तेमाल करें। images और assets को optimize करें। libraries के लिए smaller alternatives choose करें। routes और components के लिए lazy loading implement करें। compression और minification इस्तेमाल करें। size limits के साथ bundle size monitor करें।

Marathi:
large dependencies identify करण्यासाठी webpack bundle analyzer वापरा. unused code remove करण्यासाठी tree shaking implement करा. code splitting साठी dynamic imports वापरा. images आणि assets optimize करा. libraries साठी smaller alternatives choose करा. routes आणि components साठी lazy loading implement करा. compression आणि minification वापरा. size limits सोबत bundle size monitor करा.

Analogy:
Like packing for a trip: bundle optimization removes unnecessary items, compresses clothes, and chooses efficient packing methods to fit everything in the suitcase without exceeding weight limits.

Learn more:

- https://web.dev/reduce-bundle-size/

✅ 260. How do you implement virtual scrolling for large lists in React?

English:
Use libraries like react-window or react-virtualized to render only visible items. Calculate item heights and container dimensions. Implement windowing to show only items in viewport plus buffer. Handle scrolling events to update visible range. Use fixed height items for best performance. Implement dynamic sizing for variable heights.

Hindi:
large lists के लिए सिर्फ visible items render करने के लिए react-window या react-virtualized जैसे libraries इस्तेमाल करें। item heights और container dimensions calculate करें। viewport में सिर्फ items show करने के लिए windowing implement करें plus buffer। visible range update करने के लिए scrolling events handle

SKILLHUB

करें। best performance के लिए fixed height items इस्तेमाल करें। variable heights के लिए dynamic sizing implement करें।

Marathi:
large lists साठी फक्त visible items render करण्यासाठी react-window किंवा react-virtualized सारखी libraries वापरा. item heights आणि container dimensions calculate करा. viewport मध्ये फक्त items show करण्यासाठी windowing implement करा plus buffer. visible range update करण्यासाठी scrolling events handle करा. best performance साठी fixed height items वापरा. variable heights साठी dynamic sizing implement करा.

Analogy:
Like viewing a long scroll through a small window: virtual scrolling shows only what's visible in the window, keeping the rest "off-screen" until needed.

Learn more:

- https://react-window.vercel.app/

SKILLHUB

ReactJS Production & Scalability by SKILLHUB - Day 15

✅ 261. How do you use React DevTools Profiler to identify performance bottlenecks?

English:
Open React DevTools, go to Profiler tab, start recording interactions. Look for components with high render time or frequent re-renders. Use flame graph to identify slow components, ranked chart for most re-rendered components. Analyze commit times and identify unnecessary renders. Use this data to apply targeted optimizations.

Hindi:
react devtools open करें, profiler tab में जाएं, interactions record करना start करें। high render time या frequent re-renders वाले components को look करें। slow components identify करने के लिए flame graph इस्तेमाल करें, most re-rendered components के लिए ranked chart। commit times analyze करें और unnecessary renders identify करें। targeted optimizations apply करने के लिए इस data का इस्तेमाल करें।

Marathi:
react devtools open करा, profiler tab मध्ये जा, interactions record करणे start करा. high render time किंवा frequent re-renders असलेले components पाहा. slow components identify करण्यासाठी flame graph वापरा, most re-rendered components साठी ranked chart. commit times analyze करा आणि unnecessary renders identify करा. targeted optimizations apply करण्यासाठी या data चा वापर करा.

Analogy:
Like using a car's diagnostic computer: React Profiler reads the "engine diagnostics" of your app, showing exactly which components are "running rough" and need tuning.

Learn more:

- https://reactjs.org/blog/2018/09/10/introducing-the-react-profiler.html

✅ 262. What are the common causes of memory leaks in React applications and how to prevent them?

English:
Memory leaks occur from unremoved event listeners, timers/intervals not cleared, closures capturing large objects, improper cleanup in useEffect, and subscriptions not unsubscribed. Prevent by cleaning

SKILLHUB

up in useEffect return function, using abort controllers for fetch requests, avoiding unnecessary closures, and proper dependency management.

Hindi:
memory leaks unremoved event listeners, cleared नहीं हुए timers/intervals, large objects capture करने वाले closures, useeffect में improper cleanup, और unsubscribed subscriptions से होते हैं। useeffect return function में cleaning up करके, fetch requests के लिए abort controllers इस्तेमाल करके, unnecessary closures avoid करके, और proper dependency management करके prevent करें।

Marathi:
memory leaks unremoved event listeners, cleared न झालेले timers/intervals, large objects capture करणारे closures, useeffect मध्ये improper cleanup आणि unsubscribed subscriptions पासून होतात. useeffect return function मध्ये cleaning up करून, fetch requests साठी abort controllers वापरून, unnecessary closures avoid करून आणि proper dependency management करून prevent करा.

Analogy:
Like leaving taps running: memory leaks are like forgetting to turn off the faucet, gradually wasting resources until everything overflows.

Learn more:

- https://reactjs.org/docs/hooks-effect.html

✅ 263. How do you implement windowing/virtualization for rendering thousands of items?

English:
Use react-window or react-virtualized libraries. Define item height and container dimensions. Library calculates visible range and renders only those items. Implement overscan for smooth scrolling. Handle dynamic heights with cell measurer. Use fixed heights for best performance. Integrate with infinite scrolling for large datasets.

Hindi:
react-window या react-virtualized libraries इस्तेमाल करें। item height और container dimensions define करें। library visible range calculate करती है और सिर्फ उन items को render करती है। smooth scrolling के लिए overscan implement करें। dynamic heights के लिए cell measurer इस्तेमाल करें। best performance के लिए fixed heights इस्तेमाल करें। large datasets के लिए infinite scrolling के साथ integrate करें।

Marathi:
react-window किंवा react-virtualized libraries वापरा. item height आणि container dimensions define करा. library visible range calculate करते आणि फक्त त्या items ला render करते. smooth scrolling साठी overscan implement करा. dynamic heights साठी cell measurer वापरा. best performance साठी fixed heights वापरा. large datasets साठी infinite scrolling सोबत integrate करा.

SKILLHUB

Analogy:
Like viewing a massive painting through a small frame: windowing shows only the part of the "painting" (data) that's currently in view, keeping the rest ready but not rendered.

Learn more:

- https://bvaughn.github.io/react-virtualized/

✅ 264. What strategies can be used to optimize React component re-renders?

English:
Use React.memo for functional components, implement shouldComponentUpdate for class components, optimize dependency arrays in hooks, lift state up to reduce prop drilling, use context selectively, implement proper key props for lists, and use useMemo for expensive calculations. Profile with React DevTools to identify problematic components.

Hindi:
functional components के लिए react.memo इस्तेमाल करें, class components के लिए shouldcomponentupdate implement करें, hooks में dependency arrays optimize करें, prop drilling कम करने के लिए state को lift up करें, context को selectively इस्तेमाल करें, lists के लिए proper key props implement करें, और expensive calculations के लिए usememo इस्तेमाल करें। problematic components identify करने के लिए react devtools के साथ profile करें।

Marathi:
functional components साठी react.memo वापरा, class components साठी shouldcomponentupdate implement करा, hooks मध्ये dependency arrays optimize करा, prop drilling कमी करण्यासाठी state ला lift up करा, context ला selectively वापरा, lists साठी proper key props implement करा आणि expensive calculations साठी usememo वापरा. problematic components identify करण्यासाठी react devtools सोबत profile करा.

Analogy:
Like optimizing a factory assembly line: each strategy reduces wasted motion and ensures components only "work" when necessary, improving overall efficiency.

Learn more:

- https://reactjs.org/docs/optimizing-performance.html

✅ 265. How do you handle large-scale state management performance in React?

SKILLHUB

English:
Use state management libraries like Redux Toolkit or Zustand for predictable updates. Implement selector functions to prevent unnecessary re-renders. Use useMemo for derived state. Split state into smaller pieces. Implement proper normalization. Use immer for immutable updates. Consider state machines for complex state transitions.

Hindi:
predictable updates के लिए redux toolkit या zustand जैसे state management libraries इस्तेमाल करें। unnecessary re-renders prevent करने के लिए selector functions implement करें। derived state के लिए usememo इस्तेमाल करें। state को smaller pieces में split करें। proper normalization implement करें। immutable updates के लिए immer इस्तेमाल करें। complex state transitions के लिए state machines consider करें।

Marathi:
predictable updates साठी redux toolkit किंवा zustand सारखी state management libraries वापरा. unnecessary re-renders prevent करण्यासाठी selector functions implement करा. derived state साठी usememo वापरा. state ला smaller pieces मध्ये split करा. proper normalization implement करा. immutable updates साठी immer वापरा. complex state transitions साठी state machines consider करा.

Analogy:
Like organizing a large warehouse: proper state management creates efficient storage systems, clear labeling (selectors), and organized workflows to handle large inventories without confusion.

Learn more:

- https://redux-toolkit.js.org/

✅ 266. How do you prevent XSS attacks in React applications?

English:
React automatically escapes content in JSX, preventing most XSS attacks. Never use dangerouslySetInnerHTML without sanitization. Validate and sanitize all user inputs. Use libraries like DOMPurify for HTML sanitization. Implement Content Security Policy (CSP) headers. Avoid eval() and inline event handlers with dynamic content. Use React Helmet for meta tag management.

Hindi:
react jsx में content को automatically escape करता है, most xss attacks को prevent करता है। sanitization के बिना dangerouslysetinnerhtml कभी इस्तेमाल न करें। सभी user inputs को validate और sanitize करें। html sanitization के लिए dompurify जैसे libraries इस्तेमाल करें। content security policy (csp) headers implement करें। dynamic content के साथ eval() और inline event handlers avoid करें। meta tag management के लिए react helmet इस्तेमाल करें।

SKILLHUB

Marathi:
react jsx मध्ये content ला automatically escape करते, most xss attacks prevent करते. sanitization शिवाय dangerouslysetinnerhtml कधीही वापरू नका. सर्व user inputs validate आणि sanitize करा. html sanitization साठी dompurify सारखी libraries वापरा. content security policy (csp) headers implement करा. dynamic content सोबत eval() आणि inline event handlers avoid करा. meta tag management साठी react helmet वापरा.

Analogy:
Like having a smart filter on your water faucet: React's automatic escaping acts as a security filter, catching dangerous "contaminants" (malicious scripts) before they can flow through to users.

Learn more:

- https://reactjs.org/docs/introducing-jsx.html

✅ 267. What is Content Security Policy (CSP) and how do you implement it in React?

English:
CSP is security standard that helps prevent XSS, clickjacking, and code injection attacks by controlling resource loading. Implement via HTTP headers or meta tags. In React, use react-helmet to set CSP meta tags. Define allowed sources for scripts, styles, images. Use 'nonce' or 'strict-dynamic' for dynamic content. Test thoroughly as overly restrictive CSP can break functionality.

Hindi:
csp security standard है जो resource loading को control करके xss, clickjacking, और code injection attacks को prevent करने में help करता है। http headers या meta tags के माध्यम से implement करें। react में, csp meta tags set करने के लिए react-helmet इस्तेमाल करें। scripts, styles, images के लिए allowed sources define करें। dynamic content के लिए 'nonce' या 'strict-dynamic' इस्तेमाल करें। thoroughly test करें क्योंकि overly restrictive csp functionality को break कर सकता है।

Marathi:
csp security standard आहे जे resource loading control करून xss, clickjacking आणि code injection attacks prevent करण्यास help करते. http headers किंवा meta tags द्वारे implement करा. react मध्ये, csp meta tags set करण्यासाठी react-helmet वापरा. scripts, styles, images साठी allowed sources define करा. dynamic content साठी 'nonce' किंवा 'strict-dynamic' वापरा. thoroughly test करा कारण overly restrictive csp functionality ला break करू शकते.

Analogy:
Like a nightclub bouncer: CSP checks IDs and only lets approved "guests" (scripts, styles) into your application, keeping out malicious intruders.

Learn more:

SKILLHUB

- https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP

✅ 268. How do you safely handle dangerouslySetInnerHTML in React?

English:
Only use dangerouslySetInnerHTML when absolutely necessary and content is trusted. Always sanitize HTML using libraries like DOMPurify before insertion. Validate content on server-side. Use CSS for styling instead of inline styles in HTML. Consider alternatives like markdown parsers or rich text editors. Implement proper error boundaries around dangerously rendered content.

Hindi:
dangerouslysetinnerhtml सिर्फ तब इस्तेमाल करें जब absolutely necessary हो और content trusted हो। insertion से पहले हमेशा dompurify जैसे libraries का इस्तेमाल करके html को sanitize करें। server-side पर content validate करें। html में inline styles के बजाय css for styling इस्तेमाल करें। markdown parsers या rich text editors जैसे alternatives consider करें। dangerously rendered content के around proper error boundaries implement करें।

Marathi:
dangerouslysetinnerhtml फक्त तेव्हा वापरा जेव्हा absolutely necessary असेल आणि content trusted असेल. insertion पूर्वी नेहमी dompurify सारखी libraries वापरून html sanitize करा. server-side वर content validate करा. html मध्ये inline styles ऐवजी css for styling वापरा. markdown parsers किंवा rich text editors सारखे alternatives consider करा. dangerously rendered content च्या around proper error boundaries implement करा.

Analogy:
Like handling a loaded gun: dangerouslySetInnerHTML is powerful but dangerous - you need safety protocols (sanitization) and to be absolutely sure it's pointed in the right direction (trusted content).

Learn more:

- https://reactjs.org/docs/dom-elements.html

✅ 269. What are the best practices for handling user input validation in React?

English:
Validate on both client and server side. Use controlled components for real-time validation. Implement proper error messaging. Use libraries like Yup or Joi for complex validation. Sanitize inputs to prevent XSS. Implement rate limiting for forms. Use HTML5 validation attributes as fallback. Provide clear feedback for validation errors.

SKILLHUB

Hindi:
client और server दोनों sides पर validate करें। real-time validation के लिए controlled components इस्तेमाल करें। proper error messaging implement करें। complex validation के लिए yup या joi जैसे libraries इस्तेमाल करें। xss prevent करने के लिए inputs को sanitize करें। forms के लिए rate limiting implement करें। fallback के रूप में html5 validation attributes इस्तेमाल करें। validation errors के लिए clear feedback provide करें।

Marathi:
client आणि server दोन्ही sides वर validate करा. real-time validation साठी controlled components वापरा. proper error messaging implement करा. complex validation साठी yup किंवा joi सारखी libraries वापरा. xss prevent करण्यासाठी inputs sanitize करा. forms साठी rate limiting implement करा. fallback म्हणून html5 validation attributes वापरा. validation errors साठी clear feedback provide करा.

Analogy:
Like airport security: multiple layers of validation (client-side checks, server verification) ensure only safe, properly formatted "passengers" (user inputs) make it through.

Learn more:

- https://reactjs.org/docs/forms.html

✅ 270. How do you prevent clickjacking attacks in React applications?

English:
Implement X-Frame-Options header to prevent iframe embedding. Use Content Security Policy with frame-ancestors directive. Implement frame-busting techniques. Add proper CORS policies. Use React Helmet to set security headers. Regularly audit for vulnerable third-party components. Implement proper authentication checks.

Hindi:
iframe embedding prevent करने के लिए x-frame-options header implement करें। frame-ancestors directive के साथ content security policy इस्तेमाल करें। frame-busting techniques implement करें। proper cors policies add करें। security headers set करने के लिए react helmet इस्तेमाल करें। vulnerable third-party components के लिए regularly audit करें। proper authentication checks implement करें।

Marathi:
iframe embedding prevent करण्यासाठी x-frame-options header implement करा. frame-ancestors directive सोबत content security policy वापरा. frame-busting techniques implement करा. proper cors policies add करा. security headers set करण्यासाठी react helmet वापरा. vulnerable third-party components साठी regularly audit करा. proper authentication checks implement करा.

Analogy:
Like anti-theft devices on cars: clickjacking prevention adds multiple locks and alarms to prevent

SKILLHUB

malicious "car thieves" from embedding your app in invisible frames.

Learn more:

- https://owasp.org/www-community/attacks/Clickjacking

✅ 271. How do you implement CSRF protection in React applications?

English:
Use CSRF tokens generated by server, include in requests. Store token in secure httpOnly cookie. Send token in request headers or body. Validate on server-side. Use libraries like csurf for Express servers. Implement sameSite cookie attribute. Refresh tokens periodically. Handle token expiration gracefully.

Hindi:
server द्वारा generated csrf tokens इस्तेमाल करें, requests में include करें। token को secure httponly cookie में store करें। request headers या body में token send करें। server-side पर validate करें। express servers के लिए csurf जैसे libraries इस्तेमाल करें। samesite cookie attribute implement करें। periodically tokens refresh करें। token expiration को gracefully handle करें।

Marathi:
server द्वारे generated csrf tokens वापरा, requests मध्ये include करा. token ला secure httponly cookie मध्ये store करा. request headers किंवा body मध्ये token send करा. server-side वर validate करा. express servers साठी csurf सारखी libraries वापरा. samesite cookie attribute implement करा. periodically tokens refresh करा. token expiration ला gracefully handle करा.

Analogy:
Like a secret handshake: CSRF tokens ensure that requests to your server include a special "secret code" that only legitimate users (not malicious scripts) know.

Learn more:

- https://owasp.org/www-community/attacks/csrf

✅ 272. What are the best practices for handling JWT tokens in React applications?

English:
Store tokens in memory or secure httpOnly cookies, never localStorage. Implement automatic token refresh before expiration. Validate tokens on each request. Handle token expiration gracefully. Implement logout on invalid tokens. Use refresh tokens for long sessions. Encrypt sensitive data. Implement proper error handling for authentication failures.

SKILLHUB

Hindi:
tokens को memory या secure httponly cookies में store करें, कभी localstorage में नहीं। expiration से पहले automatic token refresh implement करें। हर request पर tokens validate करें। token expiration को gracefully handle करें। invalid tokens पर logout implement करें। long sessions के लिए refresh tokens इस्तेमाल करें। sensitive data को encrypt करें। authentication failures के लिए proper error handling implement करें।

Marathi:
tokens ला memory किंवा secure httponly cookies मध्ये store करा, कधीही localstorage मध्ये नाही. expiration पूर्वी automatic token refresh implement करा. प्रत्येक request वर tokens validate करा. token expiration ला gracefully handle करा. invalid tokens वर logout implement करा. long sessions साठी refresh tokens वापरा. sensitive data encrypt करा. authentication failures साठी proper error handling implement करा.

Analogy:
Like handling a VIP access card: JWT tokens need careful storage (secure cookies), regular renewal (refresh), and immediate revocation when compromised (logout).

Learn more:

- https://tools.ietf.org/html/rfc7519

✅ 273. How do you secure API communications in React applications?

English:
Use HTTPS for all communications. Implement proper CORS policies. Validate all inputs on server-side. Use API keys or tokens for authentication. Implement rate limiting. Use request signing for sensitive operations. Handle errors securely without exposing sensitive information. Implement proper logging without logging sensitive data.

Hindi:
सभी communications के लिए https इस्तेमाल करें। proper cors policies implement करें। server-side पर सभी inputs validate करें। authentication के लिए api keys या tokens इस्तेमाल करें। rate limiting implement करें। sensitive operations के लिए request signing इस्तेमाल करें। sensitive information expose किए बिना errors को securely handle करें। sensitive data logging किए बिना proper logging implement करें।

Marathi:
सर्व communications साठी https वापरा. proper cors policies implement करा. server-side वर सर्व inputs validate करा. authentication साठी api keys किंवा tokens वापरा. rate limiting implement करा. sensitive operations साठी request signing वापरा. sensitive information expose न करता errors ला securely handle करा. sensitive data logging न करता proper logging implement करा.

SKILLHUB

Analogy:
Like securing a bank vault: API communications need multiple locks (HTTPS, authentication, validation) and careful monitoring to prevent unauthorized access.

Learn more:

- https://owasp.org/www-project-api-security/

✅ 274. How do you implement secure routing and protected routes in React?

English:
Create higher-order component or custom hook for authentication checks. Use React Router's render prop or hooks for conditional rendering. Store authentication state securely. Implement route guards. Handle authentication redirects. Use lazy loading for protected components. Implement proper loading states. Handle authentication errors gracefully.

Hindi:
authentication checks के लिए higher-order component या custom hook create करें। conditional rendering के लिए react router के render prop या hooks इस्तेमाल करें। authentication state को securely store करें। route guards implement करें। authentication redirects handle करें। protected components के लिए lazy loading इस्तेमाल करें। proper loading states implement करें। authentication errors को gracefully handle करें।

Marathi:
authentication checks साठी higher-order component किंवा custom hook create करा. conditional rendering साठी react router चे render prop किंवा hooks वापरा. authentication state ला securely store करा. route guards implement करा. authentication redirects handle करा. protected components साठी lazy loading वापरा. proper loading states implement करा. authentication errors ला gracefully handle करा.

Analogy:
Like a nightclub with a bouncer: secure routing acts as a security guard, checking authentication "IDs" before allowing access to protected "VIP areas" (routes) of your application.

Learn more:

- https://reactrouter.com/web/example/auth-workflow

✅ 275. What security considerations should you keep in mind when using third-party React libraries?

SKILLHUB

English:
Audit library source code and dependencies. Check for regular security updates. Verify library maintainers. Use libraries with good security track record. Be cautious with libraries requiring excessive permissions. Implement proper input validation even with libraries. Keep dependencies updated. Monitor for security advisories.

Hindi:
library source code और dependencies को audit करें। regular security updates check करें। library maintainers को verify करें। good security track record वाले libraries इस्तेमाल करें। excessive permissions requiring libraries के साथ cautious रहें। libraries के साथ भी proper input validation implement करें। dependencies को updated रखें। security advisories के लिए monitor करें।

Marathi:
library source code आणि dependencies audit करा. regular security updates check करा. library maintainers verify करा. good security track record असलेली libraries वापरा. excessive permissions requiring libraries सोबत cautious रहा. libraries सोबतही proper input validation implement करा. dependencies updated ठेवा. security advisories साठी monitor करा.

Analogy:
Like checking references for a new employee: third-party libraries need thorough background checks to ensure they won't introduce security vulnerabilities to your application.

Learn more:

- https://snyk.io/

✅ 276. How do you implement component composition for large-scale React applications?

English:
Use compound components pattern with context. Create component families that work together. Use render props for flexible APIs. Implement container/presentational component separation. Use hooks for shared logic. Create reusable component libraries. Implement proper TypeScript interfaces. Use composition over inheritance.

Hindi:
context के साथ compound components pattern इस्तेमाल करें। component families create करें जो साथ work करते हैं। flexible apis के लिए render props इस्तेमाल करें। container/presentational component separation implement करें। shared logic के लिए hooks इस्तेमाल करें। reusable component libraries create करें। proper typescript interfaces implement करें। inheritance के बजाय composition इस्तेमाल करें।

Marathi:
context सोबत compound components pattern वापरा. component families create करा ज्या एकत्र work

SKILLHUB

करतात. flexible apis साठी render props वापरा. container/presentational component separation implement करा. shared logic साठी hooks वापरा. reusable component libraries create करा. proper typescript interfaces implement करा. inheritance ऐवजी composition वापरा.

Analogy:
Like building with LEGO: component composition creates flexible building blocks that can be combined in endless ways to construct large, maintainable applications.

Learn more:

- https://kentcdodds.com/blog/advanced-react-component-composition

---

✅ 277. What are the different state management solutions for large-scale React applications?

English:
Redux Toolkit for predictable state with middleware. Zustand for lightweight state management. Recoil for React-specific state. Context API for simple shared state. MobX for reactive state. React Query for server state. Choose based on app complexity, team experience, and performance requirements.

Hindi:
middleware के साथ predictable state के लिए redux toolkit। lightweight state management के लिए zustand। react-specific state के लिए recoil। simple shared state के लिए context api। reactive state के लिए mobx। server state के लिए react query। app complexity, team experience, और performance requirements के आधार पर choose करें।

Marathi:
middleware सोबत predictable state साठी redux toolkit. lightweight state management साठी zustand. react-specific state साठी recoil. simple shared state साठी context api. reactive state साठी mobx. server state साठी react query. app complexity, team experience आणि performance requirements वर आधारित choose करा.

Analogy:
Like choosing transportation: Redux Toolkit is a full-sized bus for complex journeys, Zustand is a nimble scooter for quick trips, Context API is walking for short distances.

Learn more:

- https://redux-toolkit.js.org/

SKILLHUB

✅ 278. How do you implement micro-frontends architecture with React?

English:
Use module federation in webpack 5 for runtime sharing. Implement different frameworks in different parts. Use single-spa for orchestration. Implement shared routing. Use shared state management across micro-frontends. Implement proper error boundaries. Use design system for consistency. Handle cross-app communication carefully.

Hindi:
runtime sharing के लिए webpack 5 में module federation इस्तेमाल करें। different parts में different frameworks implement करें। orchestration के लिए single-spa इस्तेमाल करें। shared routing implement करें। micro-frontends के across shared state management इस्तेमाल करें। proper error boundaries implement करें। consistency के लिए design system इस्तेमाल करें। cross-app communication को carefully handle करें।

Marathi:
runtime sharing साठी webpack 5 मध्ये module federation वापरा. different parts मध्ये different frameworks implement करा. orchestration साठी single-spa वापरा. shared routing implement करा. micro-frontends च्या across shared state management वापरा. proper error boundaries implement करा. consistency साठी design system वापरा. cross-app communication carefully handle करा.

Analogy:
Like a shopping mall with independent stores: micro-frontends allow different teams to build and deploy their "stores" independently while maintaining a cohesive shopping experience.

Learn more:

- https://webpack.js.org/concepts/module-federation/

✅ 279. What strategies can you use for code organization in enterprise React applications?

English:
Use feature-based folder structure. Implement barrel exports (index.js). Create shared component libraries. Use absolute imports. Implement proper TypeScript usage. Create custom hooks for business logic. Use utility functions. Implement proper testing structure. Use code generators for boilerplate.

Hindi:
feature-based folder structure इस्तेमाल करें। barrel exports (index.js) implement करें। shared component libraries create करें। absolute imports इस्तेमाल करें। proper typescript usage implement करें। business

SKILLHUB

logic के लिए custom hooks create करें। utility functions इस्तेमाल करें। proper testing structure implement करें। boilerplate के लिए code generators इस्तेमाल करें।

Marathi:
feature-based folder structure वापरा. barrel exports (index.js) implement करा. shared component libraries create करा. absolute imports वापरा. proper typescript usage implement करा. business logic साठी custom hooks create करा. utility functions वापरा. proper testing structure implement करा. boilerplate साठी code generators वापरा.

Analogy:
Like organizing a large library: proper code organization creates clear sections (features), easy navigation (barrel exports), and efficient retrieval (absolute imports) for large codebases.

Learn more:

- https://reactjs.org/docs/faq-structure.html

✅ 280. How do you implement testing strategies for large-scale React applications?

English:
Use Jest for unit testing, React Testing Library for component testing. Implement integration tests with Cypress. Use test-driven development. Implement visual regression testing. Use mocking for external dependencies. Implement proper CI/CD pipeline. Maintain high test coverage. Use snapshot testing for UI components.

Hindi:
unit testing के लिए jest इस्तेमाल करें, component testing के लिए react testing library। cypress के साथ integration tests implement करें। test-driven development इस्तेमाल करें। visual regression testing implement करें। external dependencies के लिए mocking इस्तेमाल करें। proper ci/cd pipeline implement करें। high test coverage maintain करें। ui components के लिए snapshot testing इस्तेमाल करें।

Marathi:
unit testing साठी jest वापरा, component testing साठी react testing library. cypress सोबत integration tests implement करा. test-driven development वापरा. visual regression testing implement करा. external dependencies साठी mocking वापरा. proper ci/cd pipeline implement करा. high test coverage maintain करा. ui components साठी snapshot testing वापरा.

Analogy:
Like quality control in a factory: comprehensive testing ensures each component works correctly, integration tests verify they work together, and automated pipelines catch issues before they reach production.

Learn more:

SKILLHUB

- https://testing-library.com/docs/react-testing-library/intro/

✅ 281. How do you implement CI/CD pipelines for React applications?

English:
Use GitHub Actions, GitLab CI, or Jenkins. Implement automated testing on each commit. Use linting and type checking. Implement automated deployment to staging/production. Use Docker for containerization. Implement proper environment management. Use feature flags for gradual rollouts. Monitor deployment health.

Hindi:
github actions, gitlab ci, या jenkins इस्तेमाल करें। हर commit पर automated testing implement करें। linting और type checking इस्तेमाल करें। staging/production में automated deployment implement करें। containerization के लिए docker इस्तेमाल करें। proper environment management implement करें। gradual rollouts के लिए feature flags इस्तेमाल करें। deployment health monitor करें।

Marathi:
github actions, gitlab ci किंवा jenkins वापरा. प्रत्येक commit वर automated testing implement करा. linting आणि type checking वापरा. staging/production मध्ये automated deployment implement करा. containerization साठी docker वापरा. proper environment management implement करा. gradual rollouts साठी feature flags वापरा. deployment health monitor करा.

Analogy:
Like an automated assembly line: CI/CD pipelines automatically test, build, and deploy your React application, ensuring consistent quality from development to production.

Learn more:

- https://github.com/features/actions

✅ 282. How do you handle error boundaries and crash reporting in production React apps?

English:
Implement error boundaries at different levels (page, component, feature). Use services like Sentry or LogRocket for error reporting. Implement graceful error fallbacks. Log errors with context. Implement retry mechanisms. Use error monitoring dashboards. Handle async errors with error boundaries for Suspense.

SKILLHUB

Hindi:
different levels (page, component, feature) पर error boundaries implement करें। error reporting के लिए sentry या logrocket जैसे services इस्तेमाल करें। graceful error fallbacks implement करें। context के साथ errors log करें। retry mechanisms implement करें। error monitoring dashboards इस्तेमाल करें। suspense के लिए async errors को error boundaries के साथ handle करें।

Marathi:
different levels (page, component, feature) वर error boundaries implement करा. error reporting साठी sentry किंवा logrocket सारखी services वापरा. graceful error fallbacks implement करा. context सोबत errors log करा. retry mechanisms implement करा. error monitoring dashboards वापरा. suspense साठी async errors ला error boundaries सोबत handle करा.

Analogy:
Like having emergency services on standby: error boundaries and crash reporting act as your application's emergency response team, catching crashes and reporting issues for quick resolution.

Learn more:

- https://sentry.io/for/react/

✅ 283. How do you implement A/B testing in React applications?

English:
Use libraries like react-ab-test or LaunchDarkly. Create feature flags for different variants. Randomly assign users to groups. Track metrics and conversions. Use server-side or client-side implementation. Implement proper analytics tracking. Handle flash of incorrect variant (FOIV). Use statistical significance testing.

Hindi:
react-ab-test या launchdarkly जैसे libraries इस्तेमाल करें। different variants के लिए feature flags create करें। randomly users को groups में assign करें। metrics और conversions track करें। server-side या client-side implementation इस्तेमाल करें। proper analytics tracking implement करें। flash of incorrect variant (foiv) handle करें। statistical significance testing इस्तेमाल करें।

Marathi:
react-ab-test किंवा launchdarkly सारखी libraries वापरा. different variants साठी feature flags create करा. randomly users ला groups मध्ये assign करा. metrics आणि conversions track करा. server-side किंवा client-side implementation वापरा. proper analytics tracking implement करा. flash of incorrect variant (foiv) handle करा. statistical significance testing वापरा.

Analogy:
Like testing two recipes: A/B testing serves different versions of your app to users, measuring which

SKILLHUB

"recipe" performs better based on user behavior and preferences.

Learn more:

- https://launchdarkly.com/

✅ 284. How do you implement internationalization (i18n) at scale in React?

English:
Use react-i18next or react-intl libraries. Create translation files for each language. Implement language detection. Use interpolation for dynamic content. Handle pluralization and date/number formatting. Implement lazy loading for translation files. Use context for language switching. Handle right-to-left languages.

Hindi:
react-i18next या react-intl libraries इस्तेमाल करें। हर language के लिए translation files create करें। language detection implement करें। dynamic content के लिए interpolation इस्तेमाल करें। pluralization और date/number formatting handle करें। translation files के लिए lazy loading implement करें। language switching के लिए context इस्तेमाल करें। right-to-left languages handle करें।

Marathi:
react-i18next किंवा react-intl libraries वापरा. प्रत्येक language साठी translation files create करा. language detection implement करा. dynamic content साठी interpolation वापरा. pluralization आणि date/number formatting handle करा. translation files साठी lazy loading implement करा. language switching साठी context वापरा. right-to-left languages handle करा.

Analogy:
Like a universal remote control: i18n allows your React app to speak multiple languages, adapting content and formatting based on the user's language preferences.

Learn more:

- https://react.i18next.com/

✅ 285. What are the monitoring, logging, and analytics strategies for production React applications?

English:
Implement application performance monitoring (APM) with tools like New Relic or DataDog. Use error tracking services like Sentry. Implement logging with structured data. Track user interactions with

SKILLHUB

analytics platforms. Monitor Core Web Vitals. Implement real user monitoring (RUM). Use log aggregation services. Set up alerts for critical issues.

Hindi:
new relic या datadog जैसे tools के साथ application performance monitoring (apm) implement करें। sentry जैसे error tracking services इस्तेमाल करें। structured data के साथ logging implement करें। analytics platforms के साथ user interactions track करें। core web vitals monitor करें। real user monitoring (rum) implement करें। log aggregation services इस्तेमाल करें। critical issues के लिए alerts set up करें।

Marathi:
new relic किंवा datadog सारखे tools सोबत application performance monitoring (apm) implement करा. sentry सारखे error tracking services वापरा. structured data सोबत logging implement करा. analytics platforms सोबत user interactions track करा. core web vitals monitor करा. real user monitoring (rum) implement करा. log aggregation services वापरा. critical issues साठी alerts set up करा.

Analogy:
Like a flight recorder in an airplane: monitoring, logging, and analytics provide the "black box" data that helps you understand what happened during production issues and user experiences.

Learn more:

- https://web.dev/vitals/

SKILLHUB